PrepNPlaced

ZERO TO INTERVIEW READY

DATA ENGINEERING

> *The pipeline ran successfully.*
> *The numbers were wrong.*

EVERY FIGURE MEASURED · EVERY QUERY EXECUTED



LINUX · SQL · PYTHON · DATA MODELING · PIPELINES · SPARK
KAFKA · AIRFLOW · DATA QUALITY · SYSTEM DESIGN · THE INTERVIEW LOOP

Durgesh Yadav

Senior Data Engineer · Educator · Mentor · Founder, PrepNPlaced

Data Engineering

From Zero to Production & Interview Ready

How data actually moves inside a product company — Linux, SQL, Python, data modeling, pipelines, Spark, Kafka, Airflow, data quality, system design, and the interview loop.

Durgesh Yadav

Senior Data Engineer · Data Engineering Educator · Mentor
Founder, PrepNPlaced

Data Engineering: From Zero to Production & Interview Ready

First edition. Copyright © Durgesh Yadav. All rights reserved.

No part of this book may be reproduced or transmitted in any form or by any means without the prior written permission of the author, except for brief quotations in reviews and other non-commercial uses permitted by copyright law.

This is an original educational work. Where it describes third-party technologies, those descriptions are the author's own explanation of publicly documented behaviour. Product names, logos and brand marks, including those shown on the cover, are the property of their respective owners and are used here only to identify the technologies the book covers. Their use does not imply any endorsement or affiliation.

The technologies in this book change quickly. Version numbers, service limits, pricing and default settings move faster than a printed page. Treat this book as the map and the official documentation as the compass.

Code in this book is written for clarity rather than for direct production use. Test it, adapt it, and add the error handling, security review and monitoring your own environment requires. The author accepts no liability for loss arising from its use.

Several exercises create cloud infrastructure. You are responsible for the charges you incur. Shut resources down when you are finished.

ShopKart is a fictional company. Its figures are illustrative and are not attributed to any real business. Where real companies are named, the claims carry a source reference at the end of that chapter.

Preface

Most people meet data engineering backwards. They meet a tool first. Somebody says Spark is important, so they learn Spark syntax. Then Kafka. Then Airflow. Six months later they can write `spark.read.parquet(...)` from memory and they still cannot answer the question every interviewer eventually asks: *walk me through what happens between a customer tapping Place Order and that order appearing on a dashboard.*

That question is the job. The tools are the parts you bolt together to answer it.

So this book runs the other way round. It starts with a company — a fictional Indian e-commerce business called ShopKart — and follows a single order through every system it touches. Each technology arrives at the moment the story needs it. Kafka appears when the order database starts buckling under six teams querying it. Spark appears when a Python script that used to take ten minutes starts taking nine hours. Airflow appears when there are forty scripts and nobody remembers which runs first.

A tool you meet as the answer to a problem you already have is a tool you remember. A tool you meet as a bullet point on a syllabus is one you forget by Friday.

What this book assumes. Almost nothing. If you have finished Class 12 and can use a computer, you can read this. You do not need a computer science degree. Lecture Zero and Chapters 2 to 4 build the foundations from scratch, in the specific way a data engineer needs them.

What it refuses to do. Be shallow because you are a beginner. Being a beginner means you need simpler explanations, not simpler engineering. So when we reach Spark we do not stop at "it's fast because memory". We read the actual execution plan, find the word `Exchange`, and understand why one slow task holds up a job that has already finished 199 of its 200 pieces.

A note on the numbers. Almost every figure in this book was measured rather than asserted. The 44.9% revenue inflation from a join fan-out, the 34.8× size reduction from Parquet, the tripling revenue from a non-idempotent load, the Spark execution plans, the

₹4,749 that moved between two cities when one customer changed address — all of these came from code that was executed while the book was being written, and the scripts are in the companion Field Kit so you can reproduce and challenge them. Where something could not be executed, the book says so plainly.

What is not here. No salary promises. No claim that you will be hired in thirty days. Data engineering pays well because it is genuinely difficult, and what this book offers is a straight and complete path through that difficulty.

You will get more from this book if you argue with it. When you read that a wide transformation forces a shuffle, do not underline it. Open a terminal, run something that shuffles, and check whether the picture in your head matches what the engine actually did. The gap between those two things is where an engineer is made.

Durgesh Yadav

Who This Book Is For

The non-CS graduate or Class 12 student. You have heard data pays well and you cannot find the entry point. Start at Lecture Zero and skip nothing.

The beginner programmer. You have written a few scripts. Skim Lecture Zero, then slow down at Chapter 3. SQL is the single highest-return skill here, and the level that gets you hired is well past where most tutorials stop.

The analyst moving into engineering. Your SQL is probably fine. Your gap is everything around it — pipelines, orchestration, distributed processing, and the shift from consuming tables to guaranteeing them. Read Chapters 1 and 5 for the mental model, then go hard from Chapter 8. Chapter 12 is the one that changes how interviewers see you.

The software engineer moving across. Chapter 2 will be revision. Your real gaps are usually data modeling and the fact that data systems fail *silently* in a way application systems do not. Do not skip Chapter 5 because it looks basic; it is the chapter software engineers most often get wrong.

The data engineer preparing for interviews. Chapter 14 has the loop and the framework, but the reasoning lives in the chapters the questions come from. Memorising “use salting” survives exactly one follow-up.

The working engineer patching fundamentals. The “what happens behind the scenes” sections and the measured results are written for you.

How to Use This Book

Read it in order, at least once. The book builds one continuous example and one continuous vocabulary. Chapter 9 assumes what a partition meant in Chapter 6, which assumed what a file format meant in Chapter 4.

Type the code. All of it, by hand. Copy-pasting builds nothing. When something breaks, read the error before you search for it.

The recurring boxes each do a specific job. *Think of it this way* gives you somewhere to hang an idea before the technical version arrives. *What happens behind the scenes* is the internal mechanics, and it is where senior-level interview answers come from. *Common beginner mistakes* costs you two minutes to read and saves you a weekend. *In the interview* is answered in layers — a short answer you could say out loud, a deeper explanation, the follow-up a good interviewer asks, and the senior-level version. *Practice* is not optional; reading about SQL produces the feeling of knowing SQL, which is a different thing.

No solutions are printed for the practice tasks. That is deliberate. The ability to sit with something that does not work yet is the skill.

A realistic pace. Two to four months at ten hours a week. The thirty-day plan in Chapter 14 is a revision schedule for someone who has already read the book, not a first pass. Anyone telling you this field can be learned from scratch in thirty days is selling something.

Table of Contents

Preface 3

Who This Book Is For 5

How to Use This Book 6

FOUNDATIONS

0. Living in the Terminal 14

0.1 Getting a Linux to practise on 14
0.2 Where am I, and how do I move 15
0.3 Look at a file before you load it 16
0.4 The pipe, which is the best idea in Unix 17
0.5 Finding things 18
0.6 Arithmetic on a file, without Python 18
0.7 Redirection and exit codes 19
0.8 Permissions 20
0.9 Environment variables 20
0.10 What is this machine doing? 21
0.11 Moving and packing files 21
0.12 Editing a file on a server 22
0.13 Scheduling with cron 22
0.14 Putting it together 23
The commands worth keeping 23

1. What a Data Engineer Actually Does 26

1.1 The question the whole job is built around 26
1.2 What data is, precisely enough to be useful 28
1.3 Why the application's own database cannot answer the business's questions 29
1.4 OLTP and OLAP: a physical difference, not a naming convention 30
1.5 The two clocks 32
1.6 The map 33
1.7 The work itself 35
1.8 The neighbouring jobs 36
1.9 The failure mode that defines the discipline 37
Summary 38
In the interview 38

2. The Machine Underneath 41

2.1 Four resources, one question ... 41
2.2 CPU: cores are the unit of parallelism ... 42
2.3 Memory: fast, small and forgetful ... 42
2.4 Disk: sequential is cheap, random is not ... 43
2.5 Network: the bottleneck nobody budgets for ... 45
2.6 Processes, threads, and why jobs vanish ... 45
2.7 Files and processes on a real machine ... 46
2.8 Servers, ports and the four connection errors ... 47
2.9 HTTP, JSON and the shape of an ingestion loop ... 47
2.10 Credentials ... 49
Summary ... 50
In the interview ... 51

THE TWO LANGUAGES

3. SQL to Interview Level ... 54
3.1 The dataset ... 55
3.2 Five queries to get your hands moving ... 56
3.3 How a SELECT actually runs ... 58
3.4 NULL , and the trap that eats interview candidates ... 60
3.5 Aggregation, and the word that matters most ... 61
3.6 Joins, and the bug from the first page ... 62
3.7 CTEs ... 65
3.8 Window functions ... 65
3.9 The patterns interviewers actually ask ... 67
3.10 Making a query fast ... 73
3.11 Mistakes to stop making now ... 74
Summary ... 74

4. Python for Data Engineers ... 76
4.1 What Python is actually for here ... 76
4.2 The four types that carry everything ... 77
4.3 Reading a file without breaking it ... 78
4.4 JSON and nested data ... 80
4.5 Functions, and the shape of a script ... 81
4.6 Errors: catch what you expect, keep going, exit loudly ... 82
4.7 Logging, not printing ... 83
4.8 pandas, and where it stops ... 84
4.9 Talking to a database ... 86
4.10 Virtual environments ... 87
4.11 Putting it together ... 87

Summary ... 89
In the interview ... 90

SHAPING DATA

5. Data Modeling ... 93
5.1 Grain: what one row means ... 93
5.2 Entities, attributes, relationships ... 95
5.3 Why operational tables are split up ... 96
5.4 Why analytics puts them back together ... 96
5.5 Facts and dimensions ... 97
5.6 The star schema ... 98
5.7 The snowflake schema ... 100
5.8 Keys ... 100
5.9 Slowly changing dimensions ... 101
5.10 Three kinds of fact table ... 104
5.11 What goes wrong ... 105
Summary ... 105
In the interview ... 106

6. Where Data Lives ... 109
6.1 Rows and columns, physically ... 109
6.2 The formats you will actually meet ... 110
6.3 Compression, measured ... 111
6.4 Column pruning, measured ... 111
6.5 Partitioning ... 112
6.6 The small files problem ... 113
6.7 Object storage is not a filesystem ... 114
6.8 Warehouse, lake, lakehouse ... 115
6.9 Table formats: what Delta and Iceberg actually add ... 116
6.10 The medallion pattern ... 118
Summary ... 118
In the interview ... 119

7. The Cloud, and What You Are Actually Renting ... 121
7.1 What the cloud actually is ... 121
7.2 The six things you rent ... 122
7.3 ShopKart on the cloud, drawn once ... 123
7.4 Object storage, in a little more detail ... 125
7.5 Identity, and why nothing works on your first day ... 126
7.6 What the bill is made of ... 127
7.7 Networking, in the amount you actually need ... 128

7.8 Managed Spark: four ways to run the same job 130
7.9 Infrastructure as code 130
7.10 Serverless, and when it is the wrong answer 131
7.11 A cost model you can actually do in your head 132
7.12 Getting started without a bill 133
Summary 133
In the interview 134

MOVING DATA

8. Pipelines: ETL, ELT and Loading Data Twice 137
8.1 ETL, ELT, and what actually changed 137
8.2 Full load against incremental 139
8.3 Watermarks 139
8.4 Idempotency 140
8.5 MERGE , properly 142
8.6 Change data capture 143
8.7 Late-arriving data 144
8.8 Schema evolution 145
8.9 dbt: the T in ELT, with engineering around it 146
8.10 Backfills 149
Summary 150
In the interview 151

9. Apache Spark 153
9.1 Why Spark exists 153
9.2 The pieces 155
9.3 Partitions 155
9.4 Lazy evaluation 156
9.5 Narrow, wide, and the shuffle 157
9.6 Reading a physical plan 158
9.7 Joins 159
9.8 Data skew 160
9.9 Caching 161
9.10 Memory, and why jobs vanish 162
9.11 PySpark in practice 163
Summary 163
In the interview 164

10. Orchestration with Airflow 166
10.1 The DAG 166
10.2 A real DAG 167

10.3 Idempotency again, from the other side ... 168
10.4 Sensors and the waiting problem ... 169
10.5 Backfill and catchup ... 170
10.6 Passing data between tasks ... 170
10.7 What good looks like in the UI ... 170
10.8 Running it, and watching the gate hold ... 171
Summary ... 174
In the interview ... 174

RUNNING IT

11. Streaming and Kafka ... 177
11.1 The vocabulary ... 177
11.2 Replication ... 180
11.3 Consumer lag ... 180
11.4 Delivery semantics ... 181
11.5 Schemas and poison messages ... 181
11.6 Streaming semantics: event time and windows ... 182
11.7 When not to stream ... 183
Summary ... 184
In the interview ... 185

12. When Pipelines Lie ... 187
12.1 Monitor the data, not the job ... 187
12.2 Warn, fail, or quarantine ... 189
12.3 Data contracts ... 189
12.4 The debugging framework ... 190
12.5 Fixing the data, not only the code ... 191
12.6 SLAs and being on call ... 192
12.7 “Production” is a practice, not a place ... 192
12.8 When the consumer is a model ... 195
Summary ... 196
In the interview ... 197

GETTING HIRED

13. The Portfolio Project ... 200
13.1 The whole system, before any code ... 200
13.2 Setting up, from an empty folder ... 202
13.3 Stage one — ingest ... 204
13.4 Stage two, part one — clean and quarantine ... 206
13.5 Stage two, part two — the model ... 207

13.6 Stage two, part three — the gate ... 209
13.7 Stage three — publish ... 210
13.8 Stage four — break it on purpose ... 210
13.9 Stage five — orchestrate it ... 212
13.10 The README that does the work ... 213
13.11 When it goes wrong ... 214
13.12 Levelling it up ... 214
13.13 Two more projects, and what each one proves ... 215
Summary ... 219

14. The Interview Loop ... 221
14.1 Designing a system in an interview ... 221
14.2 Three designs, worked ... 223
14.3 The loop ... 228
14.4 Talking about your work ... 229
14.5 Thirty days of revision ... 229
Summary ... 230

Glossary ... 231

The Field Kit ... 239

Further Reading ... 240

About the Author ... 241

Index ... 242

PART ONE

Foundations

Before any tool: the terminal you will live in, the job you are actually applying for, and the four physical resources every slow pipeline is waiting on.

Lecture Zero · Living in the Terminal
1 · What a Data Engineer Actually Does
2 · The Machine Underneath

0

LECTURE ZERO

Living in the Terminal

This is Lecture Zero because it comes before everything and because nobody teaches it. Every job in this book runs on Linux. Every cluster you will ever log into, every server that holds your data, every place your pipeline actually executes — Linux, with no mouse. People who skip this spend their first six months slightly afraid of a black window, and it slows down everything else they learn.

You do not need to become a system administrator. You need about thirty commands, and you need to stop being nervous. That is the whole goal of this lecture, and it takes an afternoon.

Everything here is shown with the actual output. Type the commands. Reading them does nothing.

0.1 Getting a Linux to practise on

Pick whichever is easiest for you and move on. This is not a decision worth an hour.

You have	Do this
Windows	Install WSL: open PowerShell and run `wsl --install`. You get a real Ubuntu inside Windows
Mac	Open Terminal. macOS is Unix already; nearly everything here works unchanged
Neither, or nothing installs	Use a free browser terminal from any cloud provider, or a Docker container

Once you have a prompt, you are ready. It will look something like this, and the `$` is the prompt, not something you type:

```
durgesh@laptop:~$
```

Throughout this lecture we work in a folder with one CSV of orders, a folder of raw files, and a log file:

```
$ ls -lh
total 12K
drwxr-xr-x 2 root root 4.0K Sep  2 14:08 logs
-rw-r--r-- 1 root root  508 Sep  2 14:08 orders.csv
drwxr-xr-x 3 root root 4.0K Sep  2 14:08 raw
```

Read that output once, because it repays it. The first character says what the thing is: `d` for directory, `-` for a regular file. The next nine characters are permissions, which we come back to in 0.8. Then the owner, the group, the size, the last-modified time, and the name.

0.2 Where am I, and how do I move

Three commands. That is the entire navigation system.

```
$ pwd                 # print working directory — where am I right now
/home/durgesh/shopkart

$ ls                  # what is in here
logs  orders.csv  raw

$ cd raw              # go into the raw folder
$ cd ..               # go back up one level
$ cd ~                # go to my home folder
$ cd -                # go back to wherever I just was
```

Paths come in two flavours and mixing them up is the most common beginner confusion.

An **absolute path** starts with `/` and works from anywhere: `/home/durgesh/shopkart/orders.csv`. A **relative path** starts from where you currently are: `raw/2026-03-18/part-0000.csv`.

Four shorthands appear everywhere:

Symbol	Means
`.`	the current directory
`..`	the directory above
`~`	your home directory
`/`	the very top of the tree

THE HABIT THAT PREVENTS MOST CONFUSION

When a command fails with “no such file”, run `pwd` and `ls` before anything else. Nine times out of ten you are simply not where you thought you were.

Use the **Tab key** constantly. Type `cd ord` and press Tab, and the shell completes the name for you. This is not a nicety — it prevents typos and it tells you immediately whether the file exists.

0.3 Look at a file before you load it

This is the single most useful habit in this lecture. Before any file goes near Python or Spark, look at it.

```
$ head -3 orders.csv
order_id,customer_id,order_date,city,status,order_total
9001,101,2026-03-01,Pune,DELIVERED,4749.00
9002,102,2026-03-01,Bengaluru,DELIVERED,1250.00
```

`head` shows the first lines. Straight away you know there is a header row, the delimiter is a comma, and dates are in `YYYY-MM-DD`. Three facts that decide how you write the loader.

```
$ wc -l orders.csv
11 orders.csv
```

Eleven lines, so ten data rows plus a header. `wc -l` is your fastest sanity check and you should run it every time. If yesterday’s file had 40,000 rows and today’s has 11, you have found the incident before the dashboard did.

Want the column names numbered, because column 5 is about to matter?

```
$ head -1 orders.csv | tr ',' '\n' | nl
     1  order_id
     2  customer_id
     3  order_date
     4  city
     5  status
     6  order_total
```

Other members of the family: `tail -5 file` for the last five lines, `tail -f file` to watch a log as it is being written, and `less file` to scroll through something large (press `q` to quit). Never run `cat` on a large file over a remote connection.

0.4 The pipe, which is the best idea in Unix

The vertical bar `|` takes the output of one command and feeds it into the next. Small tools, chained. This one idea replaces a surprising amount of Python.

Suppose you want the count of orders by status. In a notebook that is pandas, an import and a few lines. Here it is one line:

```
$ cut -d',' -f5 orders.csv | tail -n +2 | sort | uniq -c | sort -rn
      8 DELIVERED
      1 RETURNED
      1 CANCELLED
```

Read it left to right, because that is exactly how it runs:



FIGURE 0.1 A pipeline of small tools. Each stage takes text in and passes text out.

- `cut -d',' -f5` takes field 5, using a comma as the delimiter
- `tail -n +2` starts from line 2, throwing away the header
- `sort` puts identical values next to each other
- `uniq -c` collapses runs of identical lines and counts them
- `sort -rn` sorts numerically, largest first

`uniq` only collapses *adjacent* duplicates, which is why `sort` must come first. Forgetting that is the classic first mistake with this pipeline, and the symptom is counts that look far too small.

Another simple one — which cities do delivered orders come from?

```
$ grep DELIVERED orders.csv | cut -d',' -f4 | sort | uniq -c
      4 Bengaluru
      1 Hyderabad
      3 Pune
```

0.5 Finding things

`grep` searches inside files. `find` searches for files.

```
$ grep ERROR logs/ingest.log
2026-03-18 02:00:19 ERROR connection reset by peer
2026-03-18 02:00:25 ERROR checksum mismatch on part-0001.csv

$ grep -c ERROR logs/ingest.log       # just count them
2
```

Three flags carry most of the weight: `-i` ignores case, `-c` counts instead of printing, and `-v` inverts the match so you see everything that does *not* contain the pattern. That last one is how you filter noise out of a log.

```
$ find . -name "*.csv" -type f
./orders.csv
./raw/2026-03-18/part-0001.csv
./raw/2026-03-18/part-0000.csv
```

`find` is how you answer "how many files did that job actually write", which turns out to matter enormously once you reach Chapter 6. Add `| wc -l` to the end and you have the count.

0.6 Arithmetic on a file, without Python

`awk` processes a file line by line and can do arithmetic. It has a reputation for being cryptic, which it earns eventually, but the simple form is genuinely simple.

```
$ awk -F',' 'NR>1 && $5=="DELIVERED" {s+=$6} END {printf "%.2f\n", s}'
orders.csv
24443.00
```

Piece by piece: `-F','` sets the field separator. `NR>1` means skip line 1, the header. `$5=="DELIVERED"` is the filter. `{s+=$6}` adds field 6 to a running total. `END {...}` runs once at the end and prints it.

Grouping is barely harder:

```
$ awk -F',' 'NR>1 && $5=="DELIVERED" {rev[$4]+=$6; cnt[$4]++}
             END {for (c in rev) printf "%-12s %2d %10.2f\n", c, cnt[c],
rev[c]}' orders.csv
```

```
Bengaluru     4   10446.00
Pune          3   10199.00
Hyderabad     1    3798.00
```

That is a `GROUP BY` on a 40 GB file that never has to fit in memory. You will not write production pipelines in `awk`, but for "is this file roughly right before I spend twenty minutes loading it", nothing beats it.

0.7 Redirection and exit codes

`>` sends output to a file, replacing it. `>>` appends. `2>` sends error messages rather than normal output.

```
$ grep ERROR logs/ingest.log > errors.txt     # write errors to a file
$ wc -l errors.txt
2 errors.txt
```

`/dev/null` is a special file that discards anything written to it. `command > /dev/null` means "run this, I do not want to see the output, I only care whether it worked."

Which brings us to the thing every scheduler in the world depends on. Every command returns an **exit code**: zero means success, anything else means failure. `$?` holds the code from the last command.

```
$ grep "ERROR" logs/ingest.log > /dev/null
$ echo "exit code: $?"
exit code: 0

$ grep "FATAL" logs/ingest.log > /dev/null
$ echo "exit code: $?"
exit code: 1
```

WHY THIS MATTERS MORE THAN IT LOOKS

Airflow, cron, Kubernetes and every CI system decide whether your job succeeded by reading its exit code. A Python script that catches an exception, prints a message and exits normally reports **success** to the scheduler while having done nothing. This is one of the most common ways a broken pipeline goes unnoticed for days.

0.8 Permissions

Back to that first `ls -l` output. The nine characters after the file type are three groups of three:

```
-rwxr-xr-x  1 root root  219 Sep  2 14:11 check.sh
 ├┘├┘├┘├┘
 │ │ │ └── others:  r-x  read and execute
 │ │ └──── group:   r-x  read and execute
 │ └────── owner:   rwx  read, write, execute
 └──────── type:    -    regular file
```

`r` is read, `w` is write, `x` is execute. For a directory, `x` means "may enter it".

You will meet the numeric form because everyone uses it. Read is 4, write is 2, execute is 1, and you add them up per group:

Number	Means	Typical use
`644`	owner read/write, others read	data files, config
`755`	owner all, others read/execute	scripts and directories
`600`	owner read/write, nobody else	credential files
`777`	everyone everything	**almost always wrong**

```
$ ls -l run.sh
-rw-r--r-- 1 root root 0 Sep  2 14:08 run.sh
$ chmod +x run.sh          # make it executable
$ ls -l run.sh
-rwxr-xr-x 1 root root 0 Sep  2 14:08 run.sh
```

When a script fails with `Permission denied`, this is nearly always why. The reflex fix people reach for is `chmod 777`, and it is the wrong reflex: it makes a file world-writable, which on a shared server is a genuine security problem. `chmod +x` is what you actually wanted.

0.9 Environment variables

An environment variable is a named value handed to a program when it starts. This is how credentials, connection strings and settings reach your code without being written into it.

```
$ export SHOPKART_ENV=staging
$ echo "env is $SHOPKART_ENV"
env is staging
```

`export` makes the variable visible to programs you launch afterwards. Without `export`, it exists only in your current shell and your Python script will not see it.

Two you will meet constantly: `$HOME` is your home directory, and `$PATH` is the list of directories the shell searches for commands. When something says `command not found` even though you just installed it, `$PATH` is the reason.

Variables set with `export` vanish when you close the terminal. To make them permanent, put the line in `~/.bashrc`. For real credentials, use a `.env` file that is listed in `.gitignore`, or a secret manager. Never a file that gets committed.

0.10 What is this machine doing?

Four commands answer nearly every “why is it slow” or “why did it die” question.

```
$ df -h        # free disk space, human-readable
$ du -sh *     # how big is each thing in this directory
$ free -h      # free memory
$ top          # live view of CPU and memory by process (press q to quit)
```

`df -h` deserves special mention. A remarkable share of mysterious pipeline failures are simply a full disk, and it takes two seconds to rule out. Check it first, always.

To stop a runaway process, find it in `top`, note the process id, then `kill 12345`. If it ignores you, `kill -9 12345` is the version it cannot ignore.

0.11 Moving and packing files

```
$ scp orders.csv user@server:/data/incoming/     # copy to a remote machine
$ scp user@server:/data/report.csv .             # copy from one, to here

$ tar -czf raw-2026-03-18.tar.gz raw/2026-03-18  # compress a folder
$ tar -xzf raw-2026-03-18.tar.gz                 # extract it again
```

Our two small CSVs compressed to 159 bytes. That compression ratio is not a curiosity: it is the same property that makes columnar file formats so effective, and Chapter 6 comes back to it properly.

For anything large or repeated, `rsync -av source/ destination/` beats `scp`, because it only transfers what changed and can resume after an interruption.

0.12 Editing a file on a server

You will eventually need to change one line of a config on a machine with no graphical editor. Learn `nano`, which is the friendly one: `nano config.yaml`, type, then `Ctrl+O` to save and `Ctrl+X` to exit. The shortcuts are listed along the bottom of the screen.

`vim` is everywhere and worth exactly four commands, so that you can get out of it when it opens by accident: press `i` to start typing, `Esc` to stop, `:wq` to save and quit, and `:q!` to quit and throw away your changes.

0.13 Scheduling with cron

Before Airflow exists in your life, there is `cron`. Run `crontab -e` and add a line with five time fields followed by the command.

```
# ┌ minute   ┌ hour   ┌ day of month   ┌ month   ┌ day of week
# │          │        │                │         │
  0          2        *                *         *   /home/durgesh/daily.sh
```

That runs `daily.sh` at 02:00 every day. A few common patterns:

Schedule	Meaning
`0 * * * *`	every hour, on the hour
`*/15 * * * *`	every fifteen minutes
`0 2 * * *`	daily at 02:00
`0 2 * * 1`	Mondays at 02:00

Cron is fine for one script. It becomes painful the moment you have five scripts that depend on each other, because it has no idea that job B should wait for job A, and no way to re-run yesterday. That gap is precisely why Chapter 10 exists.

0.14 Putting it together

Here is a small script that does something real: check that today's file arrived and has enough rows. It is the ancestor of every data quality check in Chapter 12.

```
#!/bin/bash
FILE=$1

if [ ! -f "$FILE" ]; then
    echo "MISSING: $FILE"
    exit 1
fi

ROWS=$(( $(wc -l < "$FILE") - 1 ))      # subtract the header

if [ "$ROWS" -lt 5 ]; then
    echo "LOW ROW COUNT: $ROWS"
    exit 1
fi

echo "OK: $FILE has $ROWS data rows"
```

```
$ chmod +x check.sh
$ ./check.sh orders.csv
OK: orders.csv has 10 data rows

$ ./check.sh missing.csv
MISSING: missing.csv
$ echo "exit was $?"
exit was 1
```

Nineteen lines, and it already contains three ideas that scale all the way up: check that the input exists, check that it is plausible, and **exit non-zero when it is not** so the scheduler knows. A pipeline that does those three things is ahead of a great many pipelines running in production right now.

The commands worth keeping

Command	Does
`pwd` · `ls -lh` · `cd`	where am I, what is here, move
`head` · `tail -f` · `less`	look at a file, follow a log
`wc -l`	count lines — the fastest sanity check

Command	Does
`cut -d',' -f5`	pull out a column
`sort` · `uniq -c`	group and count
`grep -i` · `grep -c` · `grep -v`	search, count, exclude
`find . -name "*.csv"`	locate files, count them
`awk -F','`	filter and total a column
`>` · `>>` · `2>` · `/dev/null`	redirect output
`$?`	exit code of the last command
`chmod +x` · `chmod 600`	make executable, lock down a secret
`export VAR=value`	set an environment variable
`df -h` · `du -sh *` · `free -h` · `top`	disk, size, memory, processes
`scp` · `rsync -av` · `tar -czf`	copy, sync, compress
`crontab -e`	schedule something

COMMON BEGINNER MISTAKES

- Running `cat` on a huge file over SSH, then waiting.
- Forgetting `sort` before `uniq -c`, and getting counts that are far too low.
- Using `chmod 777` because something said permission denied.
- Setting a variable without `export` and wondering why Python cannot see it.
- Writing a script that catches every error and still exits zero, so the scheduler thinks it worked.
- Not checking `df -h` when a job fails for no visible reason.
- Deleting with `rm -rf` before running `ls` on the same path. There is no recycle bin.

PRACTICE

1. Download any public CSV. Without opening Python, find its row count, its column names numbered, and the distribution of one column.
2. Take a log file — any application on your machine has one — and count how many lines contain the word `error`, case-insensitively.
3. Write the `check.sh` script from 0.14 yourself, run it against a good file and a missing file, and print `$?` after each.

4. Make a folder with a hundred tiny files and one folder with a single file of the same total size. Time `tar -czf` on each. The difference is the small files problem, meeting you early.
5. Set an environment variable, read it from a three-line Python script, then close and reopen the terminal and watch it disappear. Then make it permanent in `~/.bashrc`.
6. Schedule the `check.sh` script with cron to run every five minutes, watch it work, then remove it. Getting comfortable adding and removing a cron entry is worth the ten minutes.

1

CHAPTER ONE

What a Data Engineer Actually Does

There is a week every October when Flipkart's data platform stops behaving like a normal system. On an ordinary day it takes in roughly 300 terabytes. During Big Billion Days that climbs to about a petabyte a day.[1] The traffic does not arrive politely. It arrives in a spike, on a date announced months in advance, watched by the entire company.

Somebody has to make sure that during that week the numbers still come out right. Not approximately right, and not right eventually — right by the morning meeting, every morning, while the volume is three times normal and half the pipelines are running behind. That work has a name, and this book is about how to do it.

You are not going to start at a petabyte a day. Almost nobody does. Your first pipeline will move a few hundred megabytes and it will break for reasons that have nothing to do with scale. What is worth knowing early is that the problems at both ends are the same problems. A file that arrived twice causes double-counted revenue whether the file is 4 MB or 4 TB. A source system that quietly changed a column name breaks a small pipeline exactly as thoroughly as a large one. Scale changes which tool you reach for. It does not change what you have to think about.

1.1 The question the whole job is built around

Sooner or later, in an interview or on your first week at work, someone will ask you a version of this:

> Walk me through what happens between a customer tapping *Place Order* and that order appearing as a number on a dashboard.

It sounds like a warm-up question. It is not. Almost everything a data engineer knows can be located somewhere on the path between those two events, and the way a candidate answers reveals within ninety seconds whether they have built systems or watched videos. People who have only studied tools answer with a list of tools. People who have built something answer with a path, and mention what could go wrong at each step.

By the end of this chapter you will be able to give a rough version of that answer. By the end of the book you will be able to give a good one.

To make the path concrete, this book follows one company from beginning to end. It is called **ShopKart**, it sells things online in India, and it does not exist. It is fictional on purpose: a real company's internals are either confidential or out of date, and a made-up one lets us be specific without pretending to knowledge we do not have. Where ShopKart's numbers matter, they are scaled down from figures that real Indian companies have published about themselves, and those real figures are cited so you can go and read them.

Here is what ShopKart looks like, in the shape of the questions a data engineer would ask on the first day:

Question	ShopKart
How many orders a day?	About 1.2 million, rising sharply during festival sales
How many customers?	22 million registered
What is the largest data source?	Clickstream — app and web events, far larger than orders
Roughly how much new data a day?	About 12 TB, of which orders are under 5 GB
Where does the data live now?	Six different systems that do not talk to each other

Notice the fourth row, because it is the first thing that surprises people. The orders are the *important* data and they are nowhere near the *big* data. A million orders at two kilobytes each is a couple of gigabytes, which fits comfortably on a laptop. The volume is in behaviour: every scroll, search, filter, product view and abandoned cart. This matters because it decides your architecture. You size your systems for the biggest thing you must move, and you design your models around the most important thing you must get right, and those are usually two different datasets.

1.2 What data is, precisely enough to be useful

A definition that is short enough to be worth memorising: **data is a recorded fact about something at a moment in time.**

Three parts of that sentence do work. A fact is about a *subject* — order 90412771. It records an *attribute* of that subject — the total amount. And it was true at a *time* — 21:14:52 on 17 March.

The third part is the one beginners drop, and dropping it causes more production incidents than anything else in this book. Data without a reliable time attached cannot be corrected, cannot be replayed after a failure, cannot be audited, and cannot settle an argument between two systems that disagree. When you eventually read about watermarks, late-arriving data, slowly changing dimensions and event-time processing, you are reading about five different consequences of that one idea.

Data arrives in three shapes, and the shape determines how much work happens before anyone can ask a question of it.

Structured data has a fixed schema that is known in advance. Rows and columns, every value in a column the same type. A database table is structured; so is a well-formed CSV. The structure is a promise, and the promise is what lets you write `SUM(total_amount)` without checking every row first.

Semi-structured data carries its structure inside itself but promises nothing about consistency. JSON is the common case:

```
{
  "order_id": 90412771,
  "customer": { "id": 5583201, "tier": "gold" },
  "items": [ { "sku": "SHO-4471", "qty": 1, "price": 3499.00 } ],
  "coupon": "RUNFAST20"
}
```

You can read that without a schema, because the keys are present. What you cannot do is assume tomorrow's message will have the same keys. The `coupon` field will be missing when nobody used a coupon. Some upstream service will one day send `"qty": "1"` as a string, and your sum will fail or, worse, silently concatenate. Most data arriving from APIs and event streams looks like this, and a large part of the pipelines chapter is about surviving it.

Unstructured data has no queryable internal structure: product photographs, support-call recordings, review text, supplier invoices as PDFs. You cannot filter a JPEG. Before unstructured data can be ana-

lysed, something has to extract structure from it — image labels, a transcript, a sentiment score — and it is that extracted output, not the original, that enters your pipeline.

A useful rough rule is that most of what a company *stores* is unstructured, while most of what it *analyses* is structured, and a good deal of a data engineer's work consists of moving things from the first category into the second.

1.3 Why the application's own database cannot answer the business's questions

ShopKart already has a database. It has the orders in it. Why not simply query it and put the result on a dashboard?

This is the right question to ask, and working through the answer properly is the fastest way to understand why this entire profession exists. There are four reasons, and they arrive in roughly this order as a company grows.

The first is that you will hurt customers. The orders database exists to serve checkout. Its resources are budgeted for thousands of small, fast operations per second. An analytical query that scans three years of orders and groups them by city can consume enough CPU, memory and disk bandwidth that live checkouts start timing out. You will have traded a report for revenue, and you will not get a second chance to make that mistake.

The second is that the data is in the wrong shape. Application databases are designed to write small amounts of data quickly without duplicating anything. That design scatters information across many narrow tables. A question as ordinary as "revenue by product category, by city, by month" might require joining eight of them. Analytical systems store the same information in a shape built for exactly that kind of question, which is the subject of Chapter 5.

The third is that the answer is not in one system. ShopKart's orders sit in PostgreSQL. Clicks are in an event stream. Payment settlements are in the payment provider's API. Support tickets are in a third-party tool. Advertising spend is inside Google Ads. The question "did the Diwali campaign actually make money?" cannot be answered from any one of them, and answering it requires deciding what counts as the *same customer* across five systems that each identify people differently.

The fourth is that production databases forget. This one is subtle and it catches almost everyone. Application tables are usually mutable: when an order ships, the `status` column is overwritten from `PLACED` to `SHIPPED`, and the previous value is gone. Now ask "how many orders were still sitting in PLACED at 6 p.m. last Tuesday?" The production database cannot tell you, because it only knows the present. Preserving history is a deliberate design decision that someone has to make on purpose, and the person who makes it is you.

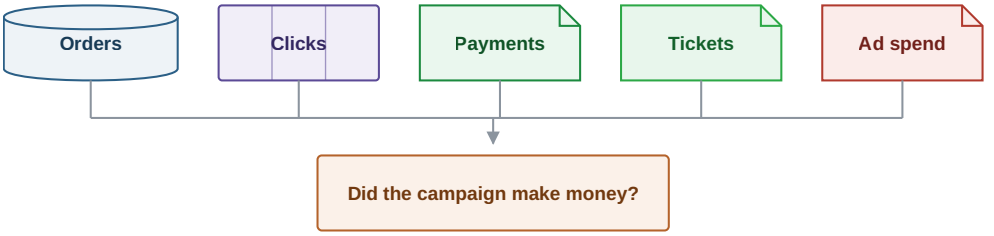


FIGURE 1.1 No single system holds the answer. Bringing five sources together, and deciding what counts as the same customer across all of them, is the work.

1.4 OLTP and OLAP: a physical difference, not a naming convention

The split described above has a formal name, and it is one of the highest-frequency interview topics in the field. A weak answer recites the acronyms. A strong answer explains why the two kinds of system are built differently at the level of bytes on disk.

OLTP, online transaction processing, describes systems that run the business one transaction at a time. Place an order, update an address, deduct stock. Enormous numbers of tiny operations, each touching a handful of rows, each expected to finish in milliseconds, each required to be exactly correct. PostgreSQL, MySQL, Oracle and SQL Server are built for this.

OLAP, online analytical processing, describes systems that analyse the business. Comparatively few queries, but each one reads millions or billions of rows across long stretches of history while touching only a few columns. Seconds or minutes are acceptable. Snowflake, BigQuery, Redshift and Databricks SQL are built for this.

The reason these cannot be the same system is physical. A row-oriented store keeps all the values of one row next to each other on disk, so fetching a complete order is a single read of one contiguous region. A column-oriented store keeps all the values of one column together, so summing `total_amount` across two billion orders reads only that

column and never touches the other forty. Column storage also compresses dramatically better, because a column holds values of one type with heavy repetition; a `city` column across a million Indian orders contains a few thousand distinct strings, and that repetition compresses to a small fraction of its raw size.

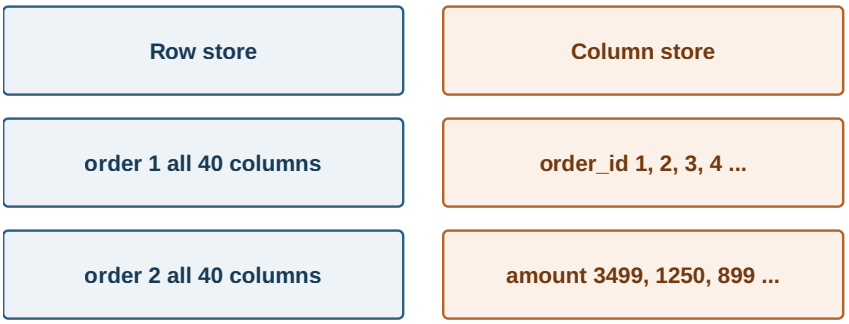


FIGURE 1.2 The same four rows, laid out two ways. Row storage is fast at "give me this whole order". Column storage is fast at "sum this one column across everything".

This is why the answer to “can’t we just add an index?” is no. An index helps when you want a small number of rows out of a large table. Analytical queries typically read a large fraction of the table, at which point the planner will often judge a full scan cheaper than the index, and you are still paying to read forty columns to use two.

	OLTP	OLAP
Purpose	Run the business	Understand the business
Typical query	Fetch or update one order	Aggregate millions of orders
Rows touched	1 to 100	Millions to billions
Expected latency	Single-digit milliseconds	Seconds to minutes
Storage layout	Row-oriented	Column-oriented
Table design	Normalised, many narrow tables	Denormalised, fewer wide tables
Writes	Constant, small, concurrent	Bulk loads on a schedule
History	Usually overwritten	Deliberately preserved
Examples	PostgreSQL, MySQL, Oracle	Snowflake, BigQuery, Redshift

1.5 The two clocks

The second great division in this field is not *where* data is processed but *when*.

Batch processing collects data over a period and processes it in one go. Every night at two o'clock, take yesterday's orders, transform them, load them into the warehouse. It is simple to reason about, cheap because the compute runs and then stops, and straightforward to re-run when something goes wrong. It is the correct default for most analytics, and a majority of the pipelines you will build in your career will be batch.

Stream processing handles each event as it arrives. An order is placed and within a second the fraud model has scored it. Streaming is much harder to run. The system runs continuously, so it must survive restarts without losing its place; events arrive out of order, so "the last five minutes" is not a simple idea; and re-running yesterday is not a matter of re-executing a script.

Swiggy is a good illustration of why a company ends up needing both. A public write-up of its platform describes order volume swinging from around a hundred orders a minute to roughly fifteen thousand at lunch and dinner peaks, with real-time event streams funnelled through Kafka and then splitting into two paths: a low-latency path through Flink for anything needing sub-second response, and a lakehouse path for analytics.[2] Confluent's account of the same platform notes that the estimated delivery time shown when a customer opens the app is served from that streaming layer.[3] The delivery estimate cannot wait for tonight's batch job. The weekly report on which restaurants are underperforming can.

That is the decision rule, and it is worth writing down somewhere you will see it again:

> Ask what decision changes if this data is five hours old instead of five minutes old. If nothing changes, streaming is a permanent operational cost buying you nothing.

Plenty of teams have built a streaming pipeline to feed a dashboard that one human looks at twice a day. The middle option, and the one most companies actually run, is **micro-batch**: small batches processed frequently, every minute or every thirty seconds. It gives most of the freshness of streaming with most of the simplicity of batch, and it is how Spark Structured Streaming works by default.

	Batch	Streaming
Triggered by	A schedule	An event arriving
Freshness	Minutes to hours	Milliseconds to seconds
Cost profile	Compute runs, then stops	Always on
Re-running	Run yesterday again	Requires replay and state handling
Suits	Reports, finance, ML training sets	Fraud, alerts, live estimates, personalisation

1.6 The map

Here is the shape almost every data platform takes. Do not try to absorb the boxes; each one has a chapter later. What matters now is the direction of travel and the reason for each stage.

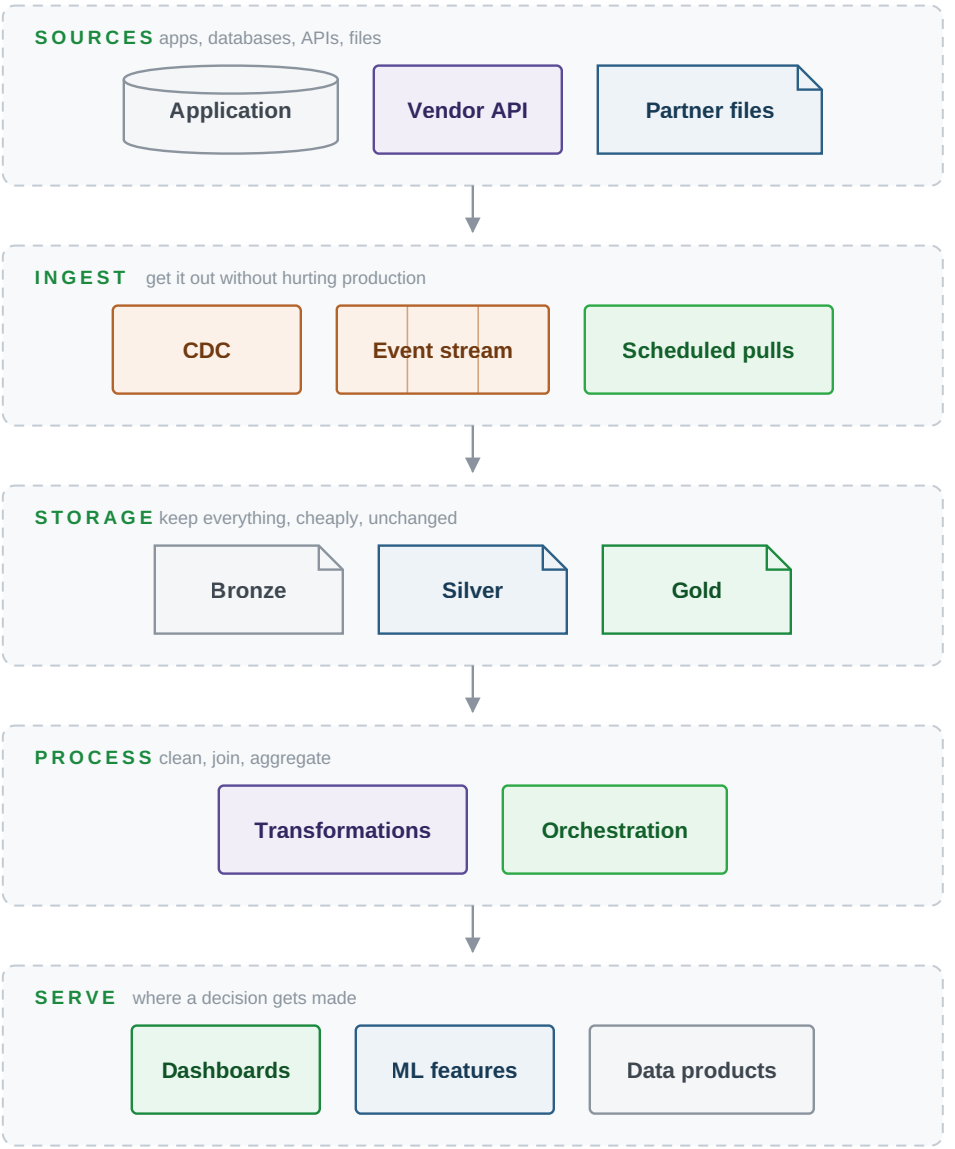


FIGURE 1.3 The canonical data platform. Every chapter after this one is a zoom into one of these zones.

Read it as a sentence. *Get the data out of the systems that created it, keep an untouched copy somewhere cheap, reshape it into something meaningful, put the result where it can be queried quickly, and hand it to whoever has to decide something.*

Two of those stages deserve a word now because beginners routinely skip them.

Keep the raw copy. It is tempting to transform on the way in and store only the clean result. Six months from now you will find a bug in a transformation you wrote today. If the raw data still exists, you fix the code and reprocess. If it does not, that history is gone permanently. Object storage is cheap enough that this is rarely the right thing to economise on.

Separate ingestion from processing. When ingestion writes files and processing reads files, neither needs to know anything about the other's schedule or health. If the processing cluster is down for two hours, ingestion keeps running and processing catches up. Systems that are wired directly together fail together.

This layering is not a theory. Swiggy's platform is described as using a layered model over open formats specifically to standardise data and avoid silos,[2] and the same pattern under the names bronze, silver and gold appears across the industry. Chapter 6 covers it properly.

1.7 The work itself

Job descriptions describe an idealised version of this role. What follows is closer to what the weeks look like.

You will spend a large share of your time **building pipelines** — writing the code that extracts data from a source, reshapes it and lands it somewhere useful. Everyone imagines the job is this part, and it is a smaller fraction of it than you would think.

You will spend at least as much time **repairing pipelines that used to work**. A vendor changes a date format without telling anyone. A partner uploads yesterday's file twice. A job that ran in twenty minutes now takes six hours because a marketing campaign doubled the volume. None of this appears on a syllabus and all of it happens every week.

You will spend real time **deciding how tables should be structured** so that questions are answerable and two people asking the same question get the same number. Chapter 5 is about this, and it is the work that most separates engineers who are trusted from engineers who are merely busy.

You will spend a surprising amount of time **explaining why a number looks wrong**. An analyst reports that revenue is down twelve per cent. Most of the time the pipeline is fine and the two of

you are counting different things — their query includes cancelled orders and yours does not. Some of the time it is a real defect and you have a few hours to find it.

And you will spend the remainder on the things that surround engineering anywhere: reviews, documentation, access requests, controlling cloud spend, and explaining to a stakeholder why "just add one column" is a two-week piece of work because four dashboards depend on the current shape of that table.

1.8 The neighbouring jobs

This comparison comes up in nearly every interview. The weak answer lists job titles. The strong answer says what each role *owns* and what each role *takes on trust* from someone else.

A **data analyst** asks questions of data and communicates the answers. They consume tables. You guarantee tables — that they exist, are fresh, are complete and did not quietly lose a day. When a table is missing yesterday's rows, the analyst notices; you are the one whose phone rings.

A **data scientist** builds statistical and machine-learning models to predict or explain, and needs clean, historical, feature-rich data to do it. You supply it. The volume of preparation work involved is precisely why data platforms are built.

A **machine learning engineer** takes a model that works in a notebook and makes it work in production — serving it, monitoring it for drift, retraining it. There is real overlap with data engineering, since both care about pipelines and reproducibility, but the ML engineer's deliverable is a functioning model and yours is trustworthy data. At a small company the same person often does both.

A **software engineer** builds the application that creates data in the first place. The deepest difference is not the technology but how failure behaves. When an application breaks it usually breaks loudly: requests error, users complain, alerts fire. When a data pipeline breaks it very often breaks silently. The job succeeds, the dashboard renders, the number is simply wrong, and nobody finds out for eleven days. Designing for silent failure is a distinct discipline, and it is why Chapter 12 exists.

	Data engineer	Data analyst	Data scientist	ML engineer
Deliverable	Reliable data systems	Insight and reporting	Models and experiments	Models in production
Core question	Is the data there and correct?	What does the data say?	What will happen next?	Does the model hold up at scale?
Main tools	SQL, Python, Spark, Airflow, cloud	SQL, BI tools, spreadsheets	Python, statistics, ML libraries	Python, serving infrastructure
Optimises for	Reliability, freshness, cost	Clarity and speed of answer	Predictive accuracy	Latency and stability
Fails when	Data is late, wrong or missing	The question is misread	The model does not generalise	The model degrades unnoticed

1.9 The failure mode that defines the discipline

It is worth ending the first chapter on the thing that makes this job different from the jobs around it.

Consider what happens when a partner uploads the same file twice. Nothing errors. The ingestion job reads both files exactly as designed. The transformation runs and succeeds. The load into the warehouse succeeds. Every monitor is green, every job shows as completed, and yesterday's revenue is exactly double what it should be. Somebody in finance may notice within a day. If the duplicated file is a smaller partner, nobody notices for a month, and by then it is in a report that has already been shown to the board.

Now consider the same class of problem in other forms. A join key that has duplicates on both sides, so a fact table quietly multiplies. A source that sends sixty per cent of its usual volume because an upstream service was degraded, producing a number that looks plausible. A timezone difference between two systems that shifts a day boundary by five and a half hours, so every daily total is a mixture of two days.

None of these throw an exception. All of them produce a dashboard that renders. This is why mature data teams monitor the *data* and not only the *job*, why pipelines are designed so that running them twice produces the same result as running them once, and why the phrase "the pipeline ran successfully" is not the same claim as "the numbers are right". Every technique in Chapter 12 — freshness checks, row-

count reconciliation, referential integrity tests, idempotent writes, quarantine tables — is a response to this single property of data systems.

Summary

- Data is a recorded fact about a subject at a moment in time. The moment is the part beginners drop, and most of the hard machinery later in this book exists to handle it.
- The application's own database cannot serve analytics, for four separate reasons: it competes with customer traffic, the shape is wrong, the answer spans several systems, and it does not remember history.
- OLTP and OLAP differ physically, in how bytes are laid out on disk. That is why an index does not rescue an analytical query.
- Batch is the sensible default. Streaming is justified when a decision genuinely changes with freshness, and micro-batch covers most of the middle ground.
- The platform's direction is always the same: sources, ingestion, raw storage, processing, modelled tables, serving. Keep the raw copy, and keep ingestion decoupled from processing.
- The failure mode that defines this discipline is the pipeline that succeeds while producing wrong data.

In the interview

What does a data engineer do?

SHORT ANSWER

I build and operate the systems that move data from where it is created — applications, databases, APIs, event streams — into a place where it can be analysed, in a modelled and trustworthy form. In practice that means ingestion, transformation, storage design, orchestration, and the quality checks and monitoring that keep it correct as the sources change.

DEEPER ANSWER

The constraint that makes it engineering rather than scripting is that everything upstream changes underneath you and nobody tells you. Schemas change, files arrive twice, an API starts rate-limiting, volume doubles after a campaign. So the work is designing for those conditions: jobs that can safely re-run, incremental loads that know where they stopped, validation that catches bad data before it reaches a dashboard, and alerting on data freshness rather than only on job crashes.

FOLLOW-UP: WHAT IS THE HARDEST PART?

Silent failure. A job that crashes is easy, because you get an alert. The dangerous case is a job that succeeds while producing wrong data — a join that duplicated rows, a source that sent partial volume, a timezone shift at a day boundary. That is why mature teams monitor the data itself, not just the pipeline.

SENIOR-LEVEL ANSWER

Frame it as owning a contract rather than a codebase. My deliverable is a set of guarantees about tables: this table is complete for day D by six in the morning, one row means exactly one order line, revenue reconciles to source within a defined tolerance, and late-arriving data is corrected within twenty-four hours. Every technical decision — incremental against full load, batch against streaming, how aggressively to partition — follows from which guarantee I have promised and what the business is willing to pay for it.

PRACTICE

1. Open any app you used today and list five pieces of data you created. For each one, name a team inside that company that would want it and a team that would not care.
2. Classify each of the five as structured, semi-structured or unstructured, and justify it in one line.
3. Take one of them and ask what changes if it arrives an hour late, a day late, a week late. Your answer is the batch-or-streaming decision, and this is exactly the reasoning an interviewer wants to see.
4. Without looking back at the chapter, write three sentences explaining why running an analytical query on a production database can hurt customers. If you cannot do it, re-read section 1.3.

> 5. Say the answer to “walk me through what happens between Place Order and the dashboard” out loud, timed. You will be bad at it today. Do it again after Chapter 6 and after Chapter 12.

SOURCES

1. Flipkart Tech Blog, *Beyond the Data Horizon: Flipkart’s Race to Cloud at Unmatched Scale* (2025). States average daily ingestion of roughly 300 TB, rising to approximately 1 PB per day during the Big Billion Days sale, across a platform holding petabytes of active data.
2. Public architecture write-up of Swiggy’s real-time data platform (2026), describing peak order rates of roughly 100 to 15,000 orders per minute, Kafka as the central durable event queue, a low-latency Flink path alongside a lakehouse path, and a layered data model over open formats.
3. Confluent customer story, *How Swiggy Meets the Demands of Data-Driven Delivery*, which describes the streaming platform powering the predicted delivery time shown when a customer opens the app.

ShopKart is fictional. Its figures are illustrative and scaled down from published Indian e-commerce and food-delivery figures; they are not attributed to any real company.

2

CHAPTER TWO

The Machine Underneath

A JOB THAT RAN IN TWELVE MINUTES yesterday takes four hours today. Nothing in the code changed. Three engineers offer three theories within a minute of each other: the cluster is too small, the data grew, add more memory. All three are guesses, and one of them will probably be right by accident, which is worse than being wrong because it teaches nothing.

An engineer who understands the machine asks a different question first. Not *what should we change*, but *what is this job waiting for*. There are only four possible answers, and narrowing to one of them turns an unbounded search into a bounded one.

This chapter is the smallest amount of computer science that changes how you debug. It is not a syllabus. Everything in it earns its place by showing up later, in Chapter 9 when a Spark job spends most of its life moving data between machines, in Chapter 6 when a query slows down by a factor of thirty because someone wrote too many files, and in Chapter 12 when a pipeline dies at three in the morning without leaving a stack trace.

2.1 Four resources, one question

Almost every computation is limited by one of four things.

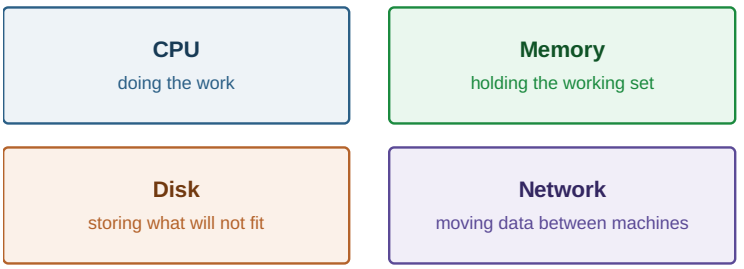


FIGURE 2.1 Every slow job is waiting on one of these. Naming which one is the difference between debugging and guessing.

When something is slow, the useful first move is to find which of the four is saturated. If the CPUs are pinned at a hundred per cent, you have a compute problem and more cores will help. If the CPUs are idle while the job crawls, you have a waiting problem, and adding cores will change nothing at all except your bill. That single distinction — busy or waiting — resolves a surprising share of performance arguments before anyone opens the code.

THE HABIT WORTH BUILDING

Before changing anything, decide whether the system is *busy* or *waiting*. Busy means CPU. Waiting means memory pressure, disk, or network. Most wasted optimisation effort comes from treating a waiting problem as a busy one.

2.2 CPU: cores are the unit of parallelism

The **CPU** executes instructions. Its clock speed, quoted in gigahertz, tells you how many basic steps it takes per second, and it is the number people quote in advertisements. The number that matters to you is **cores**.

A core is an independent execution unit. A server CPU with sixteen cores does sixteen things at the same instant, not sixteen things in rapid rotation. That physical fact sits underneath every distributed system you will ever use.

It is also the reason Spark behaves the way it does. Spark splits your data into **partitions** and runs one **task** per available core slot. A cluster with five executors of four cores each has twenty slots, so twenty tasks run at once. Give that cluster two hundred partitions and it works through them in ten waves. Give it eight partitions and twelve cores sit completely idle while you pay for all twenty.

That relationship — partitions on one side, cores on the other — is one continuous conversation that runs from here to the end of Chapter 9. When you later read advice to “increase parallelism”, it means nothing more mysterious than *make sure there is enough work to keep every core busy, and not so much that the overhead of managing it exceeds the work itself*.

2.3 Memory: fast, small and forgetful

RAM is the working area. Anything a program is actively using sits here. It has two properties that will shape your work.

It is **small** relative to your data. A large server might have 256 GB of memory and tens of terabytes of disk attached. Your dataset will routinely be bigger than memory, and systems that assume otherwise fall over at exactly the moment they become useful.

It is **volatile**. Cut the power and the contents are gone. This is why a database writes to disk before it confirms that a transaction succeeded, a mechanism we come back to in Chapter 8.

When people describe Spark as an in-memory engine, this is what they mean: it keeps intermediate results in RAM between steps rather than writing them to disk after every stage, which is the main reason it outruns the older MapReduce model. It is also the source of the most common way a job dies. `OutOfMemoryError` means one process tried to hold more than it was given. Two causes account for most cases in practice: one partition was far larger than the others, so a single task tried to load an unfair share; or somebody called an operation that pulls the entire distributed dataset back to a single machine, which works beautifully on a sample and fails catastrophically on the real thing.

2.4 Disk: sequential is cheap, random is not

Disk is where data survives a restart. A **hard disk** stores data on spinning magnetic platters and has to physically move a head to reach a location. A **solid-state drive** uses flash memory with no moving parts and is far quicker, particularly when reads jump around.

The distinction that actually drives design decisions is not the device but the **access pattern**. Reading one large file from beginning to end is fast on anything. Reading ten thousand small files scattered across storage is slow even on good hardware, because each one costs a separate lookup, a separate open, a separate round of metadata work, and none of that cost has anything to do with the size of the data.

To make the differences concrete, here are the orders of magnitude that circulate in the industry as *latency numbers every programmer should know*, a list attributed to Jeff Dean and originally compiled by Peter Norvig.[1] The absolute values date from around 2012 and hardware has moved since, but the ratios between them have held, and the ratios are the part you need.

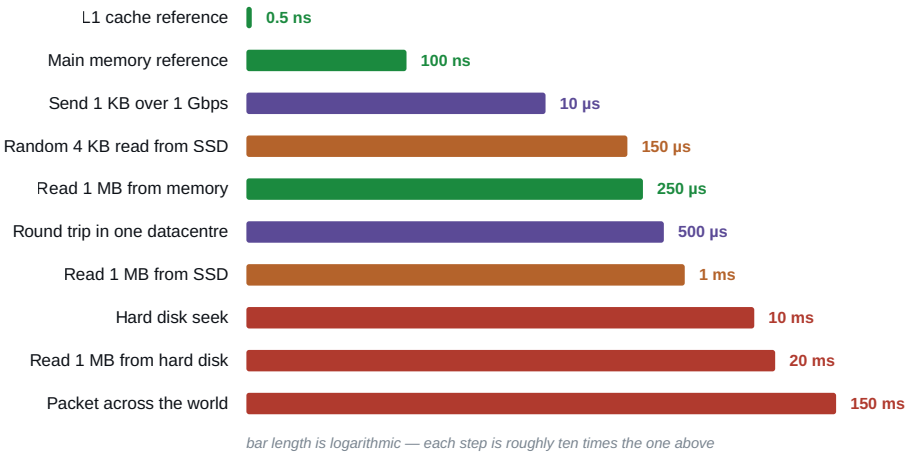


FIGURE 2.2 Orders of magnitude, from cache to intercontinental network. Reading one megabyte from memory is roughly forty thousand times cheaper than a round trip across the world.

Read the chart twice. The first reading tells you memory is faster than disk, which you knew. The second tells you something more useful: a single hard disk seek costs about as much as reading a hundred megabytes sequentially from memory. Seeks are the expensive part, not bytes.

That explains one of the most common production problems in this field, the **small files problem**. A streaming job that writes one file every ten seconds produces 8,640 files a day and around three million a year. A query that should read 200 GB in two minutes takes an hour, and almost none of that hour is spent reading data. It is spent opening files. Chapter 6 covers how to prevent it and how to repair it once it has happened; Chapter 9 covers why it punishes Spark particularly hard.

	Memory	SSD	Hard disk	Object storage
Relative speed	Fastest	Fast	Slow	Slow per request, hugely parallel
Typical capacity	Tens to hundreds of GB	Terabytes	Tens of terabytes	Effectively unlimited
Cost per terabyte	Very high	Medium	Low	Lowest
Survives a restart	No	Yes	Yes	Yes

	Memory	SSD	Hard disk	Object storage
Used for	Active computation	Databases, hot data	Bulk and archive	Data lakes

2.5 Network: the bottleneck nobody budgets for

Machines talk over a network, and network capacity is quoted in bits per second rather than bytes. A ten gigabit link moves at best about 1.25 gigabytes a second, and in practice less.

In distributed processing the network is very often the real constraint. When Spark performs a **shuffle** — redistributing rows across machines so that every record with the same key ends up in the same place, which is what a `GROUP BY` or a join between two large datasets requires — it physically moves data between machines. Moving half a terabyte across a cluster costs what it costs, no matter how many cores you have rented.

> ONE WAY TO PICTURE IT
>
> A large kitchen. The CPUs are cooks, memory is the counter space they work on, disk is the storeroom at the back, and the network is the corridor in between. Hiring four more cooks does nothing if all of them are queuing in the corridor, and a shuffle-heavy job is exactly that queue. This is why the instinct to add more machines so often fails to help, and occasionally makes things worse by adding more traffic to the same corridor.

2.6 Processes, threads, and why jobs vanish

A **process** is a running program with its own private memory. A **thread** is a stream of execution inside a process, and threads in the same process share memory, which makes communication between them fast and mistakes between them dangerous.

Three consequences matter to you.

Processes are isolated from one another. When one Spark executor crashes, the others carry on. This is why distributed systems prefer many small processes over one large one.

Memory limits are enforced per process. When a container is given eight gigabytes and the process inside it asks for nine, the operating system kills it. In Spark or Kubernetes this appears as an executor that simply disappears with an exit code, not as a friendly

Python traceback. **A job that died without a stack trace was almost certainly killed for memory**, and knowing that one fact will save you hours in your first month.

Python's threading is unusual, and it is currently changing. For most of Python's history the interpreter has had a Global Interpreter Lock, which allows only one thread to execute Python bytecode at a time. Threads still help when a program is *waiting* — on a network call, on a file read — but they do not let pure Python computation use several cores, which is why data tools distribute work across processes rather than threads.

That is now a moving target and worth stating precisely, because it is the kind of detail an interviewer uses to tell current knowledge from memorised knowledge. PEP 703 introduced an optional build of CPython with the lock disabled. Python 3.13 shipped that free-threaded build as experimental. PEP 779 was accepted for Python 3.14, moving it to officially supported while still optional, and the standard build of every supported version still ships with the lock enabled.[2] The Python documentation also notes that some third-party packages, particularly those with C extension modules, will re-enable the lock when loaded into a free-threaded build.[3] So the honest answer in 2026 is that the constraint is being dismantled deliberately and slowly, and that you should assume it applies unless you have specifically checked.

2.7 Files and processes on a real machine

Lecture Zero covered the terminal itself — navigating, inspecting files, permissions, exit codes. Two points from it are worth restating here because they connect directly to the four resources.

`df -h` tells you whether a disk is full, and a full disk is the cause of a surprising share of jobs that fail with no useful message. `top` tells you whether the machine is busy or waiting, which is the distinction section 2.1 opened with: high CPU means compute-bound, low CPU with a slow job means it is blocked on disk or network.

The third point is the one Lecture Zero introduced and this chapter can now explain. Every command returns an exit code, and schedulers read it to decide whether your job worked. A process that is killed by the operating system for exceeding its memory limit does not get to run its error handler, does not print a traceback, and returns a code that is not zero. That is why "the job died and there is nothing in the logs" is almost always a memory problem rather than a mystery.

2.8 Servers, ports and the four connection errors

A **server** is a program that waits for requests and answers them. A **client** is a program that makes requests. These are roles, not machines: one machine runs many servers, and a program is frequently a server to one thing and a client to another.

A machine has one address but must host many servers, so each server listens on a numbered **port**. The address identifies the machine; the port identifies which program on it.

Port	Service
22	SSH, remote shell access
80 / 443	HTTP / HTTPS
5432	PostgreSQL
3306	MySQL
9092	Kafka
8080	Airflow web interface, by convention

> **WHAT THE ERROR IS ACTUALLY TELLING YOU**
>
> **Connection refused** — the network reached the machine and nothing was listening on that port. The service is down, or you have the wrong port.
>
> **Connection timed out** — the packets never got a reply at all. Almost always a firewall or security group dropping them silently, or the wrong host.
>
> **Host not found** — the name could not be translated into an address. A DNS problem, or a typo.
>
> **Authentication failed** — you reached the service and the credentials were rejected. This is good news: the plumbing works, and only the last step is wrong.
>
> Learning to tell those four apart is worth more in your first month than any framework you could study.

2.9 HTTP, JSON and the shape of an ingestion loop

An **API** is a contract that lets one program ask another for something. Most of the data APIs you will meet are REST APIs over HTTP.

A request carries a method, a path, headers and sometimes a body:

```
GET /api/v1/orders?date=2026-03-17&page=2 HTTP/1.1
Host: api.shopkart.example
Authorization: Bearer eyJhbGciOi...
Accept: application/json
```

The response carries a **status code**, and reading status codes correctly is a genuine skill rather than a formality. The single most common defect in beginner ingestion code is retrying something that will never succeed, or failing to retry something that would have succeeded on the second attempt.

Code	Meaning	What you should do
200, 201	Success	Continue
400	Bad request	**Do not retry.** Your parameters are wrong; fix the code
401, 403	Not authenticated, or not permitted	**Do not retry.** Credentials or permissions
404	Not found	Wrong URL, or the resource genuinely is not there
429	Too many requests	**Retry**, after waiting — and increase the wait each time
500, 502, 503	The server failed	**Retry** with backoff and a maximum attempt count

Real ingestion code therefore looks less like a single request and more like a loop with judgement in it. Here is the shape, reduced to the parts that matter:

```
import os, time, requests

BASE = "https://api.shopkart.example/v1/orders"
RETRYABLE = {429, 500, 502, 503, 504}

def fetch_page(session, page, attempt=1):
    """One page of results, retrying only what is worth retrying."""
    response = session.get(
        BASE,
        params={"date": "2026-03-17", "page": page},
        timeout=(5, 30),          # connect timeout, then read timeout
    )
    if response.status_code in RETRYABLE:
        if attempt > 5:
```

```
            raise RuntimeError(f"page {page} failed x{attempt}")
        wait = min(2 ** attempt, 60)      # 2s, 4s, 8s, 16s, 32s, capped
        time.sleep(wait)
        return fetch_page(session, page, attempt + 1)
    response.raise_for_status()      # 4xx stops here, deliberately
    return response.json()

def fetch_all():
    session = requests.Session()
    token = os.environ["SHOPKART_TOKEN"]
    session.headers["Authorization"] = f"Bearer {token}"
    page, rows = 1, []
    while True:
        payload = fetch_page(session, page)
        rows.extend(payload["data"])
        if page >= payload["total_pages"]:
            break
        page += 1
    return rows
```

Four things in that code are there for a reason, and all four are omitted from most tutorials.

The **timeout is a pair**. One value bounds how long you wait to establish a connection, the other how long you wait for the response body. A single number leaves one of those unbounded, and a request that hangs forever is how a nightly pipeline is still running at nine in the morning.

The **retry list is explicit**. Only codes that might succeed on a second attempt are retried. A 400 is retried zero times because the request itself is wrong, and retrying it five times just means being wrong five times more slowly.

The **wait grows**. Retrying immediately against a server that is already overloaded is how a small problem becomes an outage. Doubling the wait, with a cap, is called exponential backoff and it is the standard answer.

The **loop knows when to stop**. It uses the API's own page count rather than stopping when a page comes back empty. Pipelines that stop on an empty page silently truncate on the day the API returns an empty page in the middle.

2.10 Credentials

Three authentication styles cover nearly everything you will meet. An **API key** is a long string sent in a header; it is simple and it never expires, which is convenient and also a liability. A **bearer token** is

obtained by exchanging credentials and expires after a while, which limits the damage if it leaks. A **cloud IAM role** grants an identity to the machine itself, so the SDK fetches temporary credentials automatically and no secret exists in your code at all. When the third option is available, it is the right one.

Whichever you use, the rule does not bend: **credentials never go in code**. Not in a notebook, not in a configuration file that gets committed, not in a variable you intend to remove before pushing. Public repositories are scanned by automated tools within minutes of a key appearing, and rotating a leaked production credential is a genuinely bad afternoon.

The standard mechanism is an environment variable, a value handed to your process by the operating system when it starts:

```
import os

user = os.environ["SNOWFLAKE_USER"]          # missing -> fails now
password = os.environ.get("SNOWFLAKE_PASSWORD")   # missing -> None
```

Prefer the first form. A pipeline that starts with `password=None` and fails three steps later is far harder to diagnose than one that refuses to start. In production these values come from a secret manager or, in Airflow, from a Connection, which is the same idea with a user interface attached.

Summary

- Every slow job is waiting on CPU, memory, disk or network. Decide whether the system is busy or waiting before you change anything.
- Cores are the unit of parallelism, and partition counts should be chosen against core counts.
- Memory is fast, small and volatile. A job that dies with no stack trace was almost certainly killed for memory.
- Seeks cost far more than bytes, which is why ten thousand small files are slower than one large file of the same size.
- The network is the hidden constraint in distributed processing, and shuffles are why.
- Python’s Global Interpreter Lock is being made optional rather than removed; assume it applies unless you have checked the build.

- The four connection errors each point at a different layer. Learn to tell them apart.
- Status codes decide whether to retry, fix or stop. Timeouts come in pairs, waits grow, and loops stop on the API's own page count.
- Credentials live in environment variables or a secret manager, never in code.

In the interview

A job that used to take fifteen minutes now takes three hours, and the code has not changed. How do you approach it?

SHORT ANSWER

I would find out what it is waiting on before changing anything. First I would check whether the input volume changed, then look at which stage got slower, and work out whether the time is going into shuffle, into one task that is far slower than its peers, into reading an unusually large number of files, or into garbage collection because memory is tight.

DEEPER ANSWER

The realistic causes are a short list. Volume grew. The distribution changed, so one key now dominates and a single task does most of the work while the rest finish early. The input file count exploded, because an upstream producer changed and four hundred files became four hundred thousand, at which point per-file overhead dominates and the data volume is almost irrelevant. A join that used to be broadcast now exceeds the threshold and has become a shuffle. Or the cluster is smaller or busier than it was.

FOLLOW-UP: HOW WOULD YOU TELL A VOLUME INCREASE APART FROM SKEW?

Look at the spread of task durations inside the slow stage. A volume increase moves every task's duration up together. Skew shows a stage where the median task takes seconds and the slowest takes an hour, and total input bytes may have barely moved.

SENIOR-LEVEL ANSWER

I would also want to know why this arrived as a surprise. The job should be emitting its own metrics — input row count, input file count, output row count, duration per stage — into the same monitoring as everything else, so a trend like this is something I watch build over a fortnight rather than something I discover as an incident. Most sudden slowdowns were visible for weeks in numbers nobody was collecting.

PRACTICE

1. On any Linux machine, or on Windows Subsystem for Linux, run `df -h`, `free -h` and `top`. Write down how much memory is free and how full the busiest disk is. Do it again while something heavy is running and compare.
2. Take any public CSV. Using only `wc -l`, `head`, `cut`, `sort` and `uniq -c`, find its row count, its column names, and the distribution of one column. No Python.
3. Create two directories with the same total volume of data: one holding a single large file, the other holding ten thousand small ones. Time reading each. The gap is the small files problem, and seeing it once is worth more than reading about it three times.
4. Call any free public API with `curl -i` and read the status line and headers, not just the body. Then call a URL that does not exist and compare.
5. Explain to someone non-technical why adding more machines can fail to make a job faster. If they follow it, you understand section 2.5.

SOURCES

1. *Latency Numbers Every Programmer Should Know*, a widely circulated list attributed to Jeff Dean and originally compiled by Peter Norvig. The figures in Figure 2.2 follow the commonly published version, whose absolute values date from around 2012; hardware has improved since, but the relative orders of magnitude remain the useful part.
2. PEP 703, *Making the Global Interpreter Lock Optional in CPython*, and PEP 779, *Criteria for supported status for free-threaded Python* (accepted for Python 3.14, moving the free-threaded build from experimental to officially supported while remaining optional).
3. Python documentation, *Python support for free threading*, which notes that free-threaded builds exist from Python 3.13 onward and that some third-party packages, particularly those with extension modules, will re-enable the lock.

PART TWO

The Two Languages

SQL and Python are the whole toolkit for the first year. The level that gets you hired sits well past where most tutorials stop.

3 · SQL to Interview Level

4 · Python for Data Engineers

3

CHAPTER THREE

SQL to Interview Level

Here is a query that a competent analyst wrote, that runs without error, that returns a plausible number, and that is wrong by forty-five per cent. It is the single most common serious defect in analytical SQL, it survives code review regularly, and by the end of section 3.6 you will be able to spot it in about four seconds.

> VERSIONS AND DIALECTS
>
> SQL in this chapter is ANSI-standard where possible and was executed on **DuckDB 1.5**. It runs on PostgreSQL with two exceptions noted in the text: `QUALIFY` is not available in PostgreSQL, and interval syntax inside `RANGE` frames varies slightly by engine.

```sql
SELECT SUM(o.order_total) AS revenue
FROM orders o
JOIN order_items i ON i.order_id = o.order_id
WHERE o.status = 'DELIVERED';
```

The real answer for our dataset is **30,390**. That query returns **44,036**. Nothing about it looks suspicious, which is exactly the problem.

SQL is the highest-return skill in this book. Not because it is the most sophisticated, but because it is the one you will use every single day, in every job, at every level, and because the level of SQL that gets you hired sits well past the level where most tutorials stop. Interviewers know this. A data engineering loop will nearly always contain a live SQL round, and the questions in it are chosen precisely because they separate people who can write a query from people who understand what the engine does with it.

This chapter is long. Work through it with a database open.

3.1 The dataset

Everything in this chapter runs against six small ShopKart tables. Every query printed here was executed against this data before it went into the book, and every result shown is the actual output.

You can create it in PostgreSQL, DuckDB or SQLite. DuckDB is the easiest option for learning: `pip install duckdb`, then `duckdb.connect()`, with no server to run.

```
CREATE TABLE customers (
    customer_id  INTEGER PRIMARY KEY,
    full_name    VARCHAR,
    city         VARCHAR,
    signup_date  DATE,
    tier         VARCHAR          -- nullable, and that matters later
);

CREATE TABLE orders (
    order_id     INTEGER PRIMARY KEY,
    customer_id  INTEGER,
    order_ts     TIMESTAMP,
    status       VARCHAR,         -- DELIVERED, CANCELLED, RETURNED
    order_total  DECIMAL(10,2)
);

CREATE TABLE order_items (
    order_item_id INTEGER PRIMARY KEY,
    order_id      INTEGER,
    product_id    INTEGER,
    quantity      INTEGER,
    line_amount   DECIMAL(10,2)
);

CREATE TABLE products (
    product_id   INTEGER PRIMARY KEY,
    product_name VARCHAR,
    category     VARCHAR,
    unit_price   DECIMAL(10,2)
);

CREATE TABLE payments (
    payment_id   INTEGER PRIMARY KEY,
    order_id     INTEGER,
    paid_ts      TIMESTAMP,
    method       VARCHAR,
    amount       DECIMAL(10,2)
);

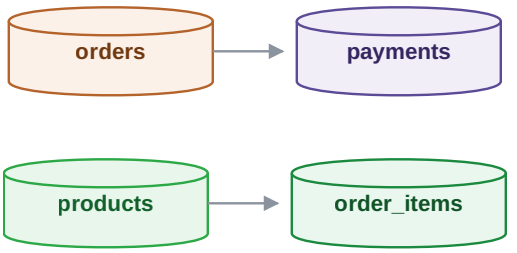
CREATE TABLE app_events (
    event_id     INTEGER PRIMARY KEY,
    customer_id  INTEGER,
```

```
    event_ts     TIMESTAMP,
    event_name   VARCHAR
);
```

Six customers, fourteen orders across February and March 2026, seventeen order lines, thirteen payments. The full seed data is in the companion Field Kit. Three deliberate imperfections are built in, because clean data teaches nothing: one customer has a `NULL` tier, one order was paid twice, and two orders have no payment at all.



FIGURE 3.1 The ShopKart tables and how they connect. One customer has many orders; one order has many items and, in this data, occasionally more than one payment.



3.2 Five queries to get your hands moving

Before the hard parts, five simple ones. Type each, run it, and check that your output matches. If you have written SQL before this takes four minutes and confirms your setup works.

```
-- 1. Pick columns, filter rows.
SELECT full_name, city
FROM customers
WHERE city = 'Pune';
```

```
 full_name    | city
--------------+------
 Priya Sharma | Pune
 Aisha Khan   | Pune
```

```sql
-- 2. Compare a number, and sort the result.
SELECT order_id, order_total
FROM orders
WHERE order_total > 4000
ORDER BY order_total DESC;
```

```
 order_id | order_total
----------+-------------
     9009 |     5099.00
     9001 |     4749.00
     9004 |     4200.00
```

```sql
-- 3. Count everything.
SELECT COUNT(*) AS total_orders FROM orders;
```

```
 total_orders
--------------
           14
```

```sql
-- 4. Count within groups.
SELECT status, COUNT(*) AS orders
FROM orders
GROUP BY status
ORDER BY orders DESC;
```

```
 status    | orders
-----------+--------
 DELIVERED |     12
 CANCELLED |      1
 RETURNED  |      1
```

```sql
-- 5. Pull a name from another table, and take the top three.
SELECT c.full_name, o.order_id, o.order_total
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
WHERE o.status = 'DELIVERED'
ORDER BY o.order_total DESC
LIMIT 3;
```

```
 full_name    | order_id | order_total
--------------+----------+-------------
 Arjun Das    |     9009 |     5099.00
 Priya Sharma |     9001 |     4749.00
 Aisha Khan   |     9004 |     4200.00
```

Query 5 already has the shape of nearly everything in this chapter: take rows from one table, attach matching rows from another, filter, sort, limit. The rest is variations on it, and the traps that hide inside those variations.

3.3 How a `SELECT` actually runs

Almost every confusing thing about SQL becomes obvious once you know this, and most people are never taught it.

You write a query in one order. SQL *defines its meaning* in a different order. This is called the logical processing order, and it is what determines what a query means — not the sequence of operations the engine physically performs.



FIGURE 3.2 The logical processing order. It defines what the query means. The optimiser is free to execute in any order that produces the same answer.

> **LOGICAL ORDER IS NOT EXECUTION ORDER**
>
> The engine does not run these steps top to bottom. The optimiser reorders joins, pushes filters down into the table scan so rows are discarded before they are ever read, and evaluates expressions in whatever order preserves the result. Section 3.10 shows exactly this: `EXPLAIN` output that looks nothing like the list above.
>
> So why learn the list? Because it defines *semantics*. It is why `WHERE` cannot see an aggregate and `HAVING` can, and why a `SELECT` alias is unavailable in `WHERE`. Those are rules about meaning, and they hold no matter what the optimiser chooses to do.

Three practical consequences fall straight out of that picture.

`WHERE` filters rows, `HAVING` filters groups. `WHERE` runs before grouping, so it cannot see an aggregate. `HAVING` runs after, so it can. The rule everyone repeats — use `WHERE` for rows and `HAVING` for aggregates — is not a convention, it is a description of the order above.

You cannot use a `SELECT` alias in `WHERE`. By the time `WHERE` runs, the alias does not exist yet, because `SELECT` has not happened. You can normally use it in `ORDER BY`, because that runs afterwards. This is why the same alias works in one clause and errors in another, which looks arbitrary until you have this diagram in your head.

`WHERE` is usually cheaper than `HAVING`. Filtering before grouping means fewer rows to group. A modern optimiser will often push a `HAVING` predicate down when it can prove the rewrite is safe, so the difference may vanish in the plan — but writing it in `WHERE` states your intent clearly and does not depend on the optimiser noticing.

```sql
-- Correct: status filters rows, so it belongs in WHERE.
-- The revenue threshold is an aggregate, so it belongs in HAVING.
SELECT c.city,
       SUM(o.order_total) AS revenue
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
WHERE o.status = 'DELIVERED'
GROUP BY c.city
HAVING SUM(o.order_total) > 5000
ORDER BY revenue DESC;
```

WORTH MEMORISING

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. If you can recite that, you can answer roughly a third of beginner SQL interview questions from first principles instead of from memory.

3.4 `NULL`, and the trap that eats interview candidates

`NULL` does not mean zero and it does not mean empty string. It means *unknown*. Once you accept that, SQL's behaviour stops being strange.

Because `NULL` is unknown, any comparison with it produces neither true nor false but a third value, also unknown. `NULL = NULL` is not true. `NULL <> 'gold'` is not true. Filters keep only rows that are definitely true, so unknown rows are silently dropped.

```sql
SELECT tier,
       COUNT(*)     AS rows_in_group,
       COUNT(tier)  AS non_null_tiers
FROM customers
GROUP BY tier
ORDER BY tier;
```

```
 tier    | rows_in_group | non_null_tiers
---------+---------------+----------------
 bronze  |             1 |              1
 gold    |             2 |              2
 silver  |             2 |              2
 NULL    |             1 |              0
```

`COUNT(*)` counts rows. `COUNT(column)` counts rows where that column is not null. That difference is a favourite one-line interview question, and the row where they diverge is the one that matters.

Now the trap. It has cost more candidates an offer than any other single behaviour in SQL.

```sql
-- Find customers whose tier is not used by anyone in Pune or Kochi.
SELECT customer_id, full_name, tier
FROM customers
WHERE tier NOT IN (
    SELECT tier FROM customers WHERE city IN ('Pune', 'Kochi')
);
```

```
 customer_id | full_name | tier
-------------+-----------+------
(0 rows)
```

Zero rows. Not an error, not a warning. The subquery returns `gold`, `gold`, and one `NULL`, because Meera Nair in Kochi has no tier. `NOT IN` expands into a chain of inequality comparisons, one of which is `tier <> NULL`, which is unknown, so the whole condition can never be definitely true for any row.

The fix is `NOT EXISTS`, which is safe with nulls because it asks a different question — does a matching row exist — rather than comparing values:

```
SELECT c.customer_id, c.full_name, c.tier
FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM customers p
    WHERE p.city IN ('Pune', 'Kochi')
      AND p.tier = c.tier
);
```

```
 customer_id | full_name    | tier
-------------+--------------+--------
         102 | Rahul Verma  | silver
         104 | Vikram Rao   | silver
         105 | Meera Nair   | NULL
         106 | Arjun Das    | bronze
```

> **COMMON BEGINNER MISTAKES WITH NULL**
>
> - Using `= NULL` instead of `IS NULL`. The first is never true.
> - Using `NOT IN` against a subquery whose column is nullable. Prefer `NOT EXISTS`, or filter the nulls out of the subquery explicitly.
> - Assuming `SUM` of an all-null column is zero. It is `NULL`. Wrap it: `COALESCE(SUM(x), 0)`.
> - Forgetting that `COUNT(column)` skips nulls while `COUNT(*)` does not.
> - Grouping by a nullable column and being surprised that all the nulls collapse into one group.

3.5 Aggregation, and the word that matters most

Grain is what one row of a result means. It is the most important idea in this chapter, in Chapter 5, and arguably in the book.

Before writing an aggregate query, finish this sentence out loud: *one row of my output is one ____ per ____*. One row is one city per month. One row is one customer per product category. If you cannot finish the sentence, the query is not ready to be written, and any number it produces is a coincidence.

`GROUP BY` sets the grain. Everything in `SELECT` must then either appear in `GROUP BY` or be wrapped in an aggregate, because anything else has no single value at that grain — the database is not being pedantic, it does not know which of the five available values you meant.

```sql
-- Grain: one row per city per order status.
SELECT c.city, o.status,
       COUNT(*)                       AS orders,
       COUNT(DISTINCT o.customer_id)  AS customers,
       SUM(o.order_total)             AS revenue,
       ROUND(AVG(o.order_total), 2)   AS avg_order_value
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
GROUP BY c.city, o.status
ORDER BY c.city, revenue DESC;
```

Note `COUNT(DISTINCT ...)`. Counting rows and counting distinct entities are different questions, and mixing them up is how "we have 41,000 customers" becomes a number nobody can reproduce.

3.6 Joins, and the bug from the first page

Most people are taught joins as "combining two tables". That framing is where the trouble starts. A better one:

WHAT A JOIN REALLY DOES

A join takes every row on the left and finds all matching rows on the right. If a left row matches three right rows, it appears three times in the result. The join type only decides what happens to rows that match nothing.

That single sentence — *if it matches three, it appears three times* — is the whole explanation for the wrong query at the start of the chapter.

Join type	Keeps unmatched left rows	Keeps unmatched right rows
`INNER JOIN`	No	No
`LEFT JOIN`	Yes	No
`RIGHT JOIN`	No	Yes
`FULL OUTER JOIN`	Yes	Yes
`CROSS JOIN`	Every left row paired with every right row	

The fan-out

Order 9001 has two order lines. Order 9006 has two. When we join `orders` to `order_items`, those orders appear twice in the result, and their `order_total` is therefore counted twice.

```
-- WRONG. order_total is at order grain, the join is at line grain.
SELECT SUM(o.order_total) AS revenue
FROM orders o
JOIN order_items i ON i.order_id = o.order_id
WHERE o.status = 'DELIVERED';
--  revenue = 44036.00

-- RIGHT, option 1: stay at order grain — do not join at all.
SELECT SUM(o.order_total) AS revenue
FROM orders o
WHERE o.status = 'DELIVERED';
--  revenue = 30390.00

-- RIGHT, option 2: work at line grain and sum a line-grain column.
SELECT SUM(i.line_amount) AS revenue
FROM orders o
JOIN order_items i ON i.order_id = o.order_id
WHERE o.status = 'DELIVERED';
--  revenue = 30390.00
```

A 44.9% overstatement, from a query with no syntax error and no obvious flaw. The rule that prevents it forever:

THE RULE

Never aggregate a column at a grain finer than the column's own. `order_total` belongs to an order, so it may only be summed in a result where each order appears once. If you must join to a finer table, either aggregate that table first, or sum a column that lives at the finer grain.

The safest general pattern is to aggregate before joining:

```
SELECT c.city,
       SUM(o.order_total) AS order_revenue,
       SUM(li.items)      AS items_sold
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
LEFT JOIN (
    SELECT order_id, SUM(quantity) AS items
    FROM order_items
    GROUP BY order_id                  -- collapsed to one row per order
) li ON li.order_id = o.order_id
WHERE o.status = 'DELIVERED'
GROUP BY c.city;
```

`LEFT JOIN`, and where the filter goes

A `LEFT JOIN` keeps every left row. Putting a condition on the right table in `WHERE` silently destroys that, because `WHERE` runs after the join and a null right-hand row fails the test.

```
-- Keeps Kochi, which has no delivered orders. Condition is in ON.
SELECT c.city,
       COUNT(DISTINCT c.customer_id)   AS customers,
       COUNT(o.order_id)               AS orders,
       COALESCE(SUM(o.order_total), 0) AS revenue
FROM customers c
LEFT JOIN orders o
       ON o.customer_id = c.customer_id
      AND o.status = 'DELIVERED'
GROUP BY c.city
ORDER BY revenue DESC;
```

```
 city      | customers | orders | revenue
-----------+-----------+--------+----------
 Pune      |         2 |      4 | 13698.00
 Bengaluru |         2 |      7 | 12894.00
 Hyderabad |         1 |      1 |  3798.00
 Kochi     |         1 |      0 |     0.00
```

Move `AND o.status = 'DELIVERED'` into a `WHERE` clause and Kochi disappears, turning your left join into an inner join without telling you. **In a `LEFT JOIN`, conditions on the right table belong in `ON`; conditions on the left table belong in `WHERE`.**

Also note `COUNT(o.order_id)` rather than `COUNT(*)`. For Kochi, the left join produces one row with nulls on the right; `COUNT(*)` would report 1 order where there are none.

3.7 CTEs

A common table expression names a result so the rest of the query can use it. It does not usually make a query faster; it makes it *readable*, which is what lets you find bugs like the fan-out.

```
WITH delivered AS (
    SELECT * FROM orders WHERE status = 'DELIVERED'
),
per_customer AS (
    SELECT customer_id,
           COUNT(*)          AS orders,
           SUM(order_total)  AS revenue
    FROM delivered
    GROUP BY customer_id
)
SELECT c.full_name, c.city, p.orders, p.revenue
FROM per_customer p
JOIN customers c ON c.customer_id = p.customer_id
ORDER BY p.revenue DESC;
```

Write in stages, and check each stage. In an interview, a query built from three named CTEs that you can explain beats a single clever nested statement that you cannot, every time.

3.8 Window functions

This is the section that separates intermediate SQL from interview SQL.

An aggregate collapses many rows into one. A **window function** computes across a set of rows and returns a value **for every row, without collapsing anything**. That one difference unlocks running totals, rankings, comparisons with the previous row, and most of the patterns in section 3.8.

The syntax has three parts inside `OVER`:

```
function() OVER (
    PARTITION BY <restart the calculation for each of these>
    ORDER BY    <the order rows are considered in>
    <frame>     <which rows around the current one to include>
)
```

`PARTITION BY` is like `GROUP BY` except nothing collapses. `ORDER BY` inside `OVER` is not about output order, it is about the sequence used for the calculation. The frame decides how much of the partition is visible from the current row.

The ranking family

```sql
SELECT o.order_id, c.full_name, o.order_total,
       RANK()       OVER (ORDER BY o.order_total DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY o.order_total DESC) AS dense_rnk,
       ROW_NUMBER() OVER (ORDER BY o.order_total DESC) AS row_num
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
WHERE o.status = 'DELIVERED'
ORDER BY o.order_total DESC;
```

```
 order_id | full_name    | order_total | rnk | dense_rnk | row_num
----------+--------------+-------------+-----+-----------+---------
     9009 | Arjun Das    |     5099.00 |   1 |         1 |       1
     9001 | Priya Sharma |     4749.00 |   2 |         2 |       2
     9004 | Aisha Khan   |     4200.00 |   3 |         3 |       3
     9006 | Vikram Rao   |     3798.00 |   4 |         4 |       4
     9000 | Priya Sharma |     3499.00 |   5 |         5 |       5
     9010 | Rahul Verma  |     3499.00 |   5 |         5 |       6
     9007 | Priya Sharma |     1250.00 |   7 |         6 |       7
```

Look only at the two tied rows and the one after them. `RANK` gives both ties 5 and then jumps to 7. `DENSE_RANK` gives both 5 and continues at 6. `ROW_NUMBER` refuses to acknowledge the tie at all and assigns 5 and 6 arbitrarily.

That last point matters more than it looks. **`ROW_NUMBER` over a non-unique ordering is non-deterministic** — run it twice and you may get a different winner. If you are using it to deduplicate, add a tiebreaker to the `ORDER BY` so the result is reproducible.

Use	Function
Pick exactly one row per group	`ROW_NUMBER`, with a deterministic tiebreaker
Top N including ties, with gaps after	`RANK`
Top N including ties, without gaps	`DENSE_RANK`

Frames

The frame clause decides which rows the function can see. Two forms cover almost everything:

- `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` — everything from the start of the partition up to here. This is a running total.
- `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW` — a sliding window of seven rows.

There is one distinction worth knowing because it appears in senior interviews. `ROWS` counts physical rows; `RANGE` counts by value. For a rolling seven-*day* average over data with missing days, `ROWS BETWEEN 6 PRECEDING` gives you the previous six *rows*, which may span three weeks. `RANGE BETWEEN INTERVAL '6 days' PRECEDING` gives you the previous six *days*, which is almost always what was actually asked for.

3.9 The patterns interviewers actually ask

Eight problems. These recur across companies because each one tests a distinct idea. Every solution below was run against the dataset in section 3.1 and the outputs are real.

Pattern 1 — Deduplication

Problem. The payments table has a duplicate: order 9001 was recorded twice, forty seconds apart. Keep the earliest payment per order.

```
WITH ranked AS (
    SELECT p.*,
           ROW_NUMBER() OVER (
               PARTITION BY p.order_id
               ORDER BY p.paid_ts, p.payment_id   -- deterministic tiebreak
           ) AS rn
    FROM payments p
)
SELECT payment_id, order_id, paid_ts, amount
FROM ranked
WHERE rn = 1
ORDER BY order_id;
```

Thirteen payment rows become twelve; `payment_id` 5009 is dropped. **Follow-up an interviewer will ask:** *what if you want to keep the row with the largest amount instead?* Change the `ORDER BY` inside `OVER` to `amount DESC`. The pattern does not change, only the definition of "best". **Second follow-up:** *why not `SELECT DISTINCT`?* Because the rows are

not identical — they have different `payment_id` and `paid_ts` values — so `DISTINCT` keeps both. Deduplication almost always means "one row per key", not "remove identical rows".

Pattern 2 — Top N per group

Problem. The two highest-revenue products in each category.

```
WITH product_rev AS (
    SELECT pr.category, pr.product_name,
           SUM(i.line_amount) AS revenue
    FROM order_items i
    JOIN products pr ON pr.product_id = i.product_id
    JOIN orders   o  ON o.order_id    = i.order_id
    WHERE o.status = 'DELIVERED'
    GROUP BY pr.category, pr.product_name
),
ranked AS (
    SELECT *,
           DENSE_RANK() OVER (PARTITION BY category
                              ORDER BY revenue DESC) AS rnk
    FROM product_rev
)
SELECT category, product_name, revenue
FROM ranked
WHERE rnk <= 2
ORDER BY category, revenue DESC;
```

```
 category | product_name   | revenue
----------+----------------+----------
 Fitness  | Yoga Mat       |  5000.00
 Fitness  | Water Bottle   |  1798.00
 Footwear | Running Shoes  | 13996.00
 Footwear | Cotton Socks   |  1196.00
 Outdoor  | Trail Backpack |  8400.00
```

The window function cannot go in `WHERE`, because `WHERE` runs before `SELECT`. That is why the ranking needs its own CTE. Snowflake, Databricks and DuckDB offer `QUALIFY` as a shortcut for exactly this; PostgreSQL does not, so the CTE form is the portable answer and the one to give unless you are asked about a specific engine.

Pattern 3 — Running total

```
SELECT CAST(order_ts AS DATE) AS order_date,
       SUM(order_total)       AS daily_revenue,
       SUM(SUM(order_total)) OVER (ORDER BY CAST(order_ts AS DATE))
```

```
           AS running_total
FROM orders
WHERE status = 'DELIVERED'
GROUP BY 1
ORDER BY 1;
```

```
 order_date | daily_revenue | running_total
------------+---------------+---------------
 2026-02-25 |       3499.00 |       3499.00
 2026-03-01 |       5999.00 |       9498.00
 2026-03-04 |       4200.00 |      13698.00
 2026-03-07 |        598.00 |      14296.00
 2026-03-09 |       3798.00 |      18094.00
 ...        |               |
 2026-03-22 |        299.00 |      30390.00
```

`SUM(SUM(...)) OVER (...)` looks wrong and is correct. The inner `SUM` aggregates to daily grain; the window function then runs over those already-aggregated rows, because window functions are evaluated after `GROUP BY`. This exact construction is a common interview question, precisely because it forces you to know the evaluation order from section 3.3.

Pattern 4 — Rolling seven-day average

```
WITH daily AS (
    SELECT CAST(order_ts AS DATE) AS order_date,
           SUM(order_total)       AS revenue
    FROM orders
    WHERE status = 'DELIVERED'
    GROUP BY 1
)
SELECT order_date, revenue,
       ROUND(AVG(revenue) OVER (
           ORDER BY order_date
           RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW
       ), 0) AS avg_7d
FROM daily
ORDER BY order_date;
```

```
 order_date | revenue | avg_7d
------------+---------+--------
 2026-02-25 | 3499.00 |   3499
 2026-03-01 | 5999.00 |   4749
 2026-03-04 | 4200.00 |   5100
```

```
 2026-03-07 |  598.00 |   3599
 2026-03-09 | 3798.00 |   2865
 2026-03-12 | 1250.00 |   1882
```

`RANGE` is deliberate. Our data has gaps — no orders on many days — so `ROWS BETWEEN 6 PRECEDING` would average the last six *rows*, reaching back nearly a month. **Follow-up:** *what if the business wants missing days counted as zero?* Then generate a complete date series and left join the daily totals onto it, because a window function cannot invent rows that are not there.

Pattern 5 — Month-over-month change

```sql
WITH monthly AS (
    SELECT DATE_TRUNC('month', order_ts) AS mth,
           SUM(order_total)              AS revenue
    FROM orders
    WHERE status = 'DELIVERED'
    GROUP BY 1
)
SELECT CAST(mth AS DATE) AS month, revenue,
       LAG(revenue) OVER (ORDER BY mth) AS prev_month,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY mth))
             / LAG(revenue) OVER (ORDER BY mth), 1) AS pct_change
FROM monthly
ORDER BY mth;
```

```
 month      | revenue  | prev_month | pct_change
------------+----------+------------+------------
 2026-02-01 |  3499.00 |     NULL   |      NULL
 2026-03-01 | 26891.00 |    3499.00 |     668.5
```

The first row is `NULL` because there is no previous month, which is correct and should be left alone rather than papered over with a zero. **Follow-up:** *what breaks if a month has no orders at all?* `LAG` returns the previous *row*, not the previous month, so a gap silently compares March to January while labelling it month-over-month. The fix is again a generated date spine.

Pattern 6 — Gaps and islands

Problem. Find runs of consecutive days on which a customer ordered.

```sql
WITH active AS (
    SELECT DISTINCT customer_id, CAST(order_ts AS DATE) AS d
    FROM orders
    WHERE status = 'DELIVERED'
),
grouped AS (
    SELECT customer_id, d,
           d - CAST(ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY d)
                    AS INTEGER) AS island
    FROM active
)
SELECT customer_id,
       MIN(d) AS streak_start,
       MAX(d) AS streak_end,
       COUNT(*) AS streak_days
FROM grouped
GROUP BY customer_id, island
HAVING COUNT(*) >= 2
ORDER BY streak_days DESC;
```

```
 customer_id | streak_start | streak_end | streak_days
-------------+--------------+------------+-------------
         102 | 2026-03-18   | 2026-03-20 |           3
```

The trick is worth understanding rather than memorising. Consecutive dates increase by one each row, and `ROW_NUMBER` also increases by one each row. Subtract one from the other and every date in a consecutive run produces the *same* constant; the moment a day is skipped, the constant changes. Grouping by that constant groups by run. Once you have seen it, you have it for life, and it appears in interviews as streaks, consecutive logins, uninterrupted subscription periods and downtime windows.

Pattern 7 — Sessionization

Problem. Group app events into sessions, where a gap of more than thirty minutes starts a new one.

```sql
WITH with_prev AS (
    SELECT customer_id, event_ts, event_name,
           LAG(event_ts) OVER (PARTITION BY customer_id
                               ORDER BY event_ts) AS prev_ts
    FROM app_events
),
marked AS (
    SELECT *,
           CASE WHEN prev_ts IS NULL
```

```
                  OR event_ts > prev_ts + INTERVAL '30 minutes'
                THEN 1 ELSE 0 END AS is_new_session
    FROM with_prev
),
sessions AS (
    SELECT *,
           SUM(is_new_session) OVER (
               PARTITION BY customer_id ORDER BY event_ts
           ) AS session_no
    FROM marked
)
SELECT customer_id, session_no,
       MIN(event_ts) AS started,
       MAX(event_ts) AS ended,
       COUNT(*)      AS events
FROM sessions
GROUP BY customer_id, session_no
ORDER BY customer_id, session_no;
```

```
 customer_id | session_no | started             | ended               |
events
-------------+------------+---------------------+---------------------
+--------
         101 |          1 | 2026-03-01 10:02:00 | 2026-03-01 10:12:00 |
4
         101 |          2 | 2026-03-01 13:40:00 | 2026-03-01 13:44:00 |
2
         102 |          1 | 2026-03-01 18:30:00 | 2026-03-01 18:44:00 |
2
```

Three ideas stacked: `LAG` to see the previous event, a `CASE` to flag boundaries, and a running `SUM` of that flag to number the sessions. The running sum turning a series of flags into group identifiers is the same manoeuvre as the islands trick, and once you notice they are the same idea you will start seeing it everywhere.

Pattern 8 — Anti-join

Problem. Which orders have no payment?

```
SELECT o.order_id, o.status, o.order_total
FROM orders o
LEFT JOIN payments p ON p.order_id = o.order_id
WHERE p.order_id IS NULL
ORDER BY o.order_id;
```

```
 order_id | status    | order_total
----------+-----------+-------------
     9003 | CANCELLED |     3499.00
     9008 | RETURNED  |      899.00
```

Both are explainable — a cancelled order and a returned one — which is what makes this a good reconciliation check rather than an alarm. `NOT EXISTS` is the other correct form and is often clearer. `NOT IN` is the third form and the one to avoid, for the reason in section 3.4.

3.10 Making a query fast

Correctness first, always. But an interviewer will ask why a query is slow, and there are four answers that cover most cases.

It is reading more than it needs. `SELECT *` on a wide table in a columnar warehouse reads every column. Name the columns you want. In a warehouse this is not a style preference, it is most of the bill.

The filter cannot use the index. A predicate that wraps a column in a function usually forces a full scan. Write `order_ts >= DATE '2026-03-01' AND order_ts < DATE '2026-04-01'` rather than `WHERE YEAR(order_ts) = 2026 AND MONTH(order_ts) = 3`. The two are logically identical and only one can use an index on `order_ts`. A predicate the index can use is called *sargable*, and the word occasionally appears in interviews.

It is joining more rows than necessary. Filter and aggregate before joining, not after. The fan-out pattern from section 3.6 is a correctness bug and a performance bug at once.

The plan is not what you assumed. Run `EXPLAIN` — or `EXPLAIN ANALYZE`, which executes the query and reports actual timings — and read what the engine actually chose. The two things to look for first are a sequential scan where you expected an index, and an estimated row count that is wildly different from the actual one, which usually means statistics are stale.

WHAT HAPPENS WHEN YOU RUN A QUERY

The database parses your SQL into a tree and checks that the tables and columns exist. The rewriter expands views and simplifies expressions. The planner then generates candidate execution strategies — which table to read first, which join algorithm, whether to use an index — and estimates the cost of each using stored statistics about table sizes and value distributions. It picks the cheapest estimate and hands that plan to the executor.

Two consequences follow. Your SQL describes *what* you want, not *how*, so rewriting a query into a logically identical form often changes nothing at all. And when the statistics are stale, the planner makes good decisions from bad information, which is why a query can suddenly become slow after a large load with no code change.

3.11 Mistakes to stop making now

COMMON BEGINNER MISTAKES

- Writing the query before deciding the grain. Say the sentence first.
- Summing an order-grain column after joining to a line-grain table.
- Putting a right-table condition in `WHERE` on a `LEFT JOIN`, converting it to an inner join by accident.
- `NOT IN` against anything nullable.
- `SELECT *` in production code, especially on a columnar warehouse.
- `ROW_NUMBER` with a non-unique `ORDER BY` and no tiebreaker.
- `ROWS` when the question was about days and the data has gaps.
- Trusting a result because it ran. Check the row count against something you already know.

Summary

- The engine evaluates `FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT`. Most beginner confusion dissolves once that order is memorised.
- `NULL` means unknown. `NOT IN` against a nullable subquery returns nothing at all, silently. Use `NOT EXISTS`.
- Grain is what one output row means. Say it out loud before writing the query.
- A join repeats a left row once per match. Never sum a column at a grain finer than the column's own.

- On a `LEFT JOIN`, right-table conditions go in `ON`, left-table conditions go in `WHERE`.
- Window functions compute per row without collapsing. `ROW_NUMBER` for one-per-group, `RANK` and `DENSE_RANK` for top-N with ties handled differently.
- `ROWS` counts rows, `RANGE` counts values. With gappy dates, `RANGE` is usually what was meant.
- The islands trick and the sessionization trick are the same idea: turn boundary flags into group identifiers with a running sum.

PRACTICE

1. Build the schema in DuckDB and load the seed data. Reproduce the fan-out bug and both fixes yourself. Do not move on until the 44,036 and the 30,390 both appear on your screen.
2. Write the sentence "one row of my output is one ____ per ____" for every query in section 3.9, before you read the query.
3. Rewrite Pattern 2 using `RANK` instead of `DENSE_RANK`. Explain the difference in the result, and then construct data where the two disagree.
4. Break Pattern 5 on purpose: delete March's orders and confirm that `LAG` compares February to April while still calling it month-over-month.
5. Take Pattern 6 and adapt it to find the longest streak of consecutive days a customer did *not* order.
6. Run `EXPLAIN` on the fan-out query and the corrected version. Compare the row estimates.

SOURCES AND VERIFICATION

Every query in this chapter was executed against the dataset in section 3.1 using DuckDB, and every result block shows the actual output rather than an expected one. Where behaviour differs between engines it is called out in the text: `QUALIFY` is available in Snowflake, Databricks and DuckDB but not in PostgreSQL, and interval syntax in `RANGE` frames varies slightly between dialects.
Reference material: PostgreSQL documentation on `SELECT`, window functions and `EXPLAIN`, which remains the clearest free explanation of evaluation order and query planning available.

4

CHAPTER FOUR

Python for Data Engineers

This is not a Python course. A Python course teaches you classes, decorators, generators, metaclasses and sorting algorithms, and after four months you still cannot load a file into a database without it breaking on Tuesday. This chapter teaches the subset of Python that a data engineer uses, in the order the job uses it, and it ends with a working ingestion script you can run.

> VERSIONS AND DIALECTS
>
> Python in this chapter is **Python 3** standard library, plus `requests`, `pandas` and `sqlite3`. It was executed as printed.

Everything here was executed. Every output block is what actually appeared on the terminal.

Here is the shape of what we are building towards, and it is the shape of most data engineering code you will ever write:

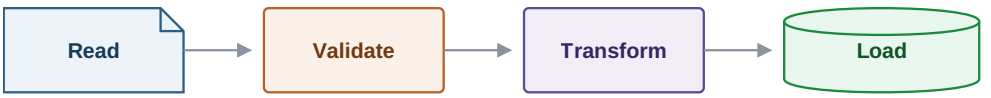


FIGURE 4.1 The anatomy of nearly every ingestion script. Chapter 8 turns this into a pipeline; Chapter 10 puts it on a schedule.

4.1 What Python is actually for here

SQL transforms data that is already inside a database. Python is what gets it there, and what handles everything a database cannot: calling APIs, reading files in awkward formats, retrying when a network fails, writing logs, and orchestrating the steps.

So the Python you need is narrow and deep rather than broad. You need to be excellent at reading files, handling dictionaries, catching the right errors, and talking to databases. You almost never need inheritance hierarchies, and you will write perhaps three classes a year.

THE TEST FOR WHETHER YOU KNOW ENOUGH PYTHON

Can you write a script that reads a file, rejects the bad rows without crashing, loads the good ones into a database, logs what it did, and returns a non-zero exit code when something goes wrong? That is the bar. It is lower than a computer science syllabus and higher than most tutorials get you.

4.2 The four types that carry everything

You will meet others. These four do ninety per cent of the work.

A **string** is text. Everything read from a file or an API starts as a string, including numbers, and converting them is your job.

```
total = "4749.00"          # this is text, not a number
print(total + "1")         # 4749.001   <- string concatenation
print(float(total) + 1)    # 4750.0     <- now it is arithmetic
```

That trap catches everyone once. A CSV column of amounts is text until you convert it, and `"10" > "9"` is `False` because strings compare alphabetically.

A **list** is an ordered collection you can add to.

```
cities = ["Pune", "Bengaluru", "Pune", "Kochi"]
print(len(cities))          # 4
print(cities[0])            # Pune       <- counting starts at zero
print(cities[-1])           # Kochi      <- negative counts from the end
cities.append("Hyderabad")
```

A **dictionary** maps keys to values, and it is the single most important type in data engineering. Every row you read, every JSON message, every API response arrives as one.

```
order = {"order_id": 9001, "city": "Pune", "total": 4749.00}
print(order["city"])              # Pune
print(order.get("coupon"))        # None      <- missing key, no crash
print(order.get("coupon", "-"))   # -         <- with a default
```

The difference between `order["coupon"]` and `order.get("coupon")` matters more than it looks. The first raises `KeyError` and stops your pipeline. The second returns `None` and carries on. Which one you want depends on whether a missing coupon is a bug or normal, and deciding that deliberately is the difference between a script and a pipeline.

A **set** holds unique values and answers "have I seen this before" instantly.

```
seen = set()
for order_id in [9001, 9002, 9001]:
    if order_id in seen:
        print("duplicate:", order_id)     # duplicate: 9001
    seen.add(order_id)
```

4.3 Reading a file without breaking it

Here is a real file with a real problem in it. One customer's name contains a comma, correctly quoted:

```
order_id,customer_name,order_date,city,status,order_total
9001,"Sharma, Priya",2026-03-01,Pune,DELIVERED,4749.00
9002,Rahul Verma,2026-03-01,Bengaluru,DELIVERED,1250.00
```

The instinct of every beginner is to split on the comma:

```
with open("orders.csv") as f:
    next(f)                                  # skip the header
    for line in f:
        parts = line.strip().split(",")
        print(len(parts), parts[1])
```

```
7 "Sharma
6 Rahul Verma
7 "Sharma
6 Aisha Khan
6 Rahul Verma
```

Two different row lengths from one file. The quoted comma split the name in half and shifted every following column by one, so `city` now holds a date and `order_total` holds a city. Nothing crashed. The load would have succeeded and every downstream number would be wrong — exactly the silent failure Chapter 1 warned about.

The `csv` module understands quoting, escaping and embedded newlines, because those rules are fiddly and somebody has already solved them:

```
import csv

with open("orders.csv", newline="", encoding="utf-8") as f:
```

```
    for row in csv.DictReader(f):
        print(row["order_id"], "|", row["customer_name"],
              "|", row["order_total"])
```

```
9001 | Sharma, Priya | 4749.00
9002 | Rahul Verma | 1250.00
9003 | Sharma, Priya | 3499.00
9004 | Aisha Khan | 4200.00
9005 | Rahul Verma | 598.00
```

Four details in that snippet are deliberate.

`with open(...)` is a **context manager**. It closes the file when the block ends, even if an exception is thrown. Files opened without it leak, and a long-running job that leaks file handles eventually dies with a confusing error about too many open files.

`newline=""` is required by the `csv` module so it can handle newlines inside quoted fields itself.

`encoding="utf-8"` should always be explicit. The default varies by operating system, so a script that works on your laptop can fail on a server the first time a customer's name contains a non-ASCII character.

`DictReader` gives you a dictionary per row keyed by column name, so your code says `row["city"]` rather than `row[3]`. When someone adds a column upstream, position-based code silently reads the wrong values and name-based code keeps working.

Now something useful — revenue by city, in nine lines:

```
import csv
from collections import defaultdict

revenue = defaultdict(float)          # missing keys default to 0.0

with open("orders.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        if row["status"] != "DELIVERED":
            continue
        revenue[row["city"]] += float(row["order_total"])

for city, total in sorted(revenue.items(), key=lambda kv: -kv[1]):
    print(f"{city:<12} {total:>10,.2f}")
```

```
Pune           8,949.00
Bengaluru      1,848.00
```

`defaultdict(float)` saves you from checking whether a key exists before adding to it. The `f"{city:<12}"` syntax is an f-string with formatting: left-align in twelve characters, and for the number, right-align in ten with thousands separators and two decimals.

4.4 JSON and nested data

APIs and event streams speak JSON. Python turns it into dictionaries and lists, which you already know.

```
{"order_id": 9001, "customer": {"id": 101, "tier": "gold"},
 "items": [{"sku": "SHO-4471", "qty": 1, "price": 3499.0},
           {"sku": "MAT-2210", "qty": 1, "price": 1250.0}]}
```

```
import json

with open("event.json", encoding="utf-8") as f:
    event = json.load(f)                # load() reads a file

print("order:", event["order_id"])
print("tier :", event["customer"]["tier"])
print("coupon:", event.get("coupon", "none"))
print("total:", sum(i["qty"] * i["price"] for i in event["items"]))
```

```
order: 9001
tier : gold
coupon: none
total: 4749.0
```

Nested access chains with square brackets: `event["customer"]["tier"]`. The last line is a **generator expression** inside `sum` — read it as “for each item, multiply quantity by price, and add them all up”. You will use that idiom constantly to flatten a nested structure into a number.

And the trap, which is the same one as section 4.2 but nested:

```
event["coupon"]        # KeyError: 'coupon'
```

Two functions to remember: `json.load(file)` and `json.loads(string)`, with the `s` meaning string. Their opposites are `json.dump` and `json.dumps`. Mixing them up is the most common JSON error in Python and the error message does not make it obvious.

> **COMMON BEGINNER MISTAKES WITH FILES**
>
> - Splitting a CSV on commas instead of using the `csv` module.
> - Reading by column position instead of column name.
> - Leaving the encoding unspecified until a name with an accent breaks it in production.
> - Reading a whole file into memory with `f.read()` when you only need one line at a time.
> - Assuming every JSON message has every field. They will not.

4.5 Functions, and the shape of a script

A function is a named block that takes inputs and returns an output. Two reasons they matter here, and neither is elegance.

You can test a function without running the whole pipeline. And you can read a script that is five well-named functions in a way you cannot read two hundred lines of top-level code.

```
def parse_amount(value: str, default: float = 0.0) -> float:
    """Convert a CSV amount to a float, or return the default."""
    try:
        return float(value)
    except (ValueError, TypeError):
        return default
```

The `-> float` and `value: str` are **type hints**. Python does not enforce them, but they document intent and your editor uses them to catch mistakes before you run anything. Use them on function signatures; do not bother elsewhere.

Every script should end with this pattern:

```
def main(argv):
    ...
    return 0                        # 0 means success

if __name__ == "__main__":
    sys.exit(main(sys.argv))
```

The `if __name__ == "__main__":` guard means the file can be imported by a test without executing. And `sys.exit(main(...))` sets the process exit code from the return value, which is what Lecture Zero showed the scheduler actually reads.

4.6 Errors: catch what you expect, keep going, exit loudly

There are two bad ways to handle errors and one good one.

The first bad way is to catch nothing, so one malformed row out of four million kills the whole job at 3 a.m. The second is worse:

```
try:
    process(row)
except:                  # never do this
    pass
```

That catches everything, including the mistakes in your own code, hides all of them, and reports success. It is the single most damaging four lines a data engineer can write.

The good way names the errors you expect, records what it rejected, and continues:

```
def read_orders(path):
    good, rejected = [], 0
    with open(path, newline="", encoding="utf-8") as f:
        for line_no, row in enumerate(csv.DictReader(f), start=2):
            try:
                row["order_total"] = float(row["order_total"])
            except ValueError:
                log.warning("line %d rejected: bad amount %r",
                            line_no, row["order_total"])
                rejected += 1
                continue
            good.append(row)
    return good, rejected
```

Run it against a file with two deliberately broken rows — one with text where a number should be and one with an empty amount:

```
14:43:39 WARNING line 7 rejected: could not convert string to
                 float: 'not_a_number'
14:43:39 WARNING line 8 rejected: could not convert string to float: ''
14:43:39 INFO    loaded 5 rows, rejected 2
```

Five good rows loaded, two quarantined and named. `enumerate(..., start=2)` numbers the rows from two so the line numbers match what you would see in a text editor, header included — a small thing that saves real time when someone asks which row was bad.

The remaining question is what to do when the rejection rate is high. Two bad rows out of seven is fine to log; two hundred thousand out of four hundred thousand means the source changed and continuing quietly is the wrong call. Chapter 12 covers thresholds properly. For now, log the count and make sure a human can see it.

4.7 Logging, not printing

`print` writes to the screen. Nobody is watching the screen at 3 a.m. Logging gives you timestamps, severity levels, and somewhere for the output to go.

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)-5s %(name)s | %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
log = logging.getLogger("ingest")

log.info("starting ingest for %s", "2026-03-17")
log.info("read %d rows from %s", 5, "orders.csv")
log.warning("row 3 has status %s, skipping", "CANCELLED")
log.error("could not parse order_total on row 7")
```

```
2026-09-02 14:43:07 INFO  ingest | starting ingest for 2026-03-17
2026-09-02 14:43:07 INFO  ingest | read 5 rows from orders.csv
2026-09-02 14:43:07 WARNING ingest | row 3 has status CANCELLED, skipping
2026-09-02 14:43:07 ERROR ingest | could not parse order_total on row 7
```

Note the commas: `log.info("read %d rows", 5)` rather than an f-string. The formatting is deferred, so if the level is turned off the string is never built. It also keeps the message template constant, which is what lets log aggregation tools group thousands of similar messages together.

Four levels are enough. `DEBUG` for things you want while developing, `INFO` for milestones, `WARNING` for something survivable that a human should know about, `ERROR` for a failure.

WHAT TO LOG IN EVERY PIPELINE

Start and end, with the date being processed. Input row count. Output row count. Rejected row count. Duration. Those five numbers turn “the job ran” into “the job ran and here is why I believe the result”, and they are what you will wish you had when Chapter 12’s incident arrives.

4.8 pandas, and where it stops

`pandas` gives you a table in memory with SQL-like operations. It is superb for files up to a few gigabytes, and it is the wrong tool above that.

```
import pandas as pd

df = pd.read_csv("orders.csv")
print("shape:", df.shape)
print(df.dtypes.to_string())
```

```
shape: (5, 6)
order_id           int64
customer_name        str
order_date           str
city                 str
status               str
order_total      float64
```

Check `dtypes` every single time. Notice `order_date` came in as text, not a date — pandas will not guess, and comparing dates as strings works by accident right up until it does not. Convert explicitly with `pd.to_datetime(df["order_date"])`.

The grouping you wrote by hand in section 4.3 is one line here:

```
out = (df[df.status == "DELIVERED"]
         .groupby("city", as_index=False)["order_total"].sum()
         .sort_values("order_total", ascending=False))
print(out.to_string(index=False))
```

```
     city  order_total
     Pune       8949.0
Bengaluru       1848.0
```

Same answer as the nine-line version, and considerably faster on a large file because the work happens in compiled code rather than a Python loop.

Now the limit. `pandas` loads everything into memory, and it needs meaningfully more memory than the file size on disk because Python objects carry overhead. You can see it directly:

```
print(df.memory_usage(deep=True).to_string())
```

```
Index            132
order_id          40
customer_name    303
order_date       295
city             275
status           290
order_total       40
```

Five rows of text occupy roughly three hundred bytes per column while the two numeric columns take forty. Text is expensive. A 4 GB CSV of mostly strings will not fit comfortably in an 8 GB machine, which is the moment people discover their laptop is not a data platform.

When the file is too large, read it in pieces:

```
total = 0.0
for chunk in pd.read_csv("orders.csv", chunksize=2):
    total += chunk.loc[chunk.status == "DELIVERED", "order_total"].sum()
    print(f"chunk of {len(chunk)} rows, running total {total:,.2f}")
```

```
chunk of 2 rows, running total 5,999.00
chunk of 2 rows, running total 10,199.00
chunk of 1 rows, running total 10,797.00
```

A `chunksize` of two is absurd for real work; it is two here so you can see the mechanism. In practice you would use fifty thousand or a hundred thousand rows. Chunking works when each row can be handled independently. It does not work when you need to join, sort or deduplicate across the whole dataset, and that limitation is precisely where Chapter 9 and Spark begin.

Data size	Reach for
Under a few hundred MB	Plain Python, or pandas
Up to a few GB, fits in memory	pandas
Larger than memory, row-independent work	pandas with `chunksize`, or DuckDB
Larger than memory, joins and shuffles	Spark (Chapter 9)

4.9 Talking to a database

Two rules, and the first one is not optional.

Never build SQL by joining strings. This is how SQL injection happens, and it also breaks the moment a value contains a quote. Pass parameters:

```
city = "Pune"

# WRONG — never do this
con.execute(f"SELECT SUM(order_total) FROM orders WHERE city = '{city}'")

# RIGHT — the driver handles quoting and escaping
for row in con.execute(
        "SELECT city, SUM(order_total) FROM orders "
        "WHERE city = ? GROUP BY city", (city,)):
    print(row)
```

```
('Pune', 12448.0)
```

The placeholder differs by driver: `?` in SQLite and DuckDB, `%s` in `psycopg` for PostgreSQL. The principle does not change.

Insert in batches, not row by row. Every individual insert is a network round trip. Ten thousand of them is ten thousand round trips.

```
con.executemany("INSERT INTO orders VALUES (?, ?, ?, ?, ?, ?)", rows)
con.commit()
```

Now the part that separates a script from a pipeline. Run that load twice and see what happens:

```
IntegrityError: UNIQUE constraint failed: orders.order_id
```

The primary key stopped a duplicate load, which is good. But your job now fails on every re-run, and pipelines get re-run constantly — after a failure, after a fix, after a backfill. What you want is a load that produces the same result whether it runs once or five times. That property is called **idempotency** and it is one of the most important ideas in this book.

```
con.executemany(
    "INSERT OR REPLACE INTO orders VALUES (?, ?, ?, ?, ?, ?)", rows)
con.commit()
```

```
after upsert re-run: 5
```

Five rows before, five rows after. In PostgreSQL the equivalent is `INSERT ... ON CONFLICT (order_id) DO UPDATE`, and in Spark and Delta Lake it is `MERGE`. Different syntax, same idea, and Chapter 8 develops it fully.

4.10 Virtual environments

A **virtual environment** is a private set of packages for one project, so that upgrading a library for one job does not break another.

```
python3 -m venv .venv           # create it
source .venv/bin/activate       # switch into it (Linux/Mac)
pip install pandas requests     # installs only inside this project
pip freeze > requirements.txt   # record exact versions
```

Commit `requirements.txt`, never commit `.venv/`. When a colleague or a server needs to reproduce your environment, `pip install -r requirements.txt` gives them the same versions you tested against. Skipping this is how “it works on my machine” becomes a genuine production incident.

4.11 Putting it together

Everything above, as one script that loads a daily orders file into a database and is safe to run twice.

```
"""Load a daily orders CSV into SQLite. Safe to run twice."""
import csv
import logging
import sqlite3
```

```
import sys
from pathlib import Path

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)-7s %(message)s",
    datefmt="%H:%M:%S",
)
log = logging.getLogger("ingest")

DDL = """
CREATE TABLE IF NOT EXISTS orders (
    order_id      INTEGER PRIMARY KEY,
    customer_name TEXT,
    order_date    TEXT,
    city          TEXT,
    status        TEXT,
    order_total   REAL
)"""


def read_orders(path: Path):
    """Return (good_rows, rejected). Bad rows are logged, not fatal."""
    good, bad = [], 0
    with path.open(newline="", encoding="utf-8") as f:
        for line_no, row in enumerate(csv.DictReader(f), start=2):
            try:
                good.append((
                    int(row["order_id"]), row["customer_name"],
                    row["order_date"], row["city"], row["status"],
                    float(row["order_total"]),
                ))
            except (ValueError, KeyError) as exc:
                bad += 1
                log.warning("line %d rejected: %s", line_no, exc)
    return good, bad


def load(rows, db_path):
    con = sqlite3.connect(db_path)
    try:
        con.execute(DDL)
        con.executemany(
            "INSERT OR REPLACE INTO orders "
            "VALUES (?, ?, ?, ?, ?, ?)", rows)
        con.commit()
    finally:
        con.close()


def main(argv):
    if len(argv) != 3:
        log.error("usage: ingest.py <orders.csv> <database.db>")
```

```
        return 2
    src = Path(argv[1])
    if not src.exists():
        log.error("input file not found: %s", src)
        return 1

    rows, rejected = read_orders(src)
    if not rows:
        log.error("no valid rows in %s", src)
        return 1

    load(rows, argv[2])
    log.info("loaded %d rows, rejected %d", len(rows), rejected)
    return 0


if __name__ == "__main__":
    sys.exit(main(sys.argv))
```

Running it, twice, and then against a file that does not exist:

```
$ python3 ingest.py orders.csv shopkart.db
14:43:33 INFO    loaded 5 rows, rejected 0
exit=0

$ python3 ingest.py orders.csv shopkart.db
14:43:33 INFO    loaded 5 rows, rejected 0
exit=0  (re-run)

$ rows in table: 5

$ python3 ingest.py missing.csv shopkart.db
14:43:33 ERROR   input file not found: missing.csv
exit=1
```

Seventy lines, and it does eight things that most beginner scripts do not: it parses CSV correctly, reads by column name, quarantines bad rows instead of dying, logs what it did with counts, closes the database connection even on failure, batches its inserts, is safe to re-run, and returns an exit code the scheduler can act on.

That is the standard. Everything in the rest of this book is this script, at larger scale, with better tools.

Summary

- Python's job here is getting data in and out, and handling everything a database cannot.

- Strings, lists, dictionaries and sets carry almost all the work. Every row and every API response is a dictionary.
- Use the `csv` module, always. Splitting on commas breaks silently on quoted fields and shifts every column.
- Read by column name, not position. Set the encoding explicitly.
- `dict["key"]` crashes on missing keys; `dict.get("key")` does not. Choose deliberately.
- Catch the exceptions you expect, log the rejected rows with line numbers, and never write a bare `except`.
- Log with levels and timestamps, and always log input count, output count and rejected count.
- pandas is excellent up to a few gigabytes and stops where memory does. Check `dtypes` every time.
- Parameterise every query, insert in batches, and make the load idempotent so a re-run is safe.
- Pin dependencies in `requirements.txt`. Commit the file, never the environment.

In the interview

You have a 30 GB CSV to load and a machine with 16 GB of memory. How do you approach it?

SHORT ANSWER

I would not load it all at once. First I would check what the work actually requires: if each row can be handled independently, I would stream it in chunks and write out as I go, so memory stays flat regardless of file size. If it needs joins or sorting across the whole dataset, chunking will not save me and I would move it to something distributed.

DEEPER ANSWER

Before any of that I would look at the file. Row count, column types, and how much of it is text, because strings dominate memory and a 30 GB CSV of mostly text needs considerably more than 30 GB as Python objects. Then the cheap wins: read only the columns I need, set explicit dtypes rather than letting them be inferred, and convert repeated strings to a categorical type. Those three often cut the footprint enough that the question disappears.

If it does not, I would convert the file to Parquet once and work from that, because columnar storage lets me read individual columns and gives me compression, so the second pass and every pass after it is dramatically cheaper.

FOLLOW-UP: AT WHAT POINT WOULD YOU REACH FOR SPARK?

When the work needs a shuffle — a join, a global sort, a deduplication across the whole set — and the data does not fit. Chunking handles row-independent work at any size. It cannot handle work where one row's answer depends on a row in another chunk. That boundary, not a file size threshold, is what decides it.

SENIOR-LEVEL ANSWER

I would also ask why the file is 30 GB of CSV in the first place. If it arrives daily in that shape, the durable fix is upstream: land it partitioned and in a columnar format at ingestion, so nobody downstream has this problem again. Solving it once in the pipeline is worth more than solving it repeatedly in every script that reads it.

PRACTICE

1. Create a CSV where one field contains a quoted comma. Parse it both ways — `split(",")` and `csv.DictReader` — and confirm the column shift with your own eyes.
2. Write the revenue-by-city loop from section 4.3 without looking at it. Then rewrite it in pandas and check the two agree.
3. Add a deliberately broken row to your CSV and make your loader quarantine it, log the line number, and still exit zero.
4. Now make it exit non-zero if more than ten per cent of rows are rejected. That single rule is a real data quality check.
5. Run the full `ingest.py` twice and confirm the row count does not change. Then swap `INSERT OR REPLACE` for plain `INSERT` and watch it fail on the second run. Understanding that difference is understanding idempotency.
6. Take any public JSON API, fetch one page, and flatten it into a CSV with the `csv` module. This is your first real ingestion.

SOURCES AND VERIFICATION

Every Python snippet and every output block in this chapter was executed on Python 3 with the standard library, plus pandas for section 4.8 and SQLite for section 4.9. Outputs are the actual terminal results, including the failure cases: the column shift from splitting on commas, the `IntegrityError` on the second load, and the exit codes from `ingest.py`.
Reference material: the Python standard library documentation for `csv`, `json`, `logging` and `sqlite3`, and the pandas user guide on scaling to larger datasets.

PART THREE

Shaping Data

What one row means, where the bytes physically sit, and whose machines they sit on. The decisions that determine whether a company can trust its own numbers — and what they cost.

5 · Data Modeling

6 · Where Data Lives

7 · The Cloud, and What You Are Actually Renting

5

CHAPTER FIVE

Data Modeling

Two dashboards at ShopKart show March revenue for Pune. One says **₹8,949**. The other says **₹4,200**. Both were built by competent people. Both queries are correct SQL. Neither has a bug. The difference is a modeling decision that somebody made eighteen months ago without writing it down, and by section 5.9 you will be able to say exactly which one, and which number the finance team should be looking at.

> VERSIONS AND DIALECTS
>
> SQL in this chapter was executed on **DuckDB 1.5**. The `MERGE` and window syntax shown is standard; SCD Type 2 implementations differ in detail between warehouses.

Data modeling is the least glamorous chapter in this book and the one that decides whether a company can trust its own numbers. It is also where interviews are won. Almost every candidate can write a join. Very few can answer *what is the grain of this table?* without pausing, and interviewers know that, which is why they ask it.

5.1 Grain: what one row means

A model is a set of decisions about meaning. The first and most important decision is the **grain**: what one row of a table represents.

Consider a table called `sales`. What is one row? There are at least three defensible answers, and they produce three different tables with three different row counts and three different correct ways to sum them.



FIGURE 5.1 One table name, three possible grains. Every number anyone ever calculates from this table depends on which one was chosen — and the name gives you no clue.

Our small dataset has four orders made up of five order lines. If one row is an order, the table has four rows and `order_total` is safe to add up. If one row is an order line, the table has five rows and adding `order_total` double-counts the multi-line order. It is the fan-out bug from Chapter 3, except now it is baked into the table rather than introduced by a query, which makes it permanent.

Here is that difference measured, on a fact table where one row is one order line:

```
SELECT COUNT(*)                AS looks_like_orders,
       COUNT(DISTINCT order_id) AS actual_orders
FROM fact_order_line;
```

```
 looks_like_orders | actual_orders
-------------------+---------------
                 5 |             4
```

Anybody who counts rows in that table and calls the result “orders” is **25% wrong**, and nothing in the table name, the column names or the query tells them so.

THE SENTENCE TO SAY OUT LOUD

Before you create any table, finish this sentence: *one row of this table is one* **_ *per* _**. One row is one order line. One row is one customer per month. One row is one product per warehouse per day. If you cannot finish it, you are not ready to write the `CREATE TABLE`, and any number that comes out of it later will be a coincidence.

Two working rules follow, and they will keep you out of most trouble.

Declare the grain in writing, in the table's description and at the top of the code that builds it. A grain that lives only in the head of the person who built the table is a grain that will be misunderstood within a year, usually by that same person.

Pick the finest grain you plausibly need. You can always aggregate a fine grain upwards; you can never recover detail you did not store. Storing order lines lets you answer questions about orders. Storing only orders means the question "which products are usually bought together" is unanswerable forever.

5.2 Entities, attributes, relationships

Modeling starts by naming the things the business cares about.

An **entity** is a thing that exists independently: a customer, a product, an order, a seller, a warehouse. An **attribute** is a property of an entity: a customer's city, a product's category. A **relationship** connects entities: a customer *places* orders, an order *contains* products.

Relationships come in three shapes, and getting the shape right early prevents a lot of rework.

Shape	Example	How it is stored
One to one	A customer has one loyalty account	Either table can hold the other's key
One to many	A customer places many orders	The *many* side holds the key — `orders.customer_id`
Many to many	An order contains many products; a product appears in many orders	A third table between them — `order_items`

That last row explains why `order_items` exists at all. A many-to-many relationship cannot be stored directly in two tables, so it gets a table of its own. Beginners often see `order_items` as an implementation detail; it is in fact the physical representation of a relationship, and it is where the interesting data lives.

> ONE WAY TO PICTURE IT
>
> A model is a floor plan drawn before the building goes up. Changing a wall on paper costs a pencil. Changing it after the concrete is poured costs a month. Data models behave the same way: restructuring a table that four dashboards, two ML features and a finance report already depend on is a quarter's work, and everyone involved will remember who chose the original grain.

5.3 Why operational tables are split up

Look at how the application stores an order. Customer details are in one table, the order header in another, the lines in a third, products in a fourth. Nothing is repeated anywhere.

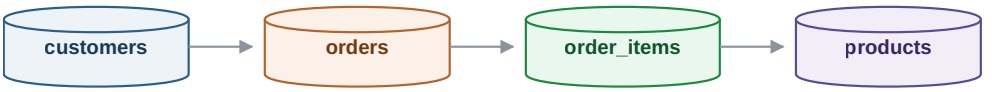


FIGURE 5.2 The operational model. Every fact is stored exactly once, which is what makes it safe to change.

This is **normalization**, and it is usually taught as a list of numbered forms to memorise. That approach makes it feel arbitrary. It is easier to learn by the symptoms it cures.

The update problem. Suppose the customer's city were stored on every order row. Priya has forty orders. She moves house. Now forty rows must be updated together, and if the job fails after twenty, the database contains two different truths about where she lives.

The insertion problem. If product details live only on order rows, a new product cannot exist until somebody buys it. You cannot list it in the catalogue, because it has no row.

The deletion problem. Delete the only order for a discontinued product and the product's details vanish with it, including the fact that it ever existed.

Normalization cures all three by storing every fact exactly once. Third normal form, which is where most operational schemas sit, is roughly: every column depends on the key, the whole key, and nothing but the key. That phrasing is worth remembering because interviewers quote it, but the three symptoms above are what it actually means.

5.4 Why analytics puts them back together

Normalization is right for the application and wrong for analysis, for two reasons.

The first is joins. "Revenue by category, by city, by month" in the normalized model requires joining four tables and knowing exactly which keys connect where. Every analyst who writes that query has a chance to write it slightly differently, and slightly different queries produce slightly different numbers.

The second is history, and it is the deeper one. The operational model deliberately stores one current truth. When Priya moves, `customers.city` is overwritten and the old value is gone. That is correct for the application — the warehouse needs to ship to where she lives now — and it destroys the ability to answer any question about the past.

So analytical models deliberately do the opposite: they duplicate, and they remember. The dimensional model is the standard way of doing that, and the rest of this chapter is about it.

	Operational (OLTP)	Analytical (dimensional)
Optimises for	Safe, fast writes	Understandable, fast reads
Duplication	Avoided	Deliberate
History	Overwritten	Preserved
Tables in a typical query	Many	Two or three
Changed by	Application code	Pipelines you own

5.5 Facts and dimensions

The dimensional model splits every table into one of two kinds, and the split is not arbitrary. It follows from a simple observation: some data is *measurements*, and some data is *context for those measurements*.

A **fact table** holds measurements of events. It is long and narrow, grows constantly, and is mostly numbers and keys. `fact_order_line` holds one row per line of one order, with quantity and amount.

A **dimension table** holds the context you filter and group by. It is short and wide, changes slowly, and is mostly text. `dim_customer`, `dim_product`, `dim_date`.

The test is a question: *would I ever put this in a `GROUP BY` or a `WHERE`?* If yes, it belongs in a dimension. *Would I sum or average it?* If yes, it is a measure in a fact.

Measures are not all the same

The numbers in a fact table divide into three kinds, and mixing them up produces confident nonsense.

Additive measures can be summed across every dimension. `line_amount` is additive: sum it by city, by month, by product, by all three at once, and the answer is meaningful.

Semi-additive measures can be summed across some dimensions but not time. An account balance or an inventory level is the classic case. Summing the warehouse stock level across all warehouses gives you total stock, which is fine. Summing Monday's stock plus Tuesday's stock gives you a number that means nothing at all. For these you sum across space and average, or take the last value, across time.

Non-additive measures cannot be summed at all. Ratios and percentages are the trap. Averaging a column of conversion rates gives the wrong answer unless every group is the same size. The usual fix is to store the numerator and denominator as additive measures and compute the ratio after aggregating, so the ratio is always derived at the grain the question is asked at.

> **COMMON BEGINNER MISTAKES WITH MEASURES**
>
> - Storing a percentage in a fact table and letting people average it.
> - Summing a balance or a stock level across days.
> - Storing `order_total` on a line-grain fact table, which guarantees somebody will sum it.
> - Putting a text column in a fact table because it was convenient, then discovering the table is three times larger than it needed to be.

5.6 The star schema

Put one fact table in the middle and its dimensions around it, each joined by a single key, and you have a **star schema**.

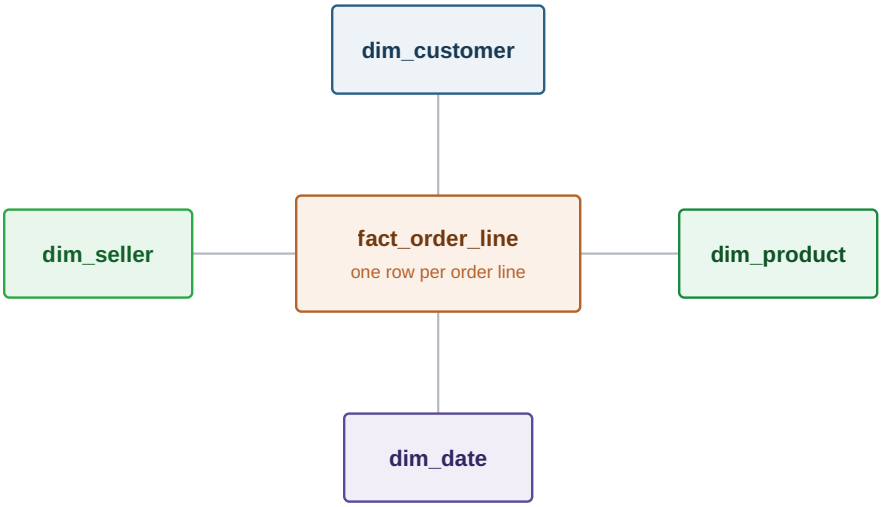


FIGURE 5.3 A star schema. Every dimension is one join away from the fact, and no dimension joins to another. That constraint is the whole point.

Two properties make this shape worth the duplication it costs.

Every dimension is exactly **one join** from the fact. There is no path to get wrong, no chain of keys to trace, no chance that two analysts take different routes to the same answer.

The query is legible even to somebody who has never seen the schema:

```
SELECT p.category, d.month_name, SUM(f.line_amount) AS revenue
FROM fact_order_line f
JOIN dim_product p ON p.product_key = f.product_key
JOIN dim_date    d ON d.date_key    = f.date_key
GROUP BY p.category, d.month_name
ORDER BY revenue DESC;
```

```
 category | month_name | revenue
----------+------------+----------
 Outdoor  | March      | 4200.00
 Fitness  | March      | 3750.00
 Footwear | March      | 3499.00
```

Note `dim_date`. Almost every star schema has one, and beginners always ask why, since the fact table already holds a date. The reason is that a date dimension lets you store things a date column cannot: whether it was a weekend, which fiscal quarter it fell in, whether it

was a public holiday in Maharashtra, whether it was a Big Billion Days trading day. “Revenue on festival days versus normal days” becomes a filter on a column instead of a hard-coded list buried in a query.

5.7 The snowflake schema

A **snowflake schema** is a star where dimensions are themselves normalized. Instead of `dim_product` carrying a `category` column, there is a separate `dim_category` that `dim_product` joins to.

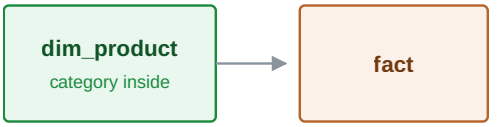
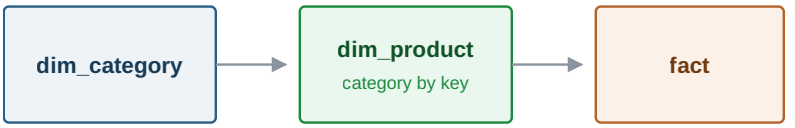


FIGURE 5.4 Star on the left, snowflake on the right. The snowflake saves storage on a table that is already small, and costs a join on every query that touches it.



The honest case for snowflaking is short: it removes duplication and gives one place to change a category name. The case against is that dimensions are tiny — a product dimension with fifty thousand rows is a rounding error next to a fact table with four billion — so you are saving storage you were never short of, in exchange for an extra join on every query and a schema that is harder to explain.

The pragmatic position most teams settle on: keep dimensions flat by default, and snowflake only when a sub-dimension is shared across several dimensions, or when one dimension has grown so large that its width is a real cost.

5.8 Keys

Three key ideas, and the first one causes more confusion than it should.

A **natural key** is the identifier the source system uses: `customer_id = 101`. A **surrogate key** is a meaningless integer the warehouse generates for itself: `customer_key = 1`.

Why bother inventing a second key? Because a natural key identifies a *customer*, and a dimension row is a *version of a customer*. When Priya moves from Pune to Bengaluru, there are two rows with `customer_id = 101` and they need to be told apart. The surrogate key is what does it. Section 5.9 makes this concrete, and once you have seen it the reason stops feeling abstract.

There are secondary benefits — surrogate keys are small integers so joins are fast, and they insulate the warehouse from a source system that changes its identifier format — but the history one is the reason they exist.

A **degenerate dimension** is a dimension attribute with nowhere to live. `order_id` is the standard example: it is not a measurement, and a `dim_order` table holding nothing but an order id would be pointless, so it sits in the fact table as a plain column. You use it to group order lines back together, as we did when counting distinct orders in section 5.1.

A **conformed dimension** is one shared, identically, across several fact tables. If `dim_customer` is used by `fact_order_line`, `fact_support_ticket` and `fact_app_session`, then customer tier means the same thing in all three, and you can compare across them. When each team builds its own customer dimension instead, the company ends up with three different customer counts and a recurring argument nobody can settle. Conformed dimensions are the mechanism that prevents that, and they are more of an organisational discipline than a technical one.

5.9 Slowly changing dimensions

Now the chapter's central problem. Dimension attributes change. Priya moves to Bengaluru on 10 March. What should the warehouse do?

There are four standard answers, numbered by convention.

Type 0 — never change it. Some attributes are historical facts and must not move: original signup date, date of birth, the channel a customer first arrived through. Overwriting these is always a bug.

Type 1 — overwrite. Update the row, lose the old value. Correct when the old value has no analytical meaning: a corrected spelling of a name, a fixed typo in a category. Wrong whenever anyone might want to report on the past.

Type 2 — add a new row. Keep the old version, mark it expired, insert a new current version. Use it for anything that affects reporting, and it is what interviewers mean when they say "walk me through SCD2".

Type 3 — add a column. Keep `city` and `previous_city` side by side. Cheap, and only ever holds one step of history. Occasionally useful for a specific "before and after" comparison, rarely the right general answer.

Type 2, implemented

The dimension carries three extra columns: `valid_from`, `valid_to`, and `is_current`.

Here is the dimension after the initial load on 1 February:

key	cust_id	full_name	city	valid_from	valid_to	current
1	101	Priya Sharma	Pune	2026-02-01	9999-12-31	true
2	102	Rahul Verma	Bengaluru	2026-02-01	9999-12-31	true
3	103	Aisha Khan	Pune	2026-02-01	9999-12-31	true

`9999-12-31` as the end date is a convention worth adopting. It means "still current", and it lets a date-range query work without a special case for `NULL`.

On 10 March, Priya moves. Two statements: expire the old version, insert the new one.

```
UPDATE dim_customer
   SET valid_to   = DATE '2026-03-10',
       is_current = false
 WHERE customer_id = 101
   AND is_current  = true;

INSERT INTO dim_customer VALUES
 (4, 101, 'Priya Sharma', 'Bengaluru', 'gold',
  DATE '2026-03-10', DATE '9999-12-31', true);
```

key	cust_id	full_name	city	valid_from	valid_to	current
1	101	Priya Sharma	**Pune**	2026-02-01	**2026-03-10**	**false**
4	101	Priya Sharma	**Bengaluru**	**2026-03-10**	9999-12-31	**true**
2	102	Rahul Verma	Bengaluru	2026-02-01	9999-12-31	true

key	cust_id	full_name	city	valid_from	valid_to	current
3	103	Aisha Khan	Pune	2026-02-01	9999-12-31	true

Two rows for one customer, distinguished by surrogate key. The new orders that arrive after 10 March point at `customer_key = 4`; the orders from before still point at `customer_key = 1`. The fact table never changes, and history is preserved automatically because each fact is bolted to the version of the customer that was true when the event happened.

In production this pair of statements is usually written as a single `MERGE`, which Chapter 8 covers along with why the whole operation has to be idempotent. The logic is what matters here, not the syntax.

Why this is the whole chapter

Now the two dashboards from the first page. Same facts, same SQL skill, two different models.

With SCD Type 2, each order joins to the customer version that was current at the time:

```
SELECT c.city, SUM(f.line_amount) AS revenue
FROM fact_order_line f
JOIN dim_customer c ON c.customer_key = f.customer_key
GROUP BY c.city ORDER BY revenue DESC;
```

```
 city      | revenue
-----------+----------
 Pune      |  8949.00
 Bengaluru |  2500.00
```

With SCD Type 1, where the city was simply overwritten, every order Priya ever placed is now attributed to Bengaluru:

```
 city      | revenue
-----------+----------
 Bengaluru |  7249.00
 Pune      |  4200.00
```

One customer moving house moved ₹4,749 of March revenue from one city to another, retroactively. The Pune regional manager's numbers changed overnight and no pipeline failed, no alert fired, and no query was wrong.

Both tables are defensible. Type 2 answers *where was this revenue earned*, which is what a finance report or a regional performance review needs. Type 1 answers *where do our customers live now*, which is what a marketing campaign needs. The failure is not choosing one; the failure is not knowing which one you have.

THE QUESTION TO ASK BEFORE CHOOSING

When this attribute changes, should last month's report change with it? If no, you need Type 2. If yes, Type 1 is correct and cheaper. Ask the person who owns the report, not the person who owns the database.

5.10 Three kinds of fact table

Not every fact table records an event.

A **transaction fact** has one row per event, inserted once and never updated. Orders, payments, clicks. This is the default and most fact tables are this kind.

A **periodic snapshot fact** has one row per entity per time period, whether or not anything happened. Daily inventory level per warehouse, monthly account balance per customer. These are how you answer "what was the state on this date", and their measures are usually semi-additive.

An **accumulating snapshot fact** has one row per process instance, updated as the process advances. An order fulfilment row gains `placed_ts`, then `packed_ts`, then `shipped_ts`, then `delivered_ts`. That shape answers "how long does each stage take", which is otherwise a painful query.



FIGURE 5.5 The three fact table shapes. Most teams build the first, need the second sooner than they expect, and discover the third when somebody asks about stage durations.

The accumulating snapshot breaks the usual rule that facts are immutable, which is why it is often avoided. When somebody asks how long orders take to move from packed to shipped, it is the answer.

5.11 What goes wrong

The wrong grain. Covered at length, and worth repeating because it is the one that cannot be fixed later without rebuilding everything downstream.

The fan trap. Two dimensions joined to a fact where one of them is at a finer grain than the fact. Every measure multiplies. If you ever join a fact table to something that is not a dimension, stop and check the row count before and after.

Double counting through a many-to-many. Some relationships really are many-to-many — an order can have several promotions applied. Joining through it multiplies the revenue by the number of promotions. The standard fix is a bridge table with an allocation factor, so each promotion gets a defined share rather than the full amount.

Late-arriving dimensions. A fact arrives referencing a customer the dimension has not seen yet, because the order event beat the customer record through the pipeline. Dropping the fact loses revenue; letting the join fail silently loses it more quietly. The usual answer is an inferred member — insert a placeholder dimension row with the natural key and unknown attributes, let the fact join to it, and fill in the details when the real record arrives.

Nobody wrote the grain down. The root cause of most of the above.

Summary

- Grain is what one row means. Say the sentence out loud, write it down, and pick the finest grain you plausibly need.
- Operational models normalize to make writes safe. Analytical models denormalize to make reads legible and to preserve history.
- Facts are measurements; dimensions are the context you filter and group by. Measures are additive, semi-additive or non-additive, and treating the second two as the first produces confident nonsense.
- A star schema puts every dimension one join from the fact. That constraint is the value.

- Surrogate keys exist so that one customer can have several versions.
- SCD Type 2 keeps history by adding rows. Type 1 overwrites. The choice is a business question: should last month's report change when this attribute changes?
- Transaction, periodic snapshot and accumulating snapshot answer three different classes of question.

In the interview

Walk me through how you would model orders for an e-commerce analytics warehouse.

SHORT ANSWER

I would start with the grain. I would model a fact table at order line level, one row per product per order, because that is the finest grain the business asks questions about and I can always aggregate up. Around it, conformed dimensions for customer, product, date and seller, joined by surrogate keys, with order id kept on the fact as a degenerate dimension so lines can be grouped back into orders.

DEEPER ANSWER

The measures on the fact would be quantity and line amount, both additive, and I would deliberately not put `order_total` there, because it belongs at order grain and putting it on a line-grain table guarantees somebody eventually sums it and reports revenue too high.
For the customer dimension I would use SCD Type 2 on the attributes that affect reporting — city, tier, segment — so that a customer moving does not retroactively change last quarter's regional numbers. Attributes where the current value is the only meaningful one, like a corrected name spelling, I would handle as Type 1.

FOLLOW-UP: HOW DO YOU DECIDE TYPE 1 AGAINST TYPE 2?

I ask the report owner one question: when this attribute changes, should last month's report change with it? If it should not, it has to be Type 2. It is a business decision about what the numbers mean, not a technical preference, and it is worth writing the answer down next to the table definition.

FOLLOW-UP: WHAT IF A FACT ARRIVES FOR A CUSTOMER THE DIMENSION HAS NOT SEEN?

Insert an inferred member — a placeholder dimension row carrying the natural key with unknown attributes — so the fact joins successfully and the revenue is not lost, then update that row when the real record arrives. Dropping the fact silently is the worst option because the loss is invisible.

SENIOR-LEVEL ANSWER

I would frame the whole thing as a contract rather than a diagram. The deliverable is a documented grain, a named owner, a defined SCD policy per attribute, and a reconciliation rule that says the fact table's revenue must agree with the source within a stated tolerance. Most of the modeling disasters I have seen were not bad diagrams. They were good diagrams where the grain was never written down, so it drifted as different people extended the table.

PRACTICE

1. Take three tables from any application you know — a food delivery app, a ride hailing app, a college portal. For each, write the grain sentence.
2. Design a star schema for a food delivery business. Name the fact table, state its grain, and list four dimensions. Then name three questions your model can answer and one it cannot.
3. Build `dim_customer` with `valid_from`, `valid_to` and `is_current` in DuckDB. Load three customers, then move one to another city using the two statements in 5.9. Confirm you get two rows for that customer.
4. Now run the two revenue-by-city queries from 5.9 and reproduce both numbers yourself. Do not move on until you have seen ₹8,949 and ₹4,200 on your own screen.
5. For each of these attributes, decide Type 0, 1, 2 or 3 and justify it in one line: customer signup date, customer email, customer city, product category, product price, employee manager.

6. Take a fact table you have designed and find the measure most likely to be summed incorrectly. Either move it or rename it so the mistake is harder to make.

SOURCES AND VERIFICATION

The star schema, the SCD Type 2 change and both revenue queries in this chapter were built and executed in DuckDB. Every result block is the actual output, including the ₹8,949 against ₹4,200 comparison in section 5.9, which is produced by the same facts joined through two different dimension policies. The build script is in the companion Field Kit.

The dimensional vocabulary used here — fact and dimension tables, grain, conformed dimensions, degenerate dimensions, slowly changing dimension types, and the transaction, periodic snapshot and accumulating snapshot fact table shapes — comes from the Kimball dimensional modeling literature, which remains the standard reference for this material.

6

CHAPTER SIX

Where Data Lives

Two million order lines. Nine columns. Written to disk two ways, the same data occupies **148.3 MB** or **4.3 MB**. Reading one column out of the nine takes **0.49 seconds** or **0.05 seconds**. Nobody changed the data, the query or the hardware. One decision about file format did all of it, and at a thousand times this volume it is the difference between a platform that costs a lakh a month and one that costs thirty.

> VERSIONS AND DIALECTS
>
> Measurements in this chapter were taken with **DuckDB 1.5** on a single machine. Table-format SQL is **Delta Lake on Spark SQL** unless stated otherwise.

Every measurement in this chapter was taken on the machine that produced the book. The script is in the Field Kit and you should run it yourself, because seeing a 34x difference appear on your own terminal changes how you think about storage more than reading about it will.

6.1 Rows and columns, physically

Chapter 1 introduced row and column storage as the reason OLTP and OLAP cannot be the same system. Here is what it means on disk.

A **row-oriented** file keeps all of one row's values together. To read `line_amount` for two million rows, the disk must pass over all nine columns two million times and discard eight of them.

A **column-oriented** file keeps all of one column's values together. Reading `line_amount` reads one contiguous region and never touches the other eight columns. This is called **column pruning**, and it is the single largest source of speed in analytical storage.



FIGURE 6.1 The same nine-column table, two layouts. To sum one column, the row store reads everything; the column store reads a ninth of it.

Column storage has a second effect that is less obvious and often larger. A column contains values of one type, frequently repeating. Our `city` column holds six distinct strings across two million rows; `status` holds three. Stored together, that repetition compresses to almost nothing. Stored row by row, each value sits between a customer id and a date and the compressor cannot see the pattern.

6.2 The formats you will actually meet

Format	Layout	Schema	Compresses	Use it for
CSV	Row	None	Poorly	Exchange with humans and legacy systems
JSON	Row	Self-describing	Poorly	API payloads, event messages
Avro	Row	Stored with the file	Well	Streaming, and writes where whole records are read
Parquet	Column	Stored with the file	Very well	Analytics. The default for a data lake
ORC	Column	Stored with the file	Very well	Analytics, common in Hive-era stacks

Two properties separate the last three from the first two.

They carry their own **schema**. A Parquet file knows that `order_date` is a date and `quantity` is an integer. A CSV knows nothing; every value is text and every reader guesses. Half the type bugs in a beginner pipeline exist because CSV forgets.

They are **splittable** and internally indexed. A Parquet file is divided into row groups, each carrying statistics — minimum, maximum, null count — for every column. A query filtering on `order_date = '2026-02-15'` can read those statistics and skip entire row groups without decompressing them. This is **predicate pushdown**, and it is why a well-organised Parquet dataset answers a filtered query without reading most of itself.

The rule of thumb: **Avro for writing, Parquet for reading**. Streaming systems favour Avro because they append whole records; analytics favours Parquet because it reads few columns from many records.

6.3 Compression, measured

The same two million rows, written four ways:

Format	Size	Versus CSV
CSV	148.3 MB	—
CSV + gzip	18.4 MB	8.0x smaller
Parquet + snappy	26.5 MB	5.6x smaller
Parquet + zstd	**4.3 MB**	**34.8x smaller**

Two things in that table are worth stopping on.

Gzipped CSV beats Snappy Parquet on pure size. That surprises people, and it is not a reason to use it. Gzip is not splittable, so a large gzipped CSV cannot be processed in parallel — one worker must decompress the whole thing. And you still cannot read one column without reading all of them. Small and unusable is not a win.

Parquet with zstd is six times smaller than Parquet with snappy on this data. Snappy is the historical default because it is fast to decompress and CPU used to be the bottleneck. On modern hardware, and with cloud storage billed per gigabyte and per byte scanned, zstd is very often the better trade. It is worth testing on your own data rather than accepting the default.

> **WHERE THE MONEY IS**
>
> Cloud warehouses and query engines bill by bytes scanned. A 34x reduction in stored size is a 34x reduction in that bill for a full scan, every day, forever. Converting an ingestion job to write Parquet instead of CSV is usually the single highest-return change available to a new data engineer, and it takes an afternoon.

6.4 Column pruning, measured

Summing one column out of nine:

```
SELECT SUM(line_amount) FROM read_csv_auto('orders.csv');       -- 0.49 s
SELECT SUM(line_amount) FROM 'orders_snappy.parquet';           --  0.05 s
```

```
CSV       0.49 s
Parquet   0.05 s      10x faster
answer    4,894,500,000.00
```

Ten times faster on identical data, on the same disk, for the same answer. The CSV read had to parse 148 MB of text and convert every field from a string. The Parquet read decompressed one column and summed numbers that were already numbers.

The gap widens as tables get wider. Nine columns gives 10x. A realistic analytical table with sixty columns, where a query touches three, gives considerably more.

6.5 Partitioning

Partitioning means splitting a dataset into directories by the value of a column, so a filter on that column can skip entire directories without opening them.

```
orders/
  order_date=2026-01-01/data.parquet
  order_date=2026-01-02/data.parquet
  ...
  order_date=2026-03-31/data.parquet
```

The directory name carries the value. A query with `WHERE order_date = '2026-02-15'` reads one directory and ignores eighty-nine others. This is **partition pruning**, and it is a different mechanism from predicate pushdown — pruning happens before any file is opened, pushdown happens inside a file that was opened.

Measured on our dataset, ninety daily partitions against one flat file, filtering to a single day:

```
one big file      0.051 s
partitioned       0.005 s      10.9x faster
```

Choosing the partition column

This is where teams go wrong, and the failure has a name.

Partition on a column that queries filter on, that has a moderate number of distinct values, and that produces partitions of a sensible size. Date is the usual answer because analytical queries almost always have a time range and dates arrive in order, so yesterday's partition is written once and never touched again.

Do not partition on something with high cardinality. Partitioning two million rows by `customer_id` with 250,000 distinct values would produce 250,000 directories averaging eight rows each. Every query would then hit the problem in the next section, and the metadata alone would be slower than the scan you were trying to avoid.

Column	Distinct values	Verdict
`order_date`	~90 per quarter	Good — queries filter on it, sizes are even
`city`	6	Usable as a second level, if queries filter on it
`status`	3	Poor — heavily skewed, `DELIVERED` holds most rows
`customer_id`	250,000	**Never**

A commonly used starting range is partitions of roughly 128 MB to 1 GB. Much smaller and per-file overhead dominates; much larger and you lose pruning granularity. The right size depends on your query patterns, your engine's split behaviour and how selective the partition column is, so treat the range as a place to begin measuring.

6.6 The small files problem

Here is the same two million rows, written once as a single file and once as two thousand:

```
2001 files, 32.3 MB total (avg 16 KB each)
one file :  26.5 MB

scan 1 file        0.06 s
scan 2001 files    0.25 s      4.5x slower for identical data
```

Four and a half times slower for exactly the same rows, plus 22% more storage, because every file repeats the schema and the footer metadata.

That measurement is taken on a local SSD, and it understates the problem substantially. On object storage every file costs a separate network request with its own latency — the ten-millisecond order of

magnitude from Figure 2.2, not the microsecond one. Two thousand files becomes two thousand round trips. At the scale where this actually bites, a job that produced three million small files over a year turns a two-minute query into an hour, and almost none of that hour is spent on data.

The causes are always the same three:

Streaming writes. A job writing every ten seconds produces 8,640 files a day.

Over-partitioning. A high-cardinality partition column, as in the previous section.

Too many writers. A Spark job with 2,000 partitions writes 2,000 files per run by default, regardless of how much data is in each.

The fixes are equally standard. Control the number of output files at write time — in Spark this is `coalesce` or `repartition` before the write, covered in Chapter 9. Run a periodic compaction job that rewrites small files into large ones. And if you are using a table format, use its built-in maintenance operation, which is the subject of section 6.9.

> **COMMON BEGINNER MISTAKES WITH LAYOUT**
>
> - Writing CSV into a data lake because it is easy to inspect. Write Parquet and use DuckDB to inspect it.
> - Partitioning by a column that queries never filter on, which costs everything and saves nothing.
> - Partitioning by a high-cardinality column and creating the small files problem on purpose.
> - Accepting the default output file count from a processing job.
> - Never compacting, so a table degrades slowly enough that nobody notices until it is a crisis.

6.7 Object storage is not a filesystem

Data lakes are built on **object storage**: Amazon S3, Azure Data Lake Storage, Google Cloud Storage. It looks like a filesystem, with paths and folders, and it is not one. The differences matter.

There are no directories. `orders/order_date=2026-02-15/data.parquet` is one flat key. The slashes are characters. Listing a "folder" is a query over key prefixes, and listing millions of keys is slow — which is another reason small files hurt.

Objects are immutable. You cannot append to an object or edit part of one. You replace the whole thing. Every update-in-place feature you will meet later is built on top of this constraint rather than around it.

Writes are not instantly consistent everywhere. Modern S3 offers strong read-after-write consistency, but listing behaviour and cross-region replication still lag, and pipelines that assume a file is visible the instant it is written occasionally discover otherwise.

It is cheap, durable and effectively unlimited, which is why the entire industry builds on it despite the above.

	Object storage	Block storage (a disk)
Unit	Whole object	Blocks within a file
Modify part of it	No — rewrite it	Yes
Capacity	Effectively unlimited	Fixed, attached to one machine
Accessed by	HTTP API	The operating system
Cost per TB	Lowest	Higher
Suits	Data lakes, backups, media	Databases, operating systems

6.8 Warehouse, lake, lakehouse

Three words that get used loosely. The distinctions are real.

A **data warehouse** loads data into its own managed, proprietary storage. Snowflake, BigQuery and Redshift do this. You get strong guarantees, excellent query performance, transactions and governance. You give up direct access to the files, and you generally pay more per terabyte.

A **data lake** is files on object storage, usually Parquet, queried by whatever engine you point at them. Cheap, open, and you own the files. What you lose is everything a database gives you: no transactions, so a reader can see a half-written table; no schema enforcement, so a bad job can corrupt a dataset; no update or delete without rewriting whole partitions by hand.

A **lakehouse** is the attempt to keep the lake's economics and openness while adding back the warehouse's guarantees. It works by adding a metadata layer on top of ordinary Parquet files. That layer is the subject of the next section.

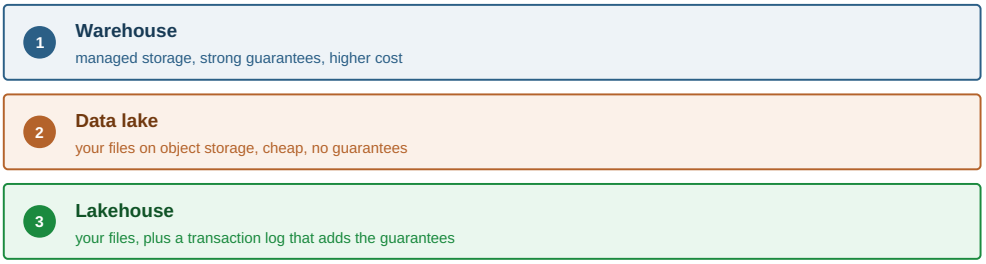


FIGURE 6.2 The three architectures. The lakehouse is not a product; it is a metadata layer over files you already own.

6.9 Table formats: what Delta and Iceberg actually add

This is the part that most beginners find mysterious, and it is simpler than it sounds.

A folder of Parquet files has one fatal weakness: nothing knows which files belong to the table right now. If a job is halfway through writing a new day's data when an analyst runs a query, the analyst sees half a day. If a job fails partway, the folder is left in a broken state with no way to tell which files are valid.

A **table format** — Delta Lake, Apache Iceberg, Apache Hudi — solves this by keeping a **transaction log** alongside the data. The log is the list of which files make up the table at each version. Writers add files and then, as a single atomic action, append a new entry to the log. Readers read the log first and then read exactly the files it names.

Everything else follows from that one idea:

Feature	How the log provides it
ACID transactions	A reader sees the table as of one log entry, never a partial write
Time travel	Read an older log entry and get the table as it was
`UPDATE` and `DELETE`	Write new files, log an entry that removes the old ones
`MERGE` (upsert)	The same, driven by a match condition
Schema enforcement	The log records the schema; incompatible writes are rejected
Compaction (`OPTIMIZE`)	Rewrite small files into large ones, log the swap atomically

```
-- Dialect: Delta Lake on Spark SQL. Iceberg supports the same operations
-- with different syntax and different metadata underneath (see below).
MERGE INTO orders AS target
USING daily_batch AS source
```

```
    ON target.order_id = source.order_id
 WHEN MATCHED THEN UPDATE SET *
 WHEN NOT MATCHED THEN INSERT *;

OPTIMIZE orders;                       -- compact small files
VACUUM  orders RETAIN 168 HOURS;       -- delete files no version needs
DELETE FROM orders WHERE order_date < '2024-01-01';
```

The three are not interchangeable

The shared idea is real: all three keep metadata alongside the data that says which files constitute the table. How they do it differs enough to matter.

	Delta Lake	Apache Iceberg	Apache Hudi
Metadata	An ordered JSON transaction log with periodic Parquet checkpoints	A snapshot tree of manifest lists and manifest files	A timeline of commits plus per-file indexes
Partitioning	Directory-based, like Hive	**Hidden partitioning** — the query need not know the partition scheme	Directory-based, with record-level indexing
Strong at	Tight Spark and Databricks integration	Engine neutrality and schema/partition evolution	Frequent upserts and streaming ingestion
Watch for	Historically Spark-centric, though engine support has broadened	More metadata layers to reason about	More configuration to get right

They are converging, and the honest position for a beginner is that the *concept* transfers completely while the *operational details* do not. Learn one properly. When you meet another, the ideas — snapshots, atomic commits, compaction, time travel — will all be there under different names.

The practical consequence: a claim like “we can swap Delta for Iceberg” is an architecture decision about metadata model, engine support and maintenance tooling, not a syntax translation.

`MERGE` is the statement that makes idempotent loading possible on a data lake, which is why Chapter 8 leans on it heavily. `VACUUM` deserves respect: it permanently deletes files that older versions still reference, so an aggressive retention setting silently destroys your ability to time travel.

6.10 The medallion pattern

With formats and layout understood, the standard organisation of a lake makes sense.



FIGURE 6.3 Bronze, silver, gold. Each layer has one job, so when a number is wrong you have three defined checkpoints to walk back through.

Bronze is a faithful copy of the source, including its mistakes, partitioned by ingestion date and never edited. Its purpose is reprocessing: when you find a bug in a transformation six months from now, bronze is what lets you fix it.

Silver is where types are enforced, duplicates removed, keys resolved and bad records quarantined. One row means one clean business event. Most of the code you write lives here.

Gold is the dimensional model from Chapter 5 — facts and dimensions, aggregates, the tables analysts and dashboards actually query.

The value of the pattern is diagnostic. When a number is wrong, you check gold against silver, and silver against bronze, and the layer where they stop agreeing is where the bug is. Without the layers you are searching one large transformation for a discrepancy with no way to bisect it.

Summary

- Column storage prunes columns and compresses far better. Measured here: 34.8x smaller and 10x faster on identical data.
- Parquet and Avro carry their schema; CSV and JSON do not, and that omission causes most beginner type bugs.
- Avro for writing and streaming, Parquet for reading and analytics.
- Partition on a column queries filter on, with moderate cardinality and even sizes. Aim for partitions of 128 MB to 1 GB.
- Small files are slow because seeks cost more than bytes. Two thousand files were 4.5x slower than one on local disk, and the penalty is far worse on object storage.
- Object storage is not a filesystem: no directories, immutable objects, listing is a query.

- A table format is a transaction log over Parquet files. Every feature — ACID, time travel, `MERGE`, `OPTIMIZE` — comes from that one idea.
- Bronze, silver and gold exist so that a wrong number can be bisected.

In the interview

Your team stores daily exports as CSV in S3 and queries are getting slower and more expensive. What would you change, and in what order?

SHORT ANSWER

Convert to Parquet first, because it gives compression and column pruning together and requires no change to how the data is produced. Then partition by event date so time-filtered queries prune whole directories. Then control the output file count so the conversion does not trade one problem for the small files problem.

DEEPER ANSWER

I would measure before and after rather than assert an improvement. On a nine-column dataset I have measured Parquet with zstd at roughly a thirty-fold size reduction over CSV, and a single-column aggregation about ten times faster. The cost saving usually tracks the size reduction directly, because cloud engines bill by bytes scanned.
Partitioning comes second because it only helps queries that filter on the partition column, so I would look at the actual query log before choosing one. Date is usually right; status or a customer identifier is usually wrong, the first because it is skewed and the second because the cardinality would create hundreds of thousands of tiny files.

FOLLOW-UP: HOW DO YOU HANDLE UPDATES AND DELETES ON A LAKE?

With bare Parquet you cannot; you rewrite whole partitions and there is a window where the table is inconsistent. That is the argument for a table format like Delta or Iceberg, which keeps a transaction log so that a `MERGE` or a `DELETE` is atomic and readers never see a partial state.

SENIOR-LEVEL ANSWER

I would also fix it at the source rather than downstream. If the export is producing CSV daily, the durable change is to have it write partitioned Parquet at ingestion, so every consumer benefits and nobody has to remember to convert. And I would add compaction as a scheduled maintenance job from day one, because small file accumulation is gradual and the failure mode is that nobody notices until a query that used to take two minutes takes an hour.

PRACTICE

1. Run the measurement script from the Field Kit. Reproduce the 148 MB against 4.3 MB and the 0.49 s against 0.05 s on your own machine.
2. Take the same dataset and partition it by `city` instead of `order_date`. Compare query times for a date filter and for a city filter. Explain the difference.
3. Deliberately over-partition by `customer_id`, then measure a full scan. Stop the experiment when you understand why it was a bad idea.
4. Write the same data as 5,000 files and as 1 file, and time a `COUNT(*)` on each. Then repeat on cloud storage if you have access, and compare the ratio.
5. Create a Delta or Iceberg table, run an `UPDATE`, then time travel to the previous version and confirm the old value is still there.

SOURCES AND VERIFICATION

Every figure in this chapter was measured on a two-million-row, nine-column dataset generated and queried with DuckDB on a single machine: format sizes, the column-pruning timing, the partition-pruning timing, and the small files comparison. The generation and measurement script is in the companion Field Kit so the numbers can be reproduced and challenged.

The small files measurement is taken on local SSD and therefore understates the effect on object storage, where each file additionally costs a network round trip. This is stated in the text rather than adjusted, because an honest measurement with its limitation named is more useful than an estimate.

Reference material: the Apache Parquet and Apache Avro format specifications, and the Delta Lake and Apache Iceberg documentation for table format behaviour.

7

CHAPTER SEVEN

The Cloud, and What You Are Actually Renting

Open any data engineering job advert in India and you will find a line like *"experience with AWS (S3, Glue, Redshift, Lambda) or Azure (ADLS, Data Factory, Synapse)"*. Nothing in the previous six chapters told you how to read that line. This chapter does, and it does it in a way that survives the services being renamed next year.

> VERSIONS AND DIALECTS
>
> Service names, free tiers, default settings and pricing in this chapter reflect the position at the time of writing and change without notice. Treat every specific figure as a prompt to check the provider's current documentation. The six categories in Figure 7.1 are what will still be true when the names have moved on.

Here is the trap. Learn "AWS Glue" as a thing and you have learned a product name. Learn *managed serverless transformation, billed per unit of compute, that reads from object storage and writes a catalogue entry* and you have learned something that transfers to Azure Data Factory, Google Dataflow and whatever replaces all three.

So this chapter is organised by **what the box does**, not by whose logo is on it.

7.1 What the cloud actually is

A data centre you rent by the hour, with an API in front of it.

That is not a simplification for beginners; it is the whole idea. Somebody else bought the machines, the disks, the network switches, the generators and the cooling, and they let you ask for a slice of it over HTTP, charge you for what you used, and take it back when you stop paying.

Three consequences shape every decision you make on it.

You pay for time, not for machines. A cluster that runs for twenty minutes a day costs a fraction of one that runs all day. This single fact is why batch is cheaper than streaming, and why "just leave it running" is the most expensive habit in the profession.

Storage and compute are separate. In the old world, a database server held the disks. In the cloud, your data sits in object storage and any amount of compute can be pointed at it, then switched off. Chapter 6's lakehouse depends entirely on this split.

Everything is deny-by-default. Nothing can talk to anything until you have written down that it may. That is the identity system in section 7.4, and it is the single largest source of first-week frustration for new data engineers.

7.2 The six things you rent

Every major cloud sells broadly the same six categories. The names differ; the boxes do not.

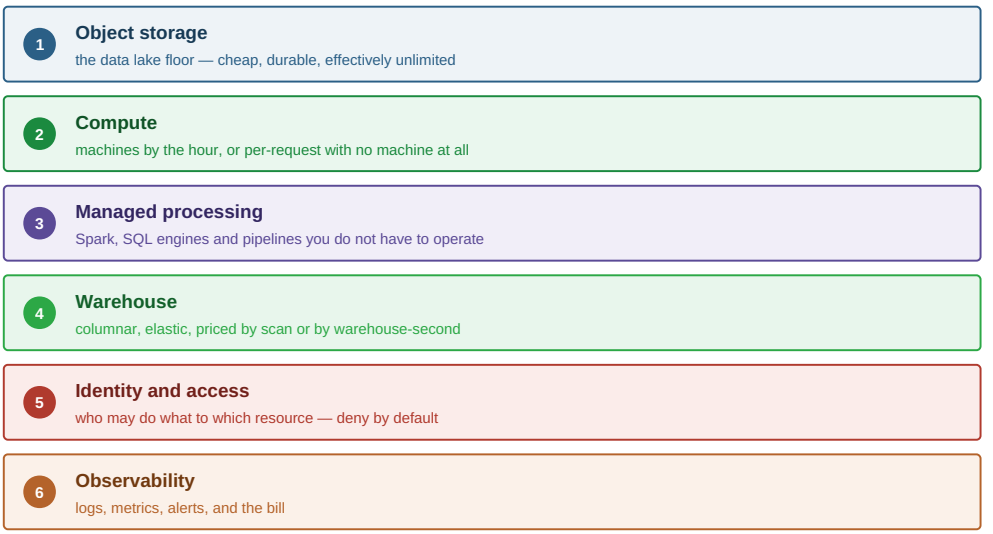


FIGURE 7.1 The six categories. Learn these, and any provider's service list becomes a translation exercise.

The translation table

Pin this to something. It converts almost any job advert into English.

What it does	AWS	Azure	Google Cloud
Object storage	S3	ADLS Gen2 / Blob	Cloud Storage

What it does	AWS	Azure	Google Cloud
Virtual machine	EC2	Virtual Machines	Compute Engine
Run code with no server	Lambda	Functions	Cloud Functions
Containers, managed	ECS / EKS	AKS	GKE / Cloud Run
Managed Spark	EMR / Glue	Synapse Spark / Databricks	Dataproc
Visual / managed pipelines	Glue, Step Functions	Data Factory	Dataflow, Composer
Managed Airflow	MWAA	Data Factory Managed Airflow	Cloud Composer
Data warehouse	Redshift	Synapse / Fabric	BigQuery
Managed relational DB	RDS	Azure SQL / PostgreSQL	Cloud SQL
Streaming log	Kinesis / MSK	Event Hubs	Pub/Sub
Table catalogue	Glue Data Catalog	Purview / Unity	Dataplex
Secrets	Secrets Manager	Key Vault	Secret Manager
Identity	IAM	Entra ID + RBAC	IAM
Logs and metrics	CloudWatch	Monitor	Cloud Logging

HOW TO USE THIS IN AN INTERVIEW

When someone asks whether you know Azure and you have only used AWS, the strong answer is not "no". It is: *"I have built on S3, Glue and Redshift. The Azure equivalents are ADLS Gen2, Data Factory or Synapse Spark, and Synapse — same six boxes, different names and a different identity model. The concepts transfer; I would need a week for the specifics."* That answer gets offers. "No" does not.

7.3 ShopKart on the cloud, drawn once

Here is the platform from Chapter 2, built on real services. Read it as five zones. Every provider gives you the same five; only the names in the boxes change.

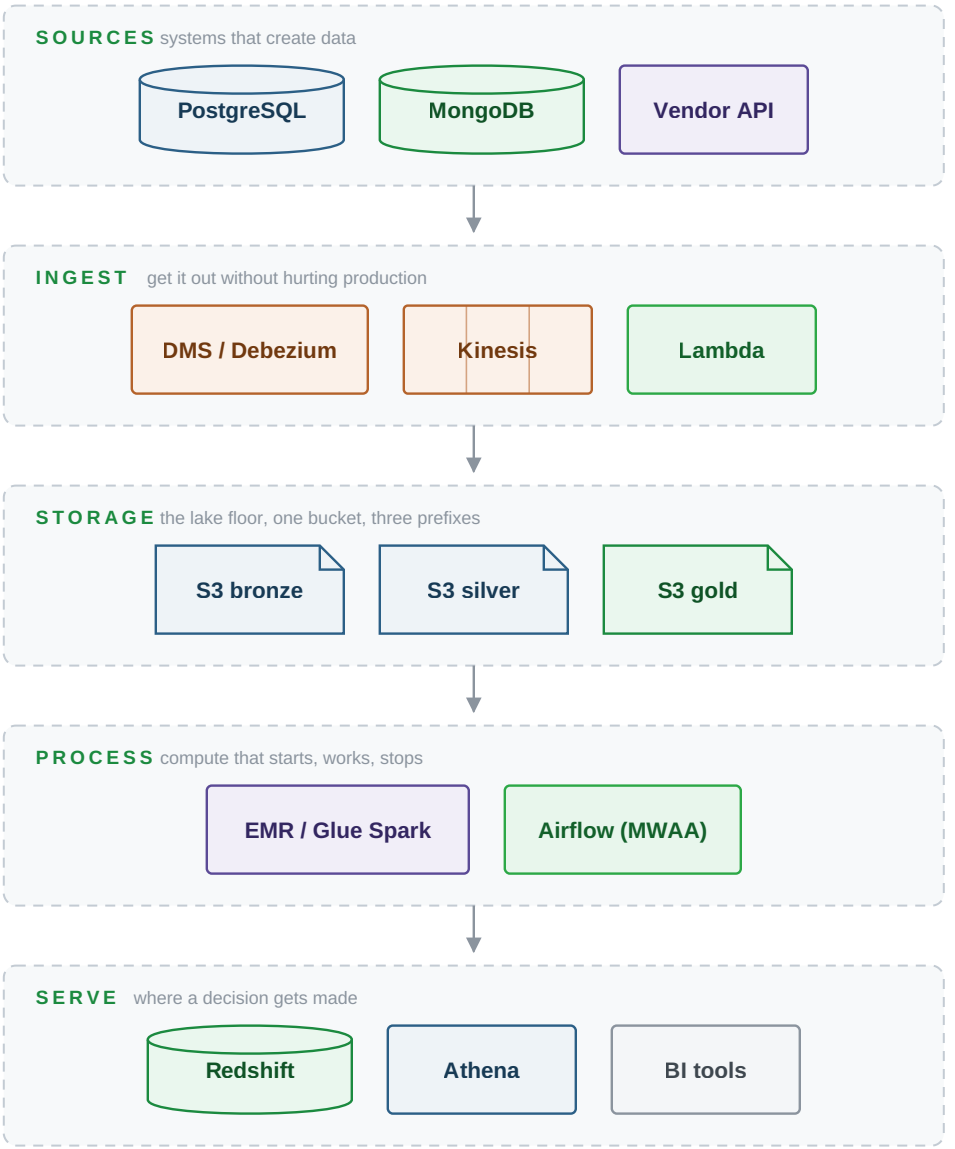


FIGURE 7.2 The reference architecture. AWS names shown; the Azure and GCP equivalents are in the table in 7.2. Five zones, and every arrow crosses a boundary you must give permission for.

Three things about that picture are worth more than the service names.

Storage is one bucket, not three. `s3://shopkart-lake/bronze/…`, `/silver/…`, `/gold/…`. Separate buckets look tidier and immediately create three sets of permissions, three lifecycle policies and three places to get the region wrong. One bucket with prefixes is the common production choice.

Compute is not always on. The Spark cluster exists for the twenty minutes it is working. Airflow runs continuously because it must, and it is small. That split is the whole cost model.

Every arrow needs a permission. The processing job may read the bronze prefix and write silver. It may not delete gold. It may not read the vendor's secret. Each of those is a line in a policy, and none of them exists until you write it.

A workable bucket layout

```
s3://shopkart-lake/
├── bronze/
│   └── orders/dt=2026-03-17/part-0000.parquet
├── silver/
│   └── orders/dt=2026-03-17/part-0000.parquet
├── gold/
│   └── fact_order/dt=2026-03-17/part-0000.parquet
├── quarantine/
│   └── orders/dt=2026-03-17/rejected.parquet
└── _tmp/                       lifecycle: delete after 7 days
```

The `dt=` prefix is Chapter 6's partitioning, and it is also Chapter 8's idempotency: a re-run replaces one prefix and touches nothing else. `_tmp/` exists because half-written output must never land where a reader can see it — write to `_tmp`, then move.

7.4 Object storage, in a little more detail

Chapter 6 established that object storage is not a filesystem. Here is what that means operationally.

Storage classes cost different amounts. Data you read daily, data you read monthly, and data you keep for a seven-year audit should not sit in the same tier. The major providers each offer hot, cool and archive tiers, at roughly descending cost and ascending retrieval time. A **lifecycle rule** moves objects between them automatically — for example, bronze partitions older than ninety days drop to a cooler tier.

That rule is often the single largest cost saving available on a data platform, and it takes ten minutes to write.

Egress is where the bill hides. Storing data is cheap. Reading it *out of the provider's network* is not. A dashboard tool running outside the cloud that scans a large table hourly can cost more in egress than the entire storage bill. Keep compute in the same region as the data.

Region matters for latency, cost and law. Data in `ap-south-1` (Mumbai) is closer to Indian users and may be required to stay there for regulatory reasons. Cross-region reads are slower and billed.



FIGURE 7.3 A lifecycle policy in one line. Most platforms leave everything in the hot tier forever and pay for it monthly.

7.5 Identity, and why nothing works on your first day

Every cloud uses the same three-part model, and understanding it converts a week of confusion into an afternoon.

- A **principal** is who is asking. A person, or a machine, or a running job.
- A **permission** is a verb on a resource. *Read objects in this bucket.*
- A **policy** binds them together, and the default is no.

The mechanism that beginners miss is the **role**. A role is a set of permissions that a machine or a job can *assume*, temporarily, with credentials that expire on their own. This is why Chapter 2 said a cloud identity is the best kind of credential: there is no secret in your code at all, because the machine proves who it is and the SDK fetches a short-lived token.

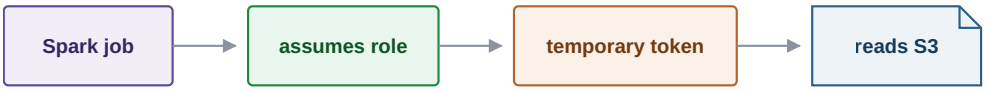


FIGURE 7.4 A job assumes a role and receives temporary credentials. No password exists anywhere in your repository.

The principle to work by is least privilege. Give the job read access to the one bucket prefix it needs and write access to the one it produces, and nothing else. It is more work on day one, and it is the reason a compromised job cannot delete your warehouse.

> **COMMON BEGINNER MISTAKES ON THE CLOUD**
>
> - Attaching an administrator policy to a pipeline because a permission error was in the way. Read the error; it names the exact permission it wanted.
> - Long-lived access keys in a repository, when a role was available.
> - Leaving a cluster running overnight. Compute is billed by the second and idle costs the same as busy.
> - Storing everything in the hot tier forever.
> - Compute in one region and storage in another, paying egress on every read.
> - Choosing a managed service to avoid learning the thing it manages. Managed Spark is still Spark, and it still shuffles.

7.6 What the bill is made of

Cloud cost is a data engineering skill, not a finance one, because the four largest lines on a data platform's bill are all consequences of engineering decisions.

Cost line	Driven by	The lever
Storage per TB-month	Volume and tier	Parquet + compression (Ch 6), lifecycle rules
Compute per second	Cluster size × runtime	Right-size, auto-terminate, spot instances
Bytes scanned	Query shape	Partition pruning and column pruning (Ch 6)
Egress and requests	Cross-region reads, small files	Co-locate; compact small files (Ch 6)

Notice that three of the four levers are chapters you have already read. Chapter 6's 34.8× reduction from Parquet is not a storage curiosity; on a warehouse billed by bytes scanned it is a 34.8× reduction in that line of the bill, every day.

THE HABIT THAT PAYS FOR ITSELF

Tag every resource with the pipeline that owns it, and look at the cost breakdown once a month. An untagged platform produces a single large number that nobody can act on. A tagged one tells you which pipeline costs ₹40,000 a month to serve a report two people read.

7.7 Networking, in the amount you actually need

Two or three days into a real cloud job, something will refuse to connect and the error will be a timeout. Networking is why. You do not need to become a network engineer; you need four ideas.

A **VPC** is your own private slice of the provider's network. Nothing outside it reaches inside unless you allow it.

A **subnet** is a section of that VPC. **Public** subnets can reach the internet; **private** subnets cannot, and that is where databases and processing clusters belong.

A **security group** is a firewall around one resource. It is deny-by-default and it lists what may connect, on which port, from where. This is what you edit when a Spark cluster cannot reach PostgreSQL: allow port 5432 from the cluster's security group, not from the whole internet.

A **private endpoint** lets a resource inside your VPC reach a service like object storage without the traffic leaving the provider's network. This is both a security control and a cost control, because traffic that leaves the network is billed as egress.

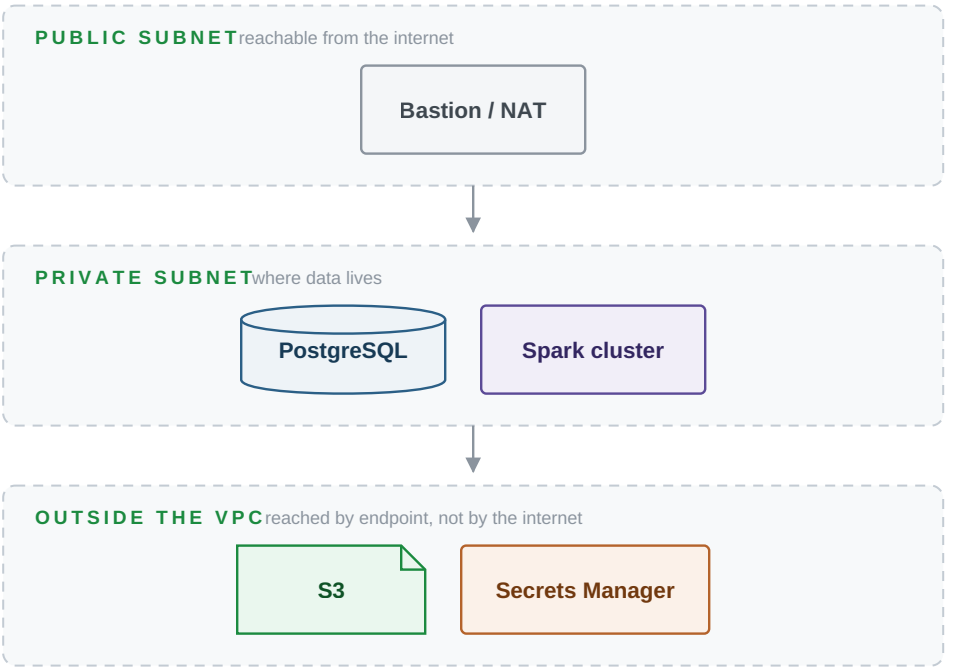


FIGURE 7.5 Why your job cannot connect. The database sits in a private subnet, so the connection must be permitted by a security group, and reaching object storage without a private endpoint costs egress.

Chapter 2 gave you the four connection errors. Here is what each usually means on a cloud:

Error	On a cloud it usually means
Connection timed out	A security group or a missing route. The packets are being dropped silently
Connection refused	You reached the machine, nothing is listening. Wrong port, or the service is down
Host not found	Wrong endpoint name, or private DNS is not enabled
Access denied	You connected fine. This is IAM, not networking

That last row matters more than it looks. **Timeout means networking; access denied means permissions.** Telling those two apart on sight will save you days over a career.

7.8 Managed Spark: four ways to run the same job

Chapter 9's Spark has to run somewhere. There are four shapes, and the choice is about operations and billing rather than about Spark.

Option	You manage	Billing	Suits
Cluster on VMs (EMR, Dataproc)	Cluster config, versions, scaling	Per second of cluster life	Steady, heavy, cost-sensitive workloads
Serverless Spark (Glue, EMR Serverless)	Nothing but the job	Per unit of compute used	Bursty or infrequent jobs
Databricks	Notebooks and job config	Cluster time plus a platform fee	Teams wanting the whole platform
Kubernetes (EKS, GKE)	Rather a lot	Node time	Organisations already standardised on it

THE DECISION, HONESTLY

For a nightly job that runs twenty minutes, serverless Spark is almost always right — you pay for twenty minutes and manage nothing. For a job running eight hours a day, a managed cluster with auto-scaling and spot instances is usually far cheaper. Work out the crossover with a calculator rather than a preference; the answer moves with your runtime, not with fashion.

7.9 Infrastructure as code

Clicking through a console to create a bucket, a role and a cluster works once. It does not survive the second environment, the audit, or the colleague who asks how it was configured.

Terraform describes infrastructure in files you commit, and creates or updates the real thing to match.

```
resource "aws_s3_bucket" "lake" {
  bucket = "shopkart-lake"
}

resource "aws_s3_bucket_lifecycle_configuration" "lake" {
  bucket = aws_s3_bucket.lake.id
  rule {
    id     = "bronze-to-cool"
    status = "Enabled"
    filter { prefix = "bronze/" }
    transition {
```

```
      days          = 90
      storage_class = "STANDARD_IA"
    }
  }
}

resource "aws_iam_role" "spark_job" {
  name = "shopkart-spark-job"
  assume_role_policy = data.aws_iam_policy_document.assume.json
}
```

Three reasons this appears in job adverts. **It is reviewable** — a permission change arrives as a pull request rather than as a click nobody saw. **It is repeatable** — dev, staging and production are the same file with different variables. And **it is reversible**, which the console is not.

You do not need to be fluent. You need to be able to read a Terraform file, find where the bucket and the role are defined, and change one safely.

7.10 Serverless, and when it is the wrong answer

Serverless means you supply code and the provider supplies the machine, for the duration of one request. You are billed per invocation and per unit of runtime, and there is nothing to leave running.

It is excellent for small, event-driven, bursty work: react to a file landing, call an API on a schedule, validate a message, trigger a pipeline.

It is a poor fit for the opposite shape. Long-running jobs hit execution time limits. Large data hits memory limits. And per-invocation billing is superb at low volume and can be worse than a running machine at high, steady volume. The crossover is real and worth calculating rather than assuming.

Use serverless when	Use a cluster when
Work is short and bursty	Work runs for many minutes
Volume is spiky or low	Volume is large and steady
You want no operations	You need control over memory and libraries
Cold start latency is acceptable	Latency must be predictable

7.11 A cost model you can actually do in your head

Interviewers ask this and beginners freeze. You do not need exact prices; you need the right order of magnitude and the right *shape* of the answer.

Take ShopKart's clickstream: **12 TB a day**, kept for two years, scanned by a handful of daily queries.

Storage. 12 TB a day for 730 days is roughly 8,760 TB of raw clickstream. What you actually pay for depends entirely on the format you land it in.

Chapter 6 measured a 34.8× size reduction moving from CSV to Parquet with zstd — on a nine-column, two-million-row order dataset. Clickstream is not that dataset: it is wider, more nested, and its compressibility depends on how repetitive the event payloads are. It may compress better or worse. So do not carry the 34.8× across; carry the *method*. Land one day in both formats, measure the ratio on your own data, and multiply.

Then take that ratio to your provider's current pricing page. Object storage is billed per GB-month and the rate differs by provider, region, storage class and any committed-use discount, so a figure printed in a book would be wrong by the time you read it. The point that survives is structural: **your storage bill is directly proportional to your compression ratio, and the compression ratio is a decision you make in one line of your writer configuration.**

Compute. A nightly Spark job on ten machines for thirty minutes is five machine-hours a day, about 150 a month. Leave the same cluster running instead of terminating it and you pay 7,200. That is the auto-terminate setting, expressed in money.

Scanned bytes. A warehouse billed per terabyte scanned charges by what your query reads, not what you store. A daily dashboard query over an unpartitioned table reads everything; the same query over a table partitioned by date reads one day. Chapter 6 measured that at roughly eleven times faster — and on this billing model, roughly eleven times cheaper, every day, forever.



FIGURE 7.6 The four levers, in the order they usually pay off. Three of them are decisions you already learned in Chapter 6.

THE SENTENCE THAT ANSWERS THE COST QUESTION

"Cost on a data platform is mostly bytes stored, bytes scanned and machine-seconds. Format and partitioning fix the first two, and auto-termination fixes the third. I would tag resources per pipeline so I can see which one is expensive before anybody has to ask."

7.12 Getting started without a bill

Every provider has a free tier, and the exercises in this book are sized for it. Three rules will keep you at zero.

Set a budget alert on day one, before you create anything. Every provider supports it and it takes two minutes.

Auto-terminate every cluster. Set an idle timeout of ten to fifteen minutes on any Spark or warehouse compute. The classic student cloud bill is a cluster left running over a weekend.

Delete what you finish with. Object storage is cheap, but a forgotten warehouse or a managed streaming cluster is not.

Summary

- The cloud is a data centre rented by the hour with an API in front of it. You pay for time, storage and compute are separate, and everything is deny-by-default.
- Six categories cover every provider: object storage, compute, managed processing, warehouse, identity, observability. Learn the boxes and the service names become a translation exercise.

- Object storage has tiers; a lifecycle rule is often the largest single saving on a platform.
- Roles give a job temporary credentials, so no secret exists in your code. Least privilege is the working principle.
- Three of the four largest bill lines are fixed by Chapter 6's file format, partitioning and compaction decisions.
- Serverless suits short, bursty, event-driven work and stops suiting long or steady work.
- Budget alert, auto-terminate, delete when finished. In that order, on day one.

In the interview

The job needs Azure and all your experience is AWS. How do you answer?

SHORT ANSWER

I would map it rather than apologise. I have built on S3, Glue and Redshift; the Azure equivalents are ADLS Gen2, Data Factory or Synapse Spark, and Synapse or Fabric. The same six categories exist on both — object storage, compute, managed processing, warehouse, identity, observability. The concepts transfer directly; the specifics are a week.

FOLLOW-UP: WHAT ACTUALLY DIFFERS, THEN?

Mostly identity and the catalogue. AWS uses IAM roles and policies attached to resources; Azure uses Entra ID with role assignments over a scope hierarchy of subscription, resource group and resource. That difference is where I would expect to spend my first week, not on the storage or the Spark.

SENIOR-LEVEL ANSWER

I would add that the choice of provider is rarely the interesting decision. What determines whether a platform works is the file format, the partitioning, the grain of the model and whether the loads are idempotent — and none of those change between clouds. I would rather be asked how I would keep the bytes-scanned bill down than which button creates a cluster.

> **PRACTICE**
>
> 1. Open a free-tier account with any provider and set a budget alert before creating anything else.

2. Create a bucket, upload the Parquet file from Chapter 6, and query it with the provider's serverless SQL engine. Note the bytes scanned.
3. Now upload the CSV version of the same data and run the same query. Compare bytes scanned and cost. You have just measured Chapter 6 in currency.
4. Write a lifecycle rule moving objects to a cooler tier after thirty days.
5. Create a role with read access to one bucket prefix only, attach it to a small compute instance, and confirm from that instance that you can read the allowed prefix and cannot read another.
6. Take a job advert you would like to apply for and translate every service it names into the six categories in Figure 7.1.

SOURCES AND VERIFICATION

Unlike Chapters 3 to 6, this chapter contains no measured results, because provisioning cloud infrastructure is outside the container used to build this book. It is written as a conceptual map and a translation table rather than a captured run, and every exercise above is designed so you can produce the measurements yourself on a free tier.
Service names change. The translation table reflects the naming at the time of writing; the six categories in Figure 7.1 are what will still be true when the names have moved on.

PART FOUR

Moving Data

Loading the same file twice, processing more than one machine can hold, and running forty jobs that depend on each other.

8 · Pipelines: ETL, ELT and Loading Data Twice

9 · Apache Spark

10 · Orchestration with Airflow

8

CHAPTER EIGHT

Pipelines: ETL, ELT and Loading Data Twice

A job runs on Monday and loads four orders worth ₹13,698. It fails on Tuesday, so somebody re-runs it. It runs normally on Wednesday. On Thursday morning the dashboard shows ₹41,094 in revenue from four orders. Nothing errored. The pipeline succeeded every time. Here is that happening, measured:

> VERSIONS AND DIALECTS
>
> SQL in this chapter was executed on **DuckDB 1.5**. `MERGE` syntax follows the SQL standard form used by PostgreSQL 15+, Snowflake, BigQuery and Delta Lake; keyword details vary. dbt examples target **dbt Core 1.x**.

```
after run 1:  4 rows, revenue 13,698.00
after run 2:  8 rows, revenue 27,396.00
after run 3: 12 rows, revenue 41,094.00
truth      :  4 rows, revenue 13,698.00
```

This chapter is about the difference between a script that moves data and a pipeline that can be trusted to move data repeatedly. There is really only one idea in it, and everything else is a technique for achieving that idea: **running the job twice must produce the same result as running it once**.

8.1 ETL, ELT, and what actually changed

ETL extracts data, transforms it on a separate machine, then loads the result. **ELT** extracts data, loads it raw, and transforms it inside the destination.

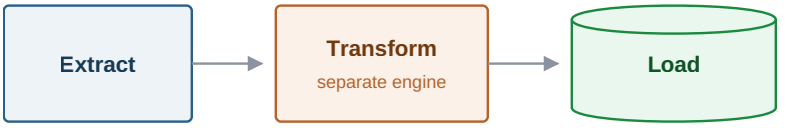
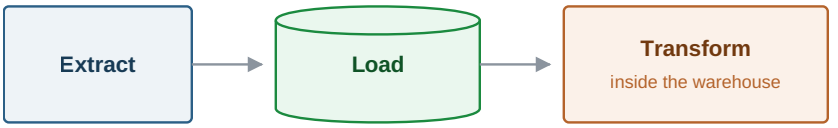


FIGURE 8.1 ETL transforms before landing. ELT lands first and transforms in place. The letters moved because warehouses became cheap and elastic.



ETL was standard when warehouse compute was expensive and fixed. You bought a machine, and you did not waste its capacity on raw data you would end up discarding, so you cleaned first and loaded only the good parts.

ELT became standard when warehouse compute became elastic and cheap, and object storage became almost free. Now it is better to land everything raw and transform inside the destination, because storage costs little, you can reprocess when you find a bug, and the transformation runs on a system that can scale for the ten minutes it needs to.

The medallion pattern from Chapter 6 is ELT with names: bronze is the load, silver and gold are the transform. Most modern platforms are ELT with a small amount of ETL at the edge — light parsing and redaction during ingestion, everything else afterwards.

	ETL	ELT
Transform runs	Before loading, on separate compute	After loading, in the destination
Raw data kept	Often not	Yes, always
Fix a bug from six months ago	Re-extract from the source, if it still has it	Reprocess from raw
Suits	Expensive fixed destinations, heavy redaction rules	Cloud warehouses and lakehouses

8.2 Full load against incremental

A **full load** reads the entire source every time and replaces the target. It is simple, self-correcting — any past mistake disappears on the next run — and it stops being viable somewhere around a few million rows or a few minutes of runtime.

An **incremental load** reads only what changed since last time. It is what almost every production pipeline does, and it introduces every hard problem in this chapter: knowing what changed, handling updates and deletes, coping with data that arrives late, and remaining safe to re-run.

> **WHEN TO STOP DOING FULL LOADS**
>
> Not at a row count. Stop when the full load no longer finishes inside its window with room to spare, or when it puts enough load on the source to be noticed. Until then, full loads are the correct choice and switching early buys you complexity you did not need.

8.3 Watermarks

The standard way to know what changed is a **watermark**: a stored value marking how far the last run got. Usually a timestamp column such as `updated_at`, sometimes a monotonically increasing id.

```sql
-- 1. read the watermark
SELECT last_seen FROM watermark WHERE table_name = 'orders';

-- 2. pull only what is newer
SELECT * FROM source_orders WHERE updated_at > :last_seen;

-- 3. load it (see 8.4 for why this statement matters)
-- 4. advance the watermark
UPDATE watermark SET last_seen = :max_updated_at
 WHERE table_name = 'orders';
```

Four runs of exactly that loop, against a source that changes between run two and run three:

```
run 1 (first load)          picked up 3 | target 3 rows | wm 2026-03-02 09:05
run 2 (nothing new)         picked up 0 | target 3 rows | wm 2026-03-02 09:05
  -- order 9001 cancelled, a new order arrives --
run 3 (update + insert)     picked up 2 | target 4 rows | wm 2026-03-03 12:30
run 4 (re-run, idempotent) picked up 0 | target 4 rows | wm 2026-03-03 12:30
```

Run 3 picked up two rows — one update and one insert — and the target grew by only one, because the update replaced a row rather than adding one. Run 4 did nothing, which is exactly right.

Three details make that loop work, and each is a bug waiting if you get it wrong.

Use a strict inequality. `> last_seen`, not `>=`. With `>=` the last row of every batch is reprocessed forever, which is harmless with an idempotent load and wasteful without one.

Advance the watermark from the data, not the clock. Set it to the maximum `updated_at` you actually loaded, never to `now()`. If the job runs at 02:00 and the source's latest row is from 23:50, using the clock skips ten minutes of rows that arrive afterwards with earlier timestamps.

Update the watermark last. If the load fails, the watermark must still point at the last successful position so the next run retries the same range. A watermark advanced before a failed load is the cleanest way to lose a day of data permanently.

8.4 Idempotency

Now the chapter's central property. A load is **idempotent** if running it twice leaves the system in the same state as running it once.

This is not an optimisation. Pipelines are re-run constantly — after a failure, after a fix, after a backfill, after somebody clicks the button twice — and a load that is not idempotent turns every re-run into a data corruption event.

Here is the non-idempotent version, run three times against an unchanging source:

```
INSERT INTO target_orders SELECT * FROM source_orders;
```

```
after run 1:  4 rows, revenue 13,698.00
after run 2:  8 rows, revenue 27,396.00
after run 3: 12 rows, revenue 41,094.00
truth      :  4 rows, revenue 13,698.00
```

Revenue tripled and nothing failed. Every job reported success. The dashboard rendered. It is the silent failure from Chapter 1, produced by a single missing word.

There are three standard ways to make a load idempotent, and you will use all of them.

Upsert on a key. The target has a primary key, and the load replaces matching rows instead of adding them. This is what produced the correct four-row result in section 8.3.

```
INSERT OR REPLACE INTO target_orders
SELECT * FROM source_orders WHERE ...;
-- PostgreSQL: INSERT ... ON CONFLICT (order_id) DO UPDATE SET ...
-- Delta/Spark/Snowflake: MERGE INTO ... WHEN MATCHED THEN UPDATE ...
```

Delete and replace a partition. For batch loads organised by date, delete the target partition for the day being processed, then insert it. Re-running replaces rather than accumulates. It is the standard pattern on a data lake and it is why partitioning by date matters beyond query speed.

```
DELETE FROM fact_order_line WHERE order_date = '2026-03-17';
INSERT INTO fact_order_line SELECT ... WHERE order_date = '2026-03-17';
```

Deduplicate on the way in. When the source can send the same event twice — which every event stream can — carry a unique event id and drop repeats using the `ROW_NUMBER` pattern from Chapter 3.

What idempotency actually requires

A common shortcut is "we use `MERGE`, so the load is idempotent". `MERGE` is a mechanism, not a guarantee. The load is idempotent when **all** of the following hold:

Condition	If it fails
The business key genuinely identifies one row	`MERGE` matches the wrong rows, or errors on ambiguity
The source batch is deduplicated on that key	Most engines reject or behave unpredictably on duplicate matches
The match condition is correct and complete	Rows update the wrong target, or insert alongside instead of replacing
Transformations are deterministic	`now()`, `rand()` or unstable sort order make each run differ
The source can be re-read for the same window	You cannot reprocess what the source no longer has

Condition	If it fails
Deletes are represented, not just absences	Cancelled records live on in the target forever

Miss any one and you have a load that *usually* behaves and occasionally corrupts. Chapter 13's project satisfies all six, which is why running it twice produces identical output — and why the chapter says so with a measurement rather than an assertion.

THE QUESTION TO ASK ABOUT EVERY PIPELINE YOU BUILD

What happens if this runs twice? If you cannot answer immediately, the pipeline is not finished. This is also one of the most reliable interview questions in the field, and a candidate who reaches for it unprompted stands out.

8.5 `MERGE`, properly

`MERGE` is the general form of the upsert and it appears in every serious system: PostgreSQL, Snowflake, BigQuery, Spark and Delta Lake.

```
MERGE INTO target_orders AS t
USING daily_batch       AS s
   ON t.order_id = s.order_id
 WHEN MATCHED AND s.updated_at > t.updated_at
      THEN UPDATE SET status = s.status, amount = s.amount,
                      updated_at = s.updated_at
 WHEN MATCHED AND s.is_deleted
      THEN DELETE
 WHEN NOT MATCHED
      THEN INSERT VALUES (s.order_id, s.customer_id, s.status,
                          s.amount, s.updated_at);
```

Two clauses in that statement are easy to leave out and painful to leave out.

`AND s.updated_at > t.updated_at` prevents an older version of a row overwriting a newer one. Without it, a replay of yesterday's batch will happily undo today's updates.

The `WHEN MATCHED AND s.is_deleted THEN DELETE` branch handles deletions, which are the most commonly forgotten case in the entire chapter. An order cancelled in the source that is never deleted in the target stays in the warehouse forever, quietly inflating every count that includes it.

There is one trap. If the source batch contains two rows with the same key, `MERGE` fails or behaves unpredictably depending on the engine. Deduplicate the batch before merging, every time:

```
WITH deduped AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY order_id
                                 ORDER BY updated_at DESC) AS rn
    FROM daily_batch
)
SELECT * FROM deduped WHERE rn = 1
```

8.6 Change data capture

A watermark on `updated_at` has two weaknesses. It relies on the source maintaining that column correctly, and it cannot see deletions — a deleted row simply stops appearing, and nothing distinguishes that from a row that did not change.

Change data capture solves both by reading the database's own write-ahead log, the internal record it keeps of every change for crash recovery. A tool such as Debezium tails that log and emits one message per change, each carrying an operation type.

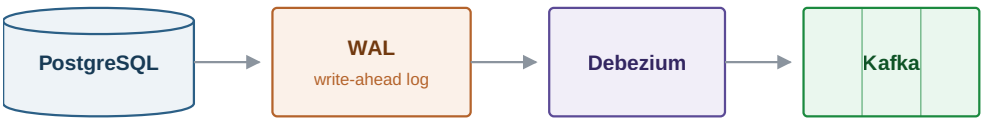


FIGURE 8.2 CDC reads the log the database already writes for itself, so the source barely notices, and deletes and updates are captured as first-class events.

Each message says what happened, not just what the row looks like now:

Operation	Meaning
`c`	create — a new row was inserted
`u`	update — with the old and new values

Operation	Meaning
`d`	delete — the row was removed
`r`	read — part of the initial full snapshot

	Watermark polling	CDC
Load on source	A query per run	Almost none — reads the log
Latency	The polling interval	Seconds
Captures deletes	No	Yes
Captures every intermediate state	No — only the latest	Yes
Setup cost	Low	Higher: connectors, permissions, ops

CDC is the right answer for replicating an operational database. Polling remains right for APIs, file drops and any source with no accessible log.

8.7 Late-arriving data

Here is the failure that catches almost everyone the first time.

A row is created in the source with `updated_at` of 2026-03-02 20:00, but it does not reach your source table until after your run has already processed up to 2026-03-03 12:30. The watermark has moved past it. The next run asks for rows newer than the watermark and never sees it.

```
watermark is 2026-03-03 12:30:00
rows silently missed by the next run: [(9005, 2026-03-02 20:00:00)]
```

That row is now invisible forever. No error, no warning, one order permanently missing from revenue.

Late data happens for ordinary reasons: a mobile app that was offline syncs three days of events, a partner's nightly file arrives a day behind, a source system's replica lags, a network partition heals and flushes a backlog.

The standard fix is a **look-back window**. Instead of processing strictly newer than the watermark, reprocess a trailing period as well:

```
INSERT OR REPLACE INTO target_orders
SELECT * FROM source_orders
WHERE updated_at > :watermark - INTERVAL '3 days';
```

```
after a 3-day look-back window: 5 rows in target (was 4)
order 9005 recovered: True
```

The look-back only works because the load is idempotent. Reprocessing three days with an appending load would triple three days of revenue, which is why these two ideas belong in the same chapter. The window length is a business decision: measure how late data actually arrives, then set the window comfortably beyond the ninety-fifth percentile, and monitor for anything arriving later than that.

8.8 Schema evolution

Sources change their shape without telling you. Three cases, in increasing order of pain.

A column is added. The friendliest case. A pipeline that reads by name and writes to a table format with schema evolution enabled handles it automatically; a pipeline that reads by position puts the new column's values into the wrong field. That is the second time in this book that reading by position has caused a silent corruption, and it will not be the last.

A column is removed. Downstream queries referencing it break loudly, which is inconvenient but honest. Handle it by selecting explicit columns and filling missing ones with `NULL` rather than letting the job die.

A column changes type. The dangerous one, because it can succeed. An `order_id` that changes from integer to string still loads if your target column is text, and every join against it silently stops matching. Type checks on ingestion are the defence, and they belong in Chapter 12's validation layer.

The practical policy that most teams settle on: additive changes are accepted automatically, everything else fails the load and raises an alert. Silent acceptance of a type change is how you find out three weeks later.

8.9 dbt: the T in ELT, with engineering around it

Chapter 8 has been about moving data. The transforming is SQL you have already learned in Chapter 3. What has been missing is the thing that turns a folder of `.sql` files into something a team can maintain, and in a large share of Indian data engineering job adverts that thing is named explicitly: **dbt**.

dbt does not move data and does not process it. It sends SQL to your warehouse and manages everything around that SQL: dependencies, testing, documentation and version control. It is the tool that made analytics engineering a job title.

What a model is

A dbt **model** is one `.sql` file containing one `SELECT`. dbt wraps it in the `CREATE TABLE` or `CREATE VIEW` for you.

```
-- models/silver/stg_orders.sql
SELECT
    order_id,
    customer_id,
    CAST(order_ts AS DATE) AS order_date,
    UPPER(status)          AS status,
    CAST(amount AS DECIMAL(10,2)) AS amount
FROM {{ source('shopkart', 'raw_orders') }}
WHERE order_id IS NOT NULL
```

```
-- models/gold/fct_order_line.sql
-- grain: one row per order line
SELECT
    l.order_line_id,
    o.order_id,
    o.customer_id,
    o.order_date,
    l.product_id,
    l.quantity,
    l.line_amount
FROM {{ ref('stg_order_items') }} l
JOIN {{ ref('stg_orders') }} o ON o.order_id = l.order_id
WHERE o.status = 'DELIVERED'
```

The two functions in braces are the whole idea.

`source()` names a raw table you did not build, in one place. `ref()` names another model. Because every model declares its inputs with `ref()`, **dbt derives the dependency graph from the SQL itself** — you never write the order of execution. It is the same DAG idea as Chapter 10, discovered rather than declared.



FIGURE 8.3 dbt reads ref() calls and builds the DAG. Run it and the models execute in dependency order, in parallel where possible.

Tests are the part that matters

This is where dbt earns its place in a data engineering book rather than an analytics one. Chapter 12's data quality checks are built in, declared in YAML next to the model:

```yaml
# models/gold/schema.yml
models:
  - name: fct_order_line
    description: "One row per order line. Grain enforced by order_line_id."
    columns:
      - name: order_line_id
        tests: [unique, not_null]          # the grain, tested every run
      - name: order_id
        tests:
          - not_null
          - relationships:                  # referential integrity
              to: ref('stg_orders')
              field: order_id
      - name: status
        tests:
          - accepted_values:
              values: ['DELIVERED', 'CANCELLED', 'RETURNED']
```

`dbt build` runs each model and then its tests, and **stops the branch if a test fails**, so a broken table does not become the input to five others. That is Chapter 12's "validate before publishing" as a default rather than as something you remember to write.

Materializations

One configuration line decides how a model is physically built, and it is the main performance decision in dbt.

Materialization	What it builds	Use when
`view`	A view, rebuilt logically each query	Small, cheap, always fresh
`table`	Dropped and recreated in full	Moderate size, full refresh is affordable
`incremental`	Appends or merges only new rows	Large tables, the Chapter 8 problem
`ephemeral`	Inlined as a CTE, no object created	Intermediate logic used once

The incremental form is this chapter's watermark and `MERGE`, expressed declaratively:

```
{{ config(materialized='incremental', unique_key='order_id') }}

SELECT * FROM {{ source('shopkart', 'raw_orders') }}

{% if is_incremental() %}
  WHERE updated_at > (SELECT MAX(updated_at) FROM {{ this }})
{% endif %}
```

On a first run it builds the whole table. On later runs the `is_incremental()` block adds the watermark filter, and `unique_key` makes dbt merge rather than append — which is section 8.4's idempotency, obtained by declaring it.

> **COMMON BEGINNER MISTAKES WITH DBT**
>
> - Writing raw table names instead of `ref()`, which breaks the DAG and the lineage.
> - No tests, which reduces dbt to an expensive way to run SQL files.
> - Everything materialized as a table, so a small lookup is rebuilt nightly for no reason.
> - `incremental` with no `unique_key`, which appends and produces the tripling revenue from section 8.4.
> - One enormous model instead of a staging layer and a mart layer.
> - Treating dbt as a processing engine. It sends SQL to a warehouse; the warehouse does the work.

WHERE DBT SITS

dbt is the *T* in ELT and nothing else. Something must still land the raw data (Chapter 8's ingestion), something must schedule it (Chapter 10), and something must store it (Chapters 6 and 7). Candidates who describe dbt as their pipeline are describing a third of one, and interviewers notice.

8.10 Backfills

A **backfill** is reprocessing history — because you fixed a bug, added a column, or onboarded a source that has two years of data.

Three rules make backfills survivable.

Process in partitions, not all at once. One day at a time, in a loop. A single statement covering two years will run for hours, hold locks, and leave you with nothing when it fails at 80%.

Make each partition independently re-runnable. Which you already have, if the load is idempotent by partition.

Throttle it. A backfill running at full speed alongside the daily pipeline will compete with it for the same resources and make both late. Run it at lower parallelism, or outside the busy window.

```
from datetime import date, timedelta

start, end = date(2026, 1, 1), date(2026, 3, 31)
day = start
while day <= end:
    run_partition(day)          # delete + insert, or MERGE, for that day
    log.info("backfilled %s", day)
    day += timedelta(days=1)
```

Keep the backfill using the same code path as the daily run. A separate "backfill script" drifts from the production logic and eventually produces history that does not match the present, which is a miserable class of bug to diagnose.

COMMON BEGINNER MISTAKES WITH LOADING

- A plain `INSERT` with no key, no partition replace and no dedup.
- Advancing the watermark to `now()` instead of the maximum value actually loaded.
- Advancing the watermark before the load succeeds.
- No look-back window, so late data is lost silently.
- Forgetting deletes, so cancelled records live in the warehouse forever.
- Merging a batch that contains duplicate keys.
- Writing a separate backfill script that drifts from the daily one.

Summary

- ETL transforms before loading; ELT lands raw and transforms in place. ELT won because storage got cheap and reprocessing matters.
- Full loads are simple and self-correcting. Move to incremental when the window gets tight, not before.
- Watermarks must use strict inequality, advance from the data rather than the clock, and update only after a successful load.
- Idempotency is the property that matters most. Measured here: an appending load tripled revenue in three runs with no error.
- Upserting on a key, replacing partitions and deduplicating on ingestion are the mechanisms. They deliver idempotency only when the six conditions in 8.4 hold — unique business key, deduplicated input, correct match condition, deterministic transformations, re-readable source, and deletes represented.
- `MERGE` needs a version guard and a delete branch, and its input must be deduplicated first.
- CDC reads the database's own log, so it captures deletes and barely touches the source.
- Late data is invisible to a naive watermark. A look-back window fixes it, and only works because the load is idempotent.
- Backfill in partitions, through the same code path as the daily run.

In the interview

Your daily pipeline failed halfway through last night. How do you recover, and what would you have built differently?

SHORT ANSWER

If the load is idempotent and partitioned by day, recovery is just re-running that day — the partition is replaced rather than appended to, so a partial write from the failed run is overwritten. The first thing I would check is whether the watermark advanced before the failure, because if it did, a plain re-run will skip the rows that were never loaded.

DEEPER ANSWER

I would design so the question is boring. Each run owns one partition, the load either replaces that partition or merges on a key, and the watermark advances only after the load commits. Then a failure at any point leaves the system in a state where re-running is always correct and always safe.

The failure mode I would specifically guard against is a partial write that looks complete. If the job wrote half a day and then died, a downstream consumer must not be able to read that half. On a lakehouse the transaction log gives that for free; without one, I would write to a staging location and swap atomically.

FOLLOW-UP: HOW DO YOU KNOW WHETHER LAST NIGHT ACTUALLY LOADED EVERYTHING?

By reconciling counts rather than trusting the exit code. Row count and a summed measure from the source against the target for that partition, checked before the table is published. A job that succeeds while loading 60% of the rows is the failure that matters, and an exit code cannot detect it.

SENIOR-LEVEL ANSWER

I would add that most of these recovery questions disappear if the pipeline is designed around a defined unit of work with an owner and a contract — this partition, complete by this time, reconciled to this tolerance. Once that is explicit, retries, backfills and late data are all the same operation applied to a different set of partitions, rather than three separate special cases with their own code.

PRACTICE

1. Run the Field Kit script and reproduce the tripling revenue yourself. Then change the one statement that fixes it.
2. Build the watermark loop from 8.3 in DuckDB. Run it four times against a source you change in between, and confirm the row counts.
3. Deliberately advance your watermark to `now()` instead of the data maximum, then insert a row with an earlier timestamp. Watch it disappear.
4. Add a three-day look-back window and confirm the lost row comes back.
5. Write a `MERGE` with a version guard, then replay an old batch and confirm it does not overwrite newer data. Remove the guard and watch it do so.
6. Feed a `MERGE` a batch containing two rows with the same key. Read the error carefully — you will meet it again.

SOURCES AND VERIFICATION

The watermark loop, the tripling revenue from a non-idempotent load, the late-arriving row missed by the watermark, and its recovery through a look-back window were all executed in DuckDB, and every output block is the actual result. The script is in the companion Field Kit.

The `MERGE` syntax shown follows the SQL standard form used by PostgreSQL, Snowflake, BigQuery and Delta Lake; keyword details vary slightly between engines. CDC operation codes follow the Debezium change event format.

9

CHAPTER NINE

Apache Spark

Two lines of code that look almost identical. One of them moves no data between machines and finishes in 0.42 seconds. The other moves every row across the cluster and takes 1.64 seconds. At two million rows the difference is a shrug. At two billion it is the difference between a job that finishes before the morning meeting and one that does not.

> VERSIONS AND DIALECTS
>
> All plans and timings in this chapter come from **Apache Spark 4.2** running in local mode on four cores. PySpark code targets Spark 3.x and later. Plan output formatting differs between Spark versions.

```
orders.filter(col("amount") > 1000).select("city", "amount")   # 0.42 s
orders.groupBy("city").agg(sum("amount"))                      # 1.64 s
```

Learning Spark is not learning its API. The API is small and you will absorb it in a week. Learning Spark is learning to look at code and predict what the engine will do with it, and the tool for that is the execution plan. Everything in this chapter builds towards being able to read one.

Every plan and timing printed here came from an actual Spark 4.2 session running on four cores. The script is in the Field Kit.

9.1 Why Spark exists

One machine has a hard ceiling. Some hundreds of gigabytes of memory, some dozens of cores, one network card. Past that you cannot buy a bigger machine, and long before that point you cannot buy one at a sensible price.

The alternative is many ordinary machines working on pieces of the same problem. That idea is old; what is hard is everything around it — splitting the work, moving data between machines, coping when one dies mid-job, and doing all of it without the programmer writing distributed systems code by hand.

Hadoop MapReduce solved it first, and wrote every intermediate result to disk between steps. Spark's improvement was to keep intermediate results in memory **where they fit**, spilling to disk when they do not. That matters for the iterative and multi-stage work analytics consists of.

Two qualifications, because "Spark is in-memory" is repeated far more often than it is understood. Shuffle output is **always written to disk**, in every version, regardless of available memory — section 9.5 explains why. And memory is not the only reason Spark is faster: whole-stage code generation, the lazy optimiser and the DAG scheduler all contribute. A job that spills heavily can still comfortably beat MapReduce.

WHEN YOU DO NOT NEED SPARK

If your data fits comfortably in memory on one machine, Spark will usually be *slower* than pandas or DuckDB, because you pay for coordination you do not need. The threshold is not a number of rows; it is the point where a single machine can no longer hold the working set or finish in the time available. Chapter 4's table still applies.

9.2 The pieces

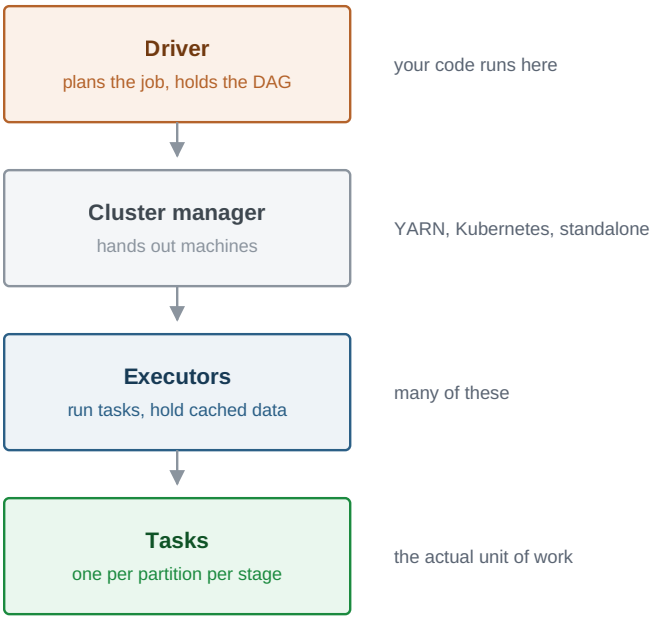


FIGURE 9.1 The driver plans and coordinates; executors do the work. When people say "the job died and there is no error", they usually mean an executor was killed and the driver only saw it disappear.

The **driver** is where your program runs. It builds the plan, decides the stages, and schedules tasks.

It does not process your *distributed* data — that is what executors are for — but it is not idle either. The driver executes every line of your program that sits outside a transformation, holds the plan and the broadcast variables, and receives results from any action that returns data: `collect()`, `take()`, `show()`, `toPandas()`. `collect()` on a large DataFrame pulls the entire dataset into the driver's memory and is the most reliable way to kill a Spark application.

Executors are the worker processes. Each has a fixed amount of memory and a fixed number of cores, and each core runs one task at a time.

A **task** processes exactly one partition. That sentence connects Chapter 2's discussion of cores to everything here.

9.3 Partitions

A **partition** is a slice of your data that one task processes. Partition count decides how much parallelism is available.

```
default partitions      : 4       (matches the 4 available cores)
after repartition(16)   : 16
after coalesce(2)       : 2
```

The arithmetic that follows is the single most useful thing to internalise. If a cluster has 5 executors with 4 cores each, 20 tasks can run at once. With 200 partitions the job runs in 10 waves. With 8 partitions, 12 cores sit idle and you pay for them anyway. With 200,000 partitions the scheduling overhead exceeds the work.

A **starting point** many teams use is two to four partitions per available core, with each partition holding somewhere in the region of 100–200 MB. Treat that as a place to begin measuring, not a rule. The right number depends on row width, how much the stage shuffles, executor memory, input file sizes, and whether Adaptive Query Execution is coalescing partitions at runtime anyway — which since Spark 3 it usually is. The reliable method is to look at task durations and shuffle sizes in the Spark UI and adjust from there.

Two operations change the count, and they are not interchangeable:

	`repartition(n)`	`coalesce(n)`
Can increase partitions	Yes	No, only decrease
Causes a full shuffle	Yes	No
Result sizes	Even	Possibly uneven
Use when	Fixing skew, increasing parallelism	Reducing file count before a write

`coalesce` is cheap because it merges neighbouring partitions in place without moving data across the network. That makes it the right tool for the small files problem from Chapter 6: `df.coalesce(10).write.parquet(...)` produces ten files instead of two hundred.

9.4 Lazy evaluation

Spark does nothing until you ask for a result.

```
building the transformation took 0.0663 s   (nothing ran)
calling .count() took            1.20 s  -> 1,200,000 rows
```

Sixty-six milliseconds to define a filter and a select over two million rows, because Spark only recorded the intent. The work happened when `count()` was called.

Transformations — `filter`, `select`, `join`, `groupBy`, `withColumn` — build up a plan and return a new DataFrame. **Actions** — `count`, `collect`, `show`, `write`, `take` — execute it.

Laziness is not a curiosity; it is what allows optimisation. Because Spark sees the whole chain before running any of it, it can push your filter down to the moment data is read rather than reading everything and filtering afterwards. A pipeline evaluated eagerly, step by step, could not do that.

The practical consequence is that timings mislead beginners constantly. The line that appears slow in your notebook is usually just the first action after twenty transformations, and the twenty transformations are what actually took the time.

9.5 Narrow, wide, and the shuffle

This is the distinction that governs Spark performance.

A **narrow transformation** computes each output partition from exactly one input partition. `filter`, `select`, `withColumn`. No data crosses the network; each executor works on what it already holds.

A **wide transformation** needs data from many input partitions to produce one output partition. `groupBy`, `join`, `distinct`, `orderBy`. Rows must be moved so that everything with the same key lands together. That movement is the **shuffle**, and it is the most expensive thing Spark does.

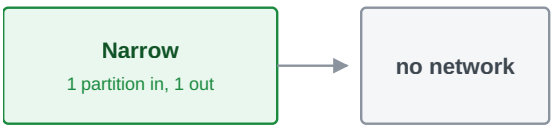
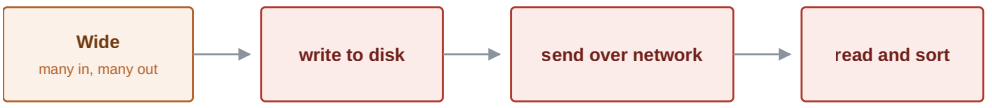


FIGURE 9.2 A narrow transformation stays inside its partition. A wide one redistributes rows across the cluster, over the network, through disk on both sides.



You do not have to guess which one you wrote. Ask the engine.

```
narrow = orders.filter(col("amount") > 1000).select("city", "amount")
```

```
*(1) Project [element_at(...) AS city, cast(...) AS amount]
+- *(1) Filter (cast(((id % 4500) + 199) as double) > 1000.0)
   +- *(1) Range (0, 2000000, step=1, splits=4)
```

No `Exchange`. Everything happens in one stage, marked `*(1)`, entirely within each partition.

```
wide = orders.groupBy("city").agg(sum("amount").alias("revenue"))
```

```
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[city], functions=[sum(amount)])
   +- Exchange hashpartitioning(city, 8), ENSURE_REQUIREMENTS
      +- HashAggregate(keys=[city], functions=[partial_sum(amount)])
         +- Project [element_at(...) AS city, cast(...) AS amount]
            +- Range (0, 2000000, step=1, splits=4)
```

There it is: `Exchange hashpartitioning(city, 8)`. **`Exchange` means shuffle.** That one word is what you are looking for in every plan you ever read.

Notice also the two `HashAggregate` nodes. Spark sums within each partition first — `partial_sum` — and only then shuffles the much smaller partial results. This is a map-side combine, and it is why `groupBy().sum()` is far cheaper than the shuffle-everything version a beginner might fear.

Measured, on identical data:

```
narrow (filter, count)   0.42 s
wide   (groupBy, sum)    1.64 s
```

9.6 Reading a physical plan

Plans read bottom to top. The bottom is where data enters; the top is the final result.

Six things to look for, in the order they matter:

Look for	It means
`Exchange`	A shuffle. Count them; each one costs
`BroadcastHashJoin`	The small side was sent to every executor. Cheap

Look for	It means
`SortMergeJoin`	Both sides shuffled and sorted. Expensive
`PushedFilters: [...]`	Your filter reached the file reader. Good
`ReadSchema:`	Which columns are actually read. Should be only what you need
`*(n)`	Whole-stage code generation. Its absence can mean a slow path

The two habits worth building: run `df.explain()` before running an expensive job, and count the `Exchange` nodes. If there are more than you expected, something in your code is shuffling that need not.

9.7 Joins

Joining is where most Spark performance work happens, because a join between two large datasets is a shuffle on both sides.

A **sort-merge join** shuffles both sides so matching keys land on the same executor, sorts each side, then merges. It works at any scale and it is expensive.

A **broadcast hash join** sends the entire small side to every executor, so the large side never moves. No shuffle on the big table at all. No optimisation in Spark repays more, and it applies to exactly the case that occurs constantly: a huge fact table joined to a small dimension.

Measured, joining two million order rows to a six-row city dimension:

```
with broadcast hint    BroadcastHashJoin   Exchange x2    0.22 s
no hint                SortMergeJoin       Exchange x3    0.23 s
```

The plan changed and the time did not. Say that plainly rather than hide it, because it teaches something: **at two million rows on one machine, the shuffle is small enough that the strategy barely matters.** The plan difference is what scales. At two billion rows across a cluster, the sort-merge join moves the entire fact table across the network and the broadcast join moves six rows. One `Exchange` fewer is the whole difference between minutes and hours.

Spark broadcasts automatically when it believes the small side is under `spark.sql.autoBroadcastJoinThreshold`, which defaults to 10 MB. It only does so when it has statistics saying the table is small — and in

this measurement, a DataFrame built from a Python list carried no reliable size estimate, so the automatic decision did not fire and an explicit hint was needed:

```
from pyspark.sql.functions import broadcast

result = big_fact.join(broadcast(small_dim), "city")
```

That is a genuinely common production situation. When you know a side is small and the plan says `SortMergeJoin`, hint it.

9.8 Data skew

Spark divides work by key. When one key holds most of the rows, one task holds most of the work, and 199 of your 200 tasks finish in seconds while the last one runs for an hour.

Here is deliberate skew, measured:

```
rows per key (top 4):
  BIG_SELLER      1,840,000  ( 92.0% of all rows)
  seller_32           6,667  (  0.3% of all rows)
  seller_36           6,667  (  0.3% of all rows)
  seller_39           6,667  (  0.3% of all rows)
```

One key holds 92% of two million rows. This is not an artificial scenario — it is exactly what real data looks like. One marketplace seller dominates a category. One `NULL` customer id absorbs every unmatched row. One country holds most of the traffic.

Recognising it is the first skill. In the Spark UI, open the slow stage and look at the task duration distribution. Skew looks like a median of two seconds and a maximum of forty minutes. A general volume increase moves all task times up together; skew moves one.

Fixing it has three standard answers.

Adaptive Query Execution is the first thing to try, because it costs nothing. AQE watches the actual shuffle output at runtime and splits oversized partitions automatically. Measured on the skewed aggregation above:

```
skewed aggregation, AQE off   0.77 s
skewed aggregation, AQE on    0.40 s
```

Nearly twice as fast from a configuration flag, on the same code and the same data. AQE is enabled by default from Spark 3.2 onward, and if you inherit an older job with it disabled, turning it on is often the cheapest win available.

Salting is the manual fix for cases AQE cannot handle. Add a random suffix to the hot key so it spreads across many partitions, aggregate, then strip the suffix and aggregate again:

```
from pyspark.sql import functions as F

SALT = 16
salted = (skewed
          .withColumn("salt", (F.rand() * SALT).cast("int"))
          .withColumn("salted_key", F.concat_ws("#", "key", "salt")))

partial = salted.groupBy("salted_key").agg(F.sum("amount").alias("part"))

final = (partial
         .withColumn("key", F.split("salted_key", "#").getItem(0))
         .groupBy("key")
         .agg(F.sum("part").alias("amount")))
```

Filtering the hot key out and handling it separately is the third option, and often the simplest. If the skew comes from a `NULL` join key, the honest fix is usually to exclude those rows from the join entirely, because they were never going to match anything.

9.9 Caching

`cache()` keeps a DataFrame in executor memory so later actions do not recompute it.

```
cleaned = orders.filter(col("status") == "DELIVERED").cache()
cleaned.count()                          # computes and caches
by_city = cleaned.groupBy("city").sum("amount")      # reuses the cache
by_month = cleaned.groupBy("month").sum("amount")    # reuses the cache
```

Cache when a DataFrame is expensive to compute and used more than once. Do not cache something used once — you pay the memory cost for no benefit and may evict something that was earning its place.

`cache()` is `persist(MEMORY_AND_DISK)`. Spark keeps what fits in memory and spills the rest to disk. And caching is lazy like everything else: nothing is cached until an action runs. Call `unpersist()` when finished, particularly in long jobs, or cached data quietly consumes memory that later stages need.

9.10 Memory, and why jobs vanish

Chapter 2 said that a job dying without a stack trace is almost always memory. Here is the Spark-specific version.

Executor memory is divided between **execution memory** — used for shuffles, joins, sorts and aggregations — and **storage memory**, used for cached data. They share a pool and borrow from each other, which is why aggressive caching can starve a shuffle and cause an out-of-memory failure in a stage that has nothing to do with the cache.

The four common causes, in the order to check them:

One partition is far too large. Usually skew. Fix the skew, not the memory setting.

`collect()` on something large. This pulls the entire dataset into the driver. Use `take(n)` or `show()` while developing, and `write` when you mean it.

Too many partitions cached. Memory pressure from storage crowding out execution.

A genuinely undersized executor. The least common cause, and the first one everyone reaches for.

> **COMMON BEGINNER MISTAKES WITH SPARK**
>
> - Using Spark for data that fits on one machine, and being slower than pandas.
> - Calling `collect()` on a large DataFrame.
> - Writing a Python `for` loop over rows instead of a transformation, which forfeits every optimisation Spark has.
> - Never running `explain()` and therefore never knowing how many shuffles the code causes.
> - Not broadcasting a small dimension, and shuffling a billion-row fact table for no reason.
> - Caching everything, or caching something used once.
> - Accepting the default output partition count and writing thousands of small files.
> - Raising executor memory to fix what is actually skew.

9.11 PySpark in practice

```
from pyspark.sql import SparkSession, functions as F

spark = SparkSession.builder.appName("daily_orders").getOrCreate()

orders = spark.read.parquet("s3://shopkart/bronze/orders/")
cities = spark.read.parquet("s3://shopkart/dim/cities/")

daily = (orders
         .filter(F.col("order_date") == "2026-03-17")     # pushed down
         .filter(F.col("status") == "DELIVERED")
         .join(F.broadcast(cities), "city")      # no shuffle on orders
         .groupBy("region", "order_date")
         .agg(F.sum("amount").alias("revenue"),
              F.countDistinct("order_id").alias("orders")))

(daily
 .coalesce(4)                        # 4 files, not 200
 .write
 .mode("overwrite")                  # idempotent re-run
 .partitionBy("order_date")
 .parquet("s3://shopkart/gold/daily_revenue/"))
```

Five deliberate choices in twelve lines: filter before joining so less data moves, broadcast the small side, aggregate at a declared grain, coalesce before writing to control file count, and `overwrite` the partition so a re-run replaces rather than appends. That last one is Chapter 8's idempotency, expressed in Spark.

Summary

- Spark exists for data that will not fit or finish on one machine. Below that threshold it is usually slower than the simpler tool.
- The driver plans; executors run one task per partition per core.
- Two to four partitions per core at roughly 100–200 MB is a starting heuristic, not a rule; measure and adjust, and remember AQE may recoalesce at runtime. `repartition` shuffles, `coalesce` does not.
- Transformations are lazy; actions execute. Laziness is what allows optimisation.
- Narrow transformations stay in place. Wide ones shuffle. `Exchange` in a plan means shuffle.
- Broadcast the small side of a join. Measured here the plan changed from `SortMergeJoin` to `BroadcastHashJoin` and dropped one `Exchange`.

- Skew means one task doing most of the work. Try AQE first — measured at 0.77 s down to 0.40 s — then salting, then filtering the hot key out.
- Cache what is expensive and reused. Nothing else.
- A job that vanishes without a trace is memory, and usually skew or a `collect()` rather than an undersized executor.

In the interview

You have a 20 TB dataset and one key holds 70% of the rows. Your Spark job is extremely slow. What is happening and how do you fix it?

SHORT ANSWER

That is data skew. Spark partitions by key on a shuffle, so 70% of the rows land in one partition and one task does most of the work while everything else finishes early. The job's duration becomes the duration of that single task, and adding executors does not help because they sit idle.

DEEPER ANSWER

I would confirm it before fixing it, by looking at the task duration distribution in the slow stage. Skew shows a median measured in seconds and a maximum measured in hours, with total input bytes unchanged. That distinguishes it from a general volume increase, where every task gets slower together.

Then, in order of cost: enable Adaptive Query Execution, which splits oversized shuffle partitions at runtime and often resolves it with no code change. I have measured a skewed aggregation going from 0.77 seconds to 0.40 on a small local dataset just from that flag. If AQE is not enough, salt the hot key — add a random suffix so it spreads across partitions, aggregate, strip the suffix and aggregate again. If the skew is a join and the other side is small, broadcast it and there is no shuffle to skew in the first place.

FOLLOW-UP: WHAT IF THE HOT KEY IS `NULL`?

Then the right question is why those rows are in the join at all. `NULL` never matches anything, so they are doing work to produce nothing. I would filter them out before the join and handle them explicitly — usually they are a data quality signal worth reporting rather than a performance problem worth optimising.

SENIOR-LEVEL ANSWER

I would also want to know whether this is a one-off or the shape of the data. If one seller produces most of the volume every day, salting is a permanent workaround for a modelling problem, and the durable fix might be to partition the physical layout differently, pre-aggregate that key upstream, or split the pipeline so the dominant key is processed on its own path. Optimising the query is the right response to a surprise; changing the design is the right response to a pattern.

PRACTICE

1. Install PySpark locally and run the Field Kit script. Reproduce the narrow and wide plans and find the word `Exchange` yourself.
2. Take any query you have written and run `explain()`. Count the `Exchange` nodes. Then rewrite it to have fewer.
3. Build a skewed dataset where one key holds 90% of the rows. Run an aggregation with AQE off and on, and compare.
4. Implement salting by hand on that dataset and confirm you get the same answer as the unsalted version.
5. Join a large DataFrame to a small one with and without `broadcast()`. Compare the plans, then compare the times, and explain why the times may be similar at small scale.
6. Call `collect()` on something too large for your driver. Read the error. You now recognise it forever.

SOURCES AND VERIFICATION

All plans, partition counts and timings in this chapter were produced by a real Apache Spark 4.2 session running in local mode on four cores over a two-million-row generated dataset: the partition counts, the lazy evaluation timings, the narrow and wide physical plans, the join strategy comparison, the skew distribution, and the AQE timings.

Two limitations are stated in the text rather than smoothed over. The broadcast and sort-merge join timings were nearly identical at this scale, because two million rows on one machine is not enough for the shuffle difference to dominate; the plan difference is what scales. And an automatic broadcast did not fire for a DataFrame built from a Python list because it carried no size statistics, which is why an explicit hint was used.

Reference material: the Apache Spark SQL programming guide and the performance tuning documentation, particularly the sections on Adaptive Query Execution and join strategies.

10

CHAPTER TEN

Orchestration with Airflow

You now have forty scripts. One pulls from an API, one lands files, one runs Spark, one loads the warehouse, one refreshes a dashboard. Cron runs each at a fixed time and knows nothing else. When the API job runs eleven minutes late, the Spark job starts anyway, reads a directory that is half written, succeeds, and produces a number that is quietly wrong.

> VERSIONS AND DIALECTS
>
> Airflow code in this chapter targets **Airflow 2.4 and later**, where `schedule=` replaced `schedule_interval` and data intervals replaced execution dates. On Airflow 3 the concepts hold but some import paths and defaults differ — check the version your team runs.

That is the problem orchestration solves. Not scheduling — cron schedules perfectly well. **Dependency**. The difference between *run this at two o'clock* and *run this when the thing it needs is actually ready*.

Cron can	Cron cannot
Run a command at a time	Know that B depends on A
Log output to a file	Retry only the step that failed
Send mail on failure	Re-run last Tuesday
	Show you where a pipeline is right now
	Stop a job whose input never arrived

10.1 The DAG

Airflow describes a pipeline as a **DAG** — a directed acyclic graph. Directed because work flows one way. Acyclic because nothing may loop back on itself, which is what makes the order computable.



FIGURE 10.1 A DAG is a dependency statement, not a timetable. Airflow works out the order and the parallelism from the arrows.

The vocabulary is small:

- A **task** is one unit of work.
- An **operator** is the template a task is built from — run a Python function, run a bash command, submit a Spark job.
- The **scheduler** decides what is ready to run.
- An **executor** decides where it runs — one machine, a pool of workers, a Kubernetes pod per task.
- A **sensor** is a task that waits for something rather than doing something.

10.2 A real DAG

```
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.operators.bash import BashOperator

default_args = {
    "owner": "data-eng",
    "retries": 3,
    "retry_delay": timedelta(minutes=5),
    "retry_exponential_backoff": True,
    "email_on_failure": True,
}

with DAG(
    dag_id="shopkart_daily_orders",
    default_args=default_args,
    schedule="0 2 * * *",              # 02:00 every day
    start_date=datetime(2026, 3, 1),
    catchup=False,                     # see 10.5 before changing this
    max_active_runs=1,
    tags=["orders", "daily"],
) as dag:

    fetch = PythonOperator(
        task_id="fetch_orders_api",
        python_callable=fetch_orders,
        op_kwargs={"run_date": "{{ ds }}"},     # the logical date
    )
```

```
    transform = BashOperator(
        task_id="transform_spark",
        bash_command=(
            "spark-submit /opt/jobs/transform_orders.py "
            "--date {{ ds }}"
        ),
    )

    load = PythonOperator(
        task_id="load_warehouse",
        python_callable=load_to_snowflake,
        op_kwargs={"run_date": "{{ ds }}"},
    )

    check = PythonOperator(
        task_id="validate_row_counts",
        python_callable=reconcile_counts,
        op_kwargs={"run_date": "{{ ds }}"},
    )

    fetch >> transform >> load >> check
```

The last line is the whole dependency graph. Airflow reads it and works out that `transform` waits for `fetch`, and that nothing downstream runs if an upstream task fails.

Two things in that file matter more than the operators.

`{{ ds }}` is the **logical date** — the date the run *represents*, not the date it executes. A run for 17 March that is retried on 19 March still processes 17 March. Every task takes the date as a parameter rather than calling `date.today()`, and that single discipline is what makes re-runs and backfills correct.

The retry block belongs in `default_args` so it applies to every task. Three retries with exponential backoff absorbs the transient network failures that otherwise wake somebody up.

10.3 Idempotency again, from the other side

Airflow will re-run your tasks. After a failure, after a code fix, after a backfill, when somebody clears a task in the UI. This is not an edge case; it is the normal operating mode.

THE RULE THAT MAKES ORCHESTRATION SAFE

Every task must produce the same result when run twice for the same logical date. Chapter 8 gave you the mechanisms — upsert on a key, replace the partition, deduplicate on ingestion. Airflow is where the absence of them becomes visible daily rather than occasionally.

The two habits that follow:

Parameterise by logical date, never by `now()`. A task that computes yesterday from the wall clock produces different results depending on when it happens to run, which makes every retry a new bug.

Write to a location derived from the logical date. `s3://.../orders/dt=2026-03-17/` with `mode="overwrite"`. A re-run replaces that partition and nothing else.

10.4 Sensors and the waiting problem

A **sensor** waits for a condition — a file arriving, a partition appearing, another DAG finishing.

```
from airflow.providers.amazon.aws.sensors.s3 import S3KeySensor

wait_for_file = S3KeySensor(
    task_id="wait_for_partner_file",
    bucket_key="s3://shopkart/partner/dt={{ ds }}/deliveries.csv",
    poke_interval=300,          # check every 5 minutes
    timeout=60 * 60 * 4,        # give up after 4 hours
    mode="reschedule",          # free the worker slot while waiting
)
```

Two settings here are not optional in production.

`timeout` must be set. A sensor with no timeout waits forever, holds resources forever, and hides the fact that a partner never sent the file. Failing after four hours is information; waiting silently is not.

`mode="reschedule"` releases the worker slot between checks. The default, `poke`, holds a slot the whole time, and a handful of long sensors in poke mode will occupy every slot you have and deadlock the scheduler while doing nothing at all. This is one of the most common self-inflicted Airflow outages.

10.5 Backfill and catchup

`catchup` decides what happens when a DAG's `start_date` is in the past. With `catchup=True`, Airflow immediately schedules every interval between the start date and now.

Turning on a DAG with a start date six months ago and `catchup=True` will queue about 180 runs at once. If they all hit the same source database, you have created an incident on your first day.

Set `catchup=False` by default. When you need history, run a deliberate backfill with controlled parallelism:

```
airflow dags backfill shopkart_daily_orders \
    --start-date 2026-01-01 --end-date 2026-03-31
```

Combine that with `max_active_runs=1` and the backfill proceeds one day at a time, in order, through exactly the same code path as the daily run. That last point is from Chapter 8 and it still holds: a separate backfill script drifts and eventually produces history that disagrees with the present.

10.6 Passing data between tasks

XCom lets one task pass a value to another. It is stored in Airflow's metadata database, and that fact is the whole guidance: **pass pointers, never payloads**.

```
# Good — a small reference
return f"s3://shopkart/bronze/orders/dt={run_date}/"

# Bad — a DataFrame or a list of rows through the metadata database
return df.to_dict()
```

Tasks should communicate through storage. Task A writes to a path, task B reads that path. XCom carries the path, a row count, a batch id. The moment real data flows through XCom, your Airflow database becomes a bottleneck and then an outage.

10.7 What good looks like in the UI

Four things you will use constantly:

The **Grid view** shows every run as a column and every task as a row, coloured by state. A horizontal red stripe means one task fails every day — a code problem. A vertical red stripe means one day failed entirely — an environment or data problem. Learning to read that grid at a glance is most of Airflow operations.

Clear re-runs a task and everything downstream. It is the standard recovery action, and it is only safe because your tasks are idempotent.

Connections and Variables hold credentials and configuration outside your code, which is Chapter 2's rule with a user interface attached.

SLA and duration tracking tells you a task that used to take eight minutes now takes fifty. That trend is visible weeks before it becomes an incident, and almost nobody looks at it.

10.8 Running it, and watching the gate hold

Everything above is theory until the scheduler runs it. Here is the DAG from 10.2 wired to the project from Chapter 13, executed on Airflow 2.10.5.

```
$ airflow tasks list orders_daily --tree
<Task(BashOperator): ingest>
    <Task(BashOperator): transform>
        <Task(BashOperator): publish>
```

The dependency line `ingest >> transform >> publish` produced that tree. You never wrote the order; Airflow derived it.

The good run

```
Running command: ['bash','-c','cd /tmp/proj && python3 ingest.py 2026-03-17']
  fetched 9 pages, wrote 438 rows to bronze/dt=2026-03-17/orders.json
Marking task as SUCCESS. task_id=ingest

Running command: ['bash','-c','cd /tmp/proj && python3 transform.py
2026-03-17']
  verdict               PASS
Marking task as SUCCESS. task_id=transform

Running command: ['bash','-c','cd /tmp/proj && python3 publish.py
2026-03-17']
  published 3 region rows for 2026-03-17
Marking task as SUCCESS. task_id=publish
```

Note the date. `{{ ds }}` was rendered to `2026-03-17` and passed to all three scripts, so every task processed the same logical day regardless of when the run actually happened. That is section 10.2's logical date doing its job.

The run where the data was wrong

Now the same DAG with the volume check failing, simulating the source sending 40% of its rows. This is the case that matters.

```
Marking task as SUCCESS. task_id=ingest

dag_id       | task_id   | state
-------------+-----------+--------------
orders_daily | ingest    | success
orders_daily | transform | up_for_retry
orders_daily | publish   | None
```

`publish` never started. The bad table was never served. That is the entire argument for exit codes, and here it is enforced by the scheduler rather than asserted in prose.

The middle row is the part worth sitting with. `transform` is not `failed` — it is **`up_for_retry`**, because `default_args` set three retries with exponential backoff. Airflow will wait, run it again, and it will fail again, because a volume problem in the source is not a transient error.

That reveals something the retry section did not say plainly:

CLASSIFY THE FAILURE BEFORE YOU SET THE RETRY POLICY

Retries help with **transient** failures — the ones where the same request may succeed a moment later:

- a network timeout or a reset connection
- an API returning 429, 502 or 503
- a broker or database briefly unavailable
- a cluster still starting up
- a lock or a resource contended for a few seconds

Retries do not help with **deterministic** failures — the ones where the same input produces the same result every time:

- a duplicate key violating a constraint
- a schema that no longer matches
- a reconciliation that does not balance
- an impossible value, such as a negative quantity
- a data quality rule that the data genuinely fails
- source volume below a hard threshold

Retrying a deterministic failure does not change the outcome. It delays the alert by the length of your backoff and leaves the DAG looking active while nothing is being fixed — which is worse than failing at the first attempt, because somebody may see “retrying” and assume it is recovering.

This is a classification exercise, not a rule about data quality tasks. Some validation failures genuinely are transient: a freshness check that fails because an upstream file is ten minutes late will pass on the next attempt, and one retry with a short delay is exactly right there. A grain violation will not.

So set the policy per task, and where a single task can fail both ways, split it: retry the part that talks to the outside world, and fail fast on the part that judges the data.

COMMON BEGINNER MISTAKES WITH AIRFLOW

- Putting real work in the DAG file itself. The scheduler parses every DAG file repeatedly — often every thirty seconds — so a database query at the top level runs constantly.
- Using `datetime.now()` instead of `{{ ds }}`, which makes retries non-deterministic.
- `catchup=True` on a DAG with an old start date.
- Sensors with no timeout, or sensors left in `poke` mode.
- Passing DataFrames through XCom.
- One enormous task that does fetch, transform and load together, so a failure at the end re-does everything.
- Tasks so small that scheduling overhead exceeds the work.
- Treating Airflow as a processing engine. It orchestrates; Spark and the warehouse compute.

Summary

- Orchestration exists for dependencies, not scheduling. Cron cannot express “when A is actually finished”.
- A DAG declares dependencies; Airflow derives order and parallelism.
- Parameterise every task by the logical date `{{ ds }}`, never by the wall clock.
- Retries with exponential backoff belong in `default_args`.
- Sensors need a `timeout` and `mode="reschedule"`. Without both they deadlock your workers.
- `catchup=False` by default. Backfill deliberately, with `max_active_runs` set.
- XCom carries pointers. Data moves through storage.
- Split pipelines into tasks that are worth retrying independently.

In the interview

Why use Airflow instead of cron?

SHORT ANSWER

Dependencies. Cron runs things at times; Airflow runs things when their inputs are ready. It also gives retries per task, backfills over a date range, visibility into where a pipeline is, and a place to keep credentials and configuration out of the code.

FOLLOW-UP: WHAT MAKES A DAG SAFE TO RE-RUN?

Every task being idempotent for its logical date, and every task taking that date as a parameter rather than reading the clock. Then clearing a task in the UI is a routine action instead of a risk, which matters because clearing tasks is how you recover from almost everything.

SENIOR-LEVEL ANSWER

I would add that the most common Airflow incidents are self-inflicted and architectural rather than bugs — sensors in poke mode exhausting the worker pool, a catchup storm hammering a source, heavy work inside the DAG file slowing the scheduler for everyone. The design questions are how tasks are split, how they communicate, and what the retry policy is; the operators themselves are the easy part.

PRACTICE

1. Install Airflow locally and build the four-task DAG from 10.2 with stub functions. Run it once.
2. Make one task fail deliberately. Confirm downstream tasks do not run, then fix it and clear the failed task.
3. Replace `{{ ds }}` with `datetime.now()`, clear a run from three days ago, and observe that it processes the wrong day.
4. Add a sensor with no timeout. Watch it wait. Then add the timeout and `mode="reschedule"`.
5. Set `catchup=True` with a start date one month back on a DAG that only prints. Count the runs it queues.

SOURCES AND VERIFICATION

The DAG in this chapter was executed on Airflow 2.10.5 with a local metadata database. Both runs shown in section 10.7 are actual output: the successful run of all three tasks, and the run where the quality gate failed and left `transform` in `up_for_retry` while `publish` never started. The DAG file and the pipeline scripts are in the companion Field Kit.
Reference material: the Apache Airflow documentation on DAG authoring, sensors, backfill and catchup, and XCom.

PART FIVE

Running It

Streams that never stop, and the failure that defines this profession: the pipeline that succeeds while producing wrong data.

11 · Streaming and Kafka

12 · When Pipelines Lie

11

CHAPTER ELEVEN

Streaming and Kafka

ShopKart's order service writes an order and then has to tell six other systems: warehouse, payments, fraud, notifications, analytics, and the seller dashboard. Calling all six directly means the order service knows about six systems, waits for six responses, and fails when any one of them is down. Adding a seventh consumer means changing the order service and deploying it again.

> VERSIONS AND DIALECTS
>
> Kafka configuration in this chapter follows **Apache Kafka 3.x**. The Structured Streaming example targets **Spark 3.x or later**.

The alternative is to write the fact once, somewhere durable, and let each system read it independently at its own pace. That place is a **log**, and Kafka is the industry's log.

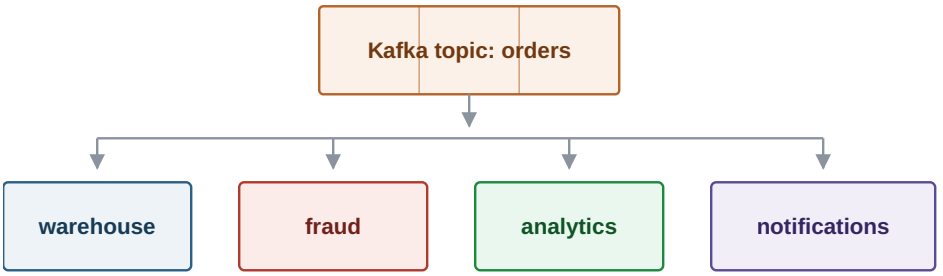


FIGURE 11.1 One write, many independent readers. The producer does not know who consumes, and a slow or dead consumer affects nobody else.

11.1 The vocabulary

A **producer** writes messages. A **consumer** reads them. A **broker** is a server holding data; a cluster is several brokers.

A **topic** is a named stream — `orders`, `payments`, `clicks`. A topic is split into **partitions**, and this is where the design becomes interesting.

Each partition is an **append-only, ordered sequence**. Every message in it has an **offset**, its position. Kafka does not delete a message when it is read; it keeps it for a configured **retention** period, which is why several consumers can read the same data independently, and why you can replay history.



FIGURE 11.2 A topic is a set of partitions. Order is guaranteed within a partition and nowhere else, which is the single most misunderstood fact about Kafka.

THE THING TO REMEMBER

Kafka guarantees ordering *within a partition*, never across a topic. If every event for one order must be processed in order, all of them must land in the same partition — which means producing them with the same key. Get that wrong and a cancellation can be processed before the order it cancels.

Partitioning is by key: Kafka hashes the message key and picks a partition. Key by `order_id` and every event for an order is ordered. Send no key and messages are spread evenly with no ordering guarantee at all.

A **consumer group** is how you scale reading. Each partition is assigned to exactly one consumer in a group, so three consumers across six partitions get two each. Add a fourth and Kafka **rebalances**. Two consequences follow: more consumers than partitions leaves some idle, so **partition count is your maximum parallelism**; and two different groups reading the same topic each get every message, which is what makes fan-out work.

Which partition does a key land in?

This is worth showing rather than asserting, because “key by entity and ordering is preserved” only means something once you have seen the mechanism.

Kafka’s default partitioner hashes the message key with **murmur2** and takes the result modulo the partition count. Here is the real client partitioner, on a topic with six partitions:

```
key 90000001       -> partition 5   (murmur2 = 3881133991)
key 90000002       -> partition 4   (murmur2 = 3385203162)
key 90000003       -> partition 3   (murmur2 = 1583969691)
key BIG_SELLER     -> partition 3   (murmur2 = 3971053589)

same key, ten calls: [5, 5, 5, 5, 5, 5, 5, 5, 5, 5]
```

That last line is the **mechanism** the ordering guarantee rests on, and it is worth being precise about the difference. What this experiment demonstrates is client-side determinism: order 90000001 always maps to partition 5, so every event for that order is sent to the same log. The guarantee itself — that a partition is an append-only sequence and a consumer reads it in offset order — is a broker property, and this experiment did not test a broker. It tested the routing decision that makes the guarantee usable.

Which is still the part beginners get wrong. Ordering "works" because keying is deterministic, not because Kafka sorts anything.

Now the distribution. Sixty thousand sequential order ids across six partitions:

```
partition 0:   9948 (16.6%)     partition 3:   9981 (16.6%)
partition 1:  10022 (16.7%)     partition 4:  10218 (17.0%)
partition 2:   9959 (16.6%)     partition 5:   9872 (16.5%)
```

Even, as you would hope — 16.7% is a perfect sixth. But key by something skewed instead, and the same sixty thousand events do this:

```
partition 3:  60000 (100.0%)
```

One partition holds everything. Five consumers idle while one drowns. This is the **hot partition** problem, and it is the streaming twin of Chapter 9's data skew — same cause, same shape, different system. Adding partitions does not help, and adding consumers does not help, because parallelism in Kafka is bounded by partitions and every event is going to one of them.

The fix is the same family as Chapter 9's: choose a key with enough distinct values, or add a salt to the hot key and accept that ordering is then only guaranteed within each salted sub-stream. Which is a real trade-off, not a free fix — you cannot have both global ordering for a key and parallelism across it.

11.2 Replication

Each partition has a **leader** on one broker and **followers** on others. Producers and consumers talk to the leader; followers copy it. If the leader's broker dies, a follower that is sufficiently caught up — an in-sync replica — is promoted.

Two settings decide how much you can lose:

```
replication.factor=3        # three copies of every partition
min.insync.replicas=2       # a write must reach at least two
acks=all                    # producer waits for those two
```

With `acks=1` the producer is told the write succeeded as soon as the leader has it. If that broker dies before a follower copies the message, the message is gone and nobody knows. `acks=all` with `min.insync.replicas=2` costs latency and buys durability. For orders and payments, pay it.

11.3 Consumer lag

Consumer lag is the gap between the latest offset in a partition and the offset a consumer has processed. It is the single most important metric in a streaming system, and it is what you alert on.

Steady lag means the consumer is keeping up. Growing lag means it is falling behind, and the shape of the growth tells you what is wrong.

Pattern	Usually means
Lag grows steadily on all partitions	The consumer is too slow for the volume — scale out, up to the partition count
Lag grows on one partition only	A hot key. Rethink the partition key
Lag jumps, then recovers	A restart or rebalance. Normal, unless frequent
Lag flat and non-zero, no progress	The consumer is stuck — often a poison message it retries forever
Lag hits zero and stays	Healthy, or the producer stopped. Check both

If lag grows for long enough that data ages past the retention period, **messages are deleted before they are ever read**. Retention is therefore a data-loss deadline, not a storage setting.

11.4 Delivery semantics

Three guarantees, in ascending order of cost.

At most once. Commit the offset before processing. If the consumer dies mid-work, that message is skipped. Fast, lossy. Acceptable for metrics, never for orders.

At least once. Process, then commit. If the consumer dies after processing but before committing, the message is redelivered. Nothing is lost; duplicates happen. **This is the default and the right default.**

Exactly once. Kafka can provide it within Kafka, using idempotent producers and transactions that commit offsets and output atomically. It is useful and frequently oversold, because it does not extend to whatever external system you write to.

THE PRACTICAL ANSWER

Use at-least-once delivery and make the consumer idempotent. Then a duplicate is harmless, and you have not bought a distributed transaction you did not need. That is the same idempotency from Chapter 8 arriving from a third direction, which should tell you how central it is.

11.5 Schemas and poison messages

A topic with no schema is a shared mutable structure that dozens of teams write to. Producers change a field type on a Tuesday and every consumer breaks on Wednesday.

A **schema registry** stores the schema for each topic, versions it, and rejects producer changes that would break existing consumers. Adding an optional field is compatible; removing a field or changing a type is not. This is Chapter 8's schema evolution problem, enforced at the boundary instead of discovered downstream.

A **poison message** is one the consumer cannot process — malformed, a null where the code assumes a value. With at-least-once delivery, the consumer fails, does not commit, receives the same message again, and fails again. Forever. Lag stops moving and no data flows.

The fix is a **dead-letter queue**: after a bounded number of attempts, write the message to a separate topic with the error attached, commit the offset, and continue.

```
for message in consumer:
    try:
        process(message)
    except RetryableError:
        raise                                # let the retry policy handle it
    except Exception as exc:
        dead_letter.send("orders.dlq", {
            "payload": message.value,
            "error": str(exc),
            "topic": message.topic,
            "partition": message.partition,
            "offset": message.offset,
        })
    consumer.commit()
```

A dead-letter queue that nobody reads is a silent data loss mechanism with extra steps. Alert on its depth.

11.6 Streaming semantics: event time and windows

The hard part of streaming is not throughput. It is time.

Event time is when something happened. **Processing time** is when your system saw it. They differ, sometimes by hours — a phone was offline, a network partition healed, a consumer restarted.

Neither is right in general; they answer different questions.

Use event time when the question is about the business. "How many orders were placed between 09:00 and 10:00?" has one true answer and it does not change because your consumer restarted. Aggregating that by processing time puts an event from 09:58 that arrived at 10:03 in the wrong hour, and your numbers never reconcile with the source. Event time costs you a decision about how long to wait for stragglers.

Use processing time when the question is about the system. "How many events did we ingest in the last minute?", "is the consumer keeping up?", "should we rate-limit this producer?" — these are questions about your infrastructure, and the answer *should* move when the system misbehaves. Processing time is also cheaper: no watermark, no state held open, no late-data policy.

The failure is not choosing one. It is using processing time for a business question, or event time for an operational one, and then being surprised that the number does not mean what someone assumed.

A **watermark** — the same word as Chapter 8, a different mechanism — is the system's statement that it does not expect events older than a certain point. Anything later is **late data**, and you must choose: drop it, or reopen the window and emit a correction. Neither is free, and the business has to pick.

```
(events
 .withWatermark("event_ts", "30 minutes")
 .groupBy(window("event_ts", "1 hour"), "city")
 .agg(sum("amount")))
```

That declares a thirty-minute tolerance for lateness. Chapter 9's Spark is doing the work; the streaming-specific part is the watermark and the window.

11.7 When not to stream

Streaming costs a permanently running system, harder debugging, harder reprocessing, and an on-call rotation.

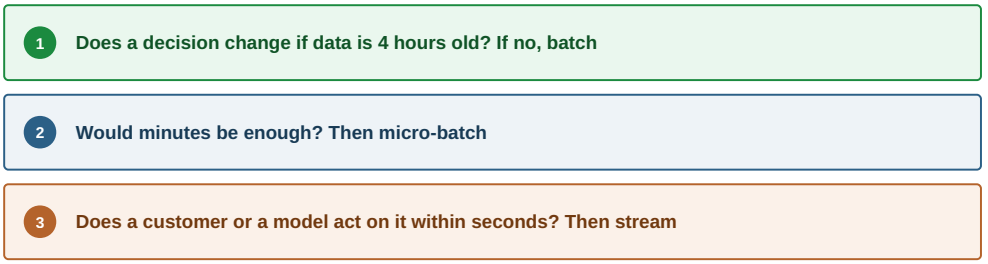


FIGURE 11.3 The decision, in order. Most teams arrive at micro-batch and stop, which is usually correct.

Fraud scoring, delivery estimates, live inventory and alerting need seconds. A daily revenue report does not, and a dashboard a human opens twice a day certainly does not.

COMMON BEGINNER MISTAKES WITH KAFKA

- Assuming a topic is ordered. Only a partition is.
- Producing without a key when ordering per entity matters.
- Choosing a partition count too low to scale, or so high that rebalances become expensive.
- `acks=1` on data you cannot afford to lose.
- Not monitoring consumer lag, which is the only number that tells you the system is healthy.
- No dead-letter queue, so one malformed message halts a partition indefinitely.
- Treating retention as storage policy rather than as a deadline for consumers.
- Aggregating on processing time and then wondering why hourly totals never reconcile.

Summary

- Kafka is a durable, replayable log. One write, many independent readers.
- Ordering is guaranteed within a partition only. Key by entity when order matters.
- Consumer group parallelism is capped by partition count.
- `acks=all` with `min.insync.replicas=2` is the durable configuration.
- Consumer lag is the health metric. The shape of its growth diagnoses the cause.
- Use at-least-once delivery and make consumers idempotent, rather than reaching for exactly-once.
- A schema registry enforces compatibility at the boundary. A dead-letter queue stops one bad message halting a partition.
- Event time answers business questions; processing time answers questions about your own system. Using the wrong one is what makes totals irreconcilable. Watermarks decide how long you wait for late events.
- Stream only when a decision genuinely changes with seconds of freshness.

In the interview

Consumer lag on one topic has been climbing for six hours. Walk me through it.

SHORT ANSWER

First I would check whether it is all partitions or one. All partitions climbing means the consumer cannot keep up with volume, so I would scale out consumers up to the partition count, and check whether processing per message got slower. One partition climbing means a hot key, and the fix is the partition key rather than more consumers.

DEEPER ANSWER

I would also check whether lag is climbing or simply flat and non-zero. Flat with no progress usually means a poison message being retried forever, which is a dead-letter queue problem, not a scaling problem. And I would check for repeated rebalances, because a consumer that keeps timing out and rejoining spends its life rebalancing and never processes anything.

The urgent question underneath all of this is retention. If the lag in time terms is approaching the retention period, messages will be deleted before they are read and that loss is unrecoverable. That converts a performance problem into an incident with a deadline.

SENIOR-LEVEL ANSWER

I would want to know why nobody was paged three hours earlier. Lag is the one metric that predicts this class of failure, and it should alert on both absolute value and rate of change well before it approaches retention. Most streaming incidents I would expect to see are visible for hours in a metric nobody had an alert on.

> PRACTICE
>
> 1. Run Kafka locally with Docker. Create a topic with three partitions.
> 2. Produce messages with and without a key. Consume from all partitions and observe where ordering holds and where it does not.
> 3. Start two consumers in one group, then a third. Watch the partition assignment rebalance. Start a fourth and confirm it is idle.
> 4. Produce a malformed message and let your consumer crash on it. Watch lag stop moving. Then add a dead-letter queue and confirm the partition recovers.

5. Set retention to five minutes, stop your consumer for ten, and confirm the messages are gone. Do this once so the risk is real to you.

SOURCES AND VERIFICATION

Partition assignment in section 11.1 was executed: the murmur2 values, the partition each key maps to, the even distribution across sixty thousand sequential ids, and the hot-partition result are all actual output from Kafka's default client partitioner.

The broker-side material — replication, consumer group rebalancing, delivery semantics and retention — was **not** executed, because running a broker requires infrastructure beyond the container used to build this book. Those sections are worked patterns, and the practice tasks are written so you can reproduce every claim yourself with Docker in about an hour. This distinction is stated plainly rather than left for you to guess.

Reference material: the Apache Kafka documentation on topics, partitions, consumer groups, replication and delivery semantics, and the Spark Structured Streaming guide for watermarks and event-time windows.

12

CHAPTER TWELVE

When Pipelines Lie

Every green tick in this book so far has been earned. This chapter is about the ones that are not. A pipeline that crashes is a good pipeline having a bad day — you get an alert, you fix it, the numbers are late but correct. A pipeline that **succeeds while producing wrong data** is the failure that defines this profession, and it is why senior data engineers are paid what they are.

You have already seen four of them, each measured:

Chapter	The failure	The tell
3	A join fan-out inflated revenue by 44.9%	Every job green
4	Splitting a CSV on commas shifted every column	Every job green
5	An SCD1 overwrite moved ₹4,749 between cities retroactively	Every job green
8	An appending load tripled revenue in three runs	Every job green

None threw an exception. All four would have reached a dashboard. This chapter is the systematic answer.

12.1 Monitor the data, not the job

Job monitoring answers *did the code finish*. Data monitoring answers *is the output plausible*. You need both, and almost every team has only the first.



FIGURE 12.1 Two independent questions. A green job with a red dataset is the case that hurts, and only the right-hand column can see it.

Six checks cover most of it. Run them before publishing, not after.

Freshness. Does this table contain data for the period it should? The most valuable single check, because a stalled pipeline usually looks fine from every other angle.

Volume. Is today's row count within a sane range of recent days? A source sending 60% of normal volume produces a plausible-looking number that is simply wrong.

Nulls. Has the null rate in a key column jumped? A join key that becomes 30% null means an upstream change, and it will silently drop rows.

Uniqueness. Is the declared grain still true? One row per order line means `COUNT(*) = COUNT(DISTINCT order_line_key)`. This is the check that catches every fan-out.

Referential integrity. Does every fact row's key exist in its dimension? Orphans mean a late-arriving dimension or a broken load.

Reconciliation. Does a summed measure agree with the source within a tolerance? It is the strongest check available and the least used.

```
-- run before the table is published, not after
WITH checks AS (
  SELECT
    (SELECT COUNT(*) FROM stg_orders WHERE order_date = :d)          AS rows_
today,
    (SELECT AVG(c) FROM (SELECT COUNT(*) c FROM stg_orders
        WHERE order_date BETWEEN :d - 7 AND :d - 1
        GROUP BY order_date))                                        AS rows_a
vg_7d,
    (SELECT COUNT(*) FROM stg_orders
        WHERE order_date = :d AND customer_id IS NULL)               AS null_
keys,
    (SELECT COUNT(*) - COUNT(DISTINCT order_line_key)
        FROM stg_orders WHERE order_date = :d)                       AS
```

```
dupes,
    (SELECT ABS(SUM(amount) - :source_total) FROM stg_orders
        WHERE order_date = :d)                                    AS rev-
enue_gap
)
SELECT *,
       CASE WHEN rows_today < 0.7 * rows_avg_7d THEN 'FAIL_VOLUME'
            WHEN null_keys > 0                  THEN 'FAIL_NULLS'
            WHEN dupes     > 0                  THEN 'FAIL_GRAIN'
            WHEN revenue_gap > 1.0              THEN 'FAIL_RECON'
            ELSE 'PASS' END AS verdict
FROM checks;
```

12.2 Warn, fail, or quarantine

Not every check should stop a pipeline. Three responses, chosen deliberately per check:

Fail the run. Nothing downstream sees bad data. Correct for grain violations, reconciliation breaks and missing partitions, where publishing is worse than being late.

Warn and continue. Log it, alert a human, publish anyway. Correct for a null rate that moved 2% — real signal, not worth an outage.

Quarantine the rows. Publish the good ones, route the bad ones to a side table with the reason attached. This is usually the best answer for row-level problems, and it is what Chapter 4's loader did.

> **CHOOSING BETWEEN THEM**
>
> Ask what a stakeholder would prefer: a dashboard that is four hours late, or a dashboard that is wrong and they do not know it. For anything financial or externally reported, late beats wrong every time. For an exploratory dataset, the opposite may be true. Write the answer down next to the check.

12.3 Data contracts

Most breakages are not bugs. They are a producer changing something they never promised not to change.

A **data contract** makes the promise explicit — schema, types, nullability, freshness, volume range, and who to contact:

```
dataset: orders
owner: order-platform-team
grain: one row per order
sla:
```

```
  freshness: 2 hours
  completeness: 99.9%
schema:
  order_id:    {type: integer, nullable: false, unique: true}
  customer_id: {type: integer, nullable: false}
  order_total: {type: decimal(10,2), nullable: false, min: 0}
  status:      {type: string, allowed: [PLACED, SHIPPED, DELIVERED,
                                        CANCELLED, RETURNED]}
breaking_changes: 30 days notice, announced in #data-platform
```

The value is not the file. It is that a producer now knows they made a promise, and a consumer has something to point at. The technical enforcement — validating incoming data against the contract and rejecting what violates it — is the easy half.

12.4 The debugging framework

The dashboard shows revenue 40% below normal. You have an hour. Work backwards, and bisect.

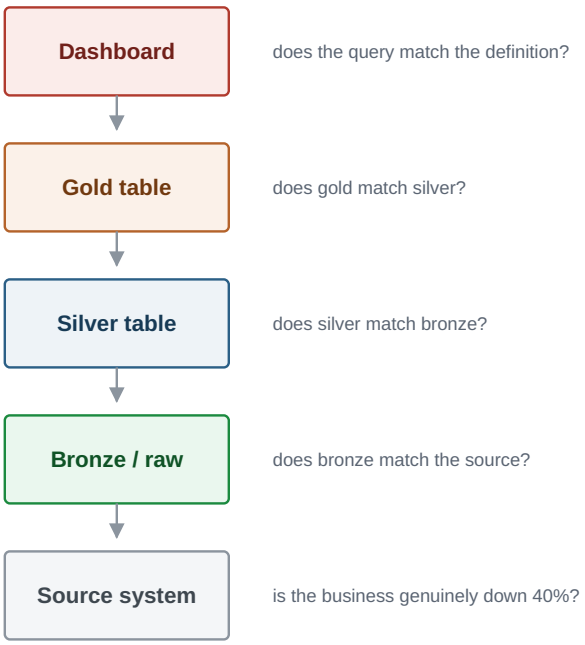


FIGURE 12.2 Work backwards through the layers, comparing each to the one before. The layer where they stop agreeing is where the bug is.

Run the questions in this order, because they are ordered by how often they are the answer.

Is the business actually down? A holiday, an outage, a failed marketing campaign. Check a completely independent signal — app installs, support ticket volume, the payment provider's own dashboard. Roughly a third of "the data is wrong" reports are the data being right.

Are the two people counting the same thing? The analyst includes cancelled orders; the pipeline excludes them. Different timezone. Different date column — order date against ship date. That is the most common technical answer by a wide margin, and it is a definition problem, not a code problem.

Is the data there at all? Row counts by day for the last fortnight, at each layer. A partition that is missing or half-loaded shows up immediately.

Did the volume drop or did the values change? Same row count with lower totals means a value problem — a currency, a units change, a transformation bug. Fewer rows means a filter or a load problem.

Bisect the layers. Compare gold to silver to bronze to source. Wherever agreement breaks is where you look.

What changed? Deployments, config changes, upstream releases, schema changes. In practice this is where the answer usually is, and it is why every pipeline should log the code version it ran.

> **WHAT TO WRITE DOWN WHILE YOU WORK**
>
> Timeline: when the data was last correct, when it was first noticed, what happened in between. Impact: which tables, which dashboards, which decisions. Root cause: what actually happened, not the first plausible story. Fix: what you did to the data, separately from what you did to the code. Prevention: the check that would have caught it in under an hour.
>
> The last one is the only line that has any lasting value, and it is the one people skip when they are relieved it is over.

12.5 Fixing the data, not only the code

Once you know the cause, three separate jobs follow, and beginners conflate them.

Fix the code, so it stops happening.

Fix the history, by backfilling the affected partitions with the corrected code. Chapter 8's partition-replace pattern makes this safe.

Tell people. Anyone who made a decision on the wrong number needs to know it changed. A silent correction is worse than the original error, because it destroys trust in every number you have ever produced.

12.6 SLAs and being on call

An **SLA** is a promise to consumers: *the orders table is complete for day D by 06:00, reconciled to within 0.1%.* An **SLO** is the internal target you run against, usually stricter.

Two useful disciplines follow. Alert on **the SLA being at risk**, not on the job having already failed — a job that normally finishes at 05:00 and is still running at 05:40 should page somebody before six. And route alerts to a human who can act; an alert channel nobody reads is a compliance artefact, not a control.

12.7 "Production" is a practice, not a place

This book is called *From Zero to Production*, and so far production has meant reliable data. It also means a set of engineering habits, and a pipeline can have perfect data checks and still not be production-grade. Here is the minimum, and none of it is optional in a real team.

Version control, with review. The pipeline lives in Git. Changes arrive as pull requests, not as edits on a server. Someone else reads them. This is not bureaucracy: the most common cause of a broken pipeline is a change, and review is the cheapest place to catch one.

Two kinds of test, and they are not the same.

Code tests check your logic with fixed inputs. A transformation function given three known rows returns three known rows. These run in seconds on every pull request and catch the bug before it meets real data.

```
def test_parse_amount_rejects_text():
    assert parse_amount("N/A", default=None) is None
    assert parse_amount("4749.00") == 4749.00
```

Data tests check real data at runtime — the six checks from section 12.1. Both are necessary. Code tests would not have caught the source sending 40% volume; data tests would not have caught a rounding bug that only appears on negative amounts.

Environments. At minimum development and production, with separate storage locations and separate credentials. The rule that prevents most accidents: a development job must not be able to write to production storage, enforced by permissions rather than by care.

CI/CD. On every pull request, run the code tests, lint, and check the pipeline definition parses. On merge, deploy. The value is not automation, it is that the same steps run every time regardless of who is deploying and how late it is.

Configuration and secrets. Configuration in files that vary by environment; secrets in a secret manager or the orchestrator’s connection store, never in the repository. Chapter 2 and Chapter 7 covered the mechanism; the practice is that this is checked in review.

Rollback. Two separate questions, and beginners conflate them. Rolling back *code* is a deploy of the previous version. Rolling back *data* means reprocessing affected partitions with the corrected code — which is only possible because bronze was kept and the load is idempotent. Chapters 6 and 8 were quietly building this capability.

Documentation that is actually read. Not a wiki nobody opens. The grain at the top of the model file, the owner and SLA in the data contract, and a README that says what the pipeline survives.

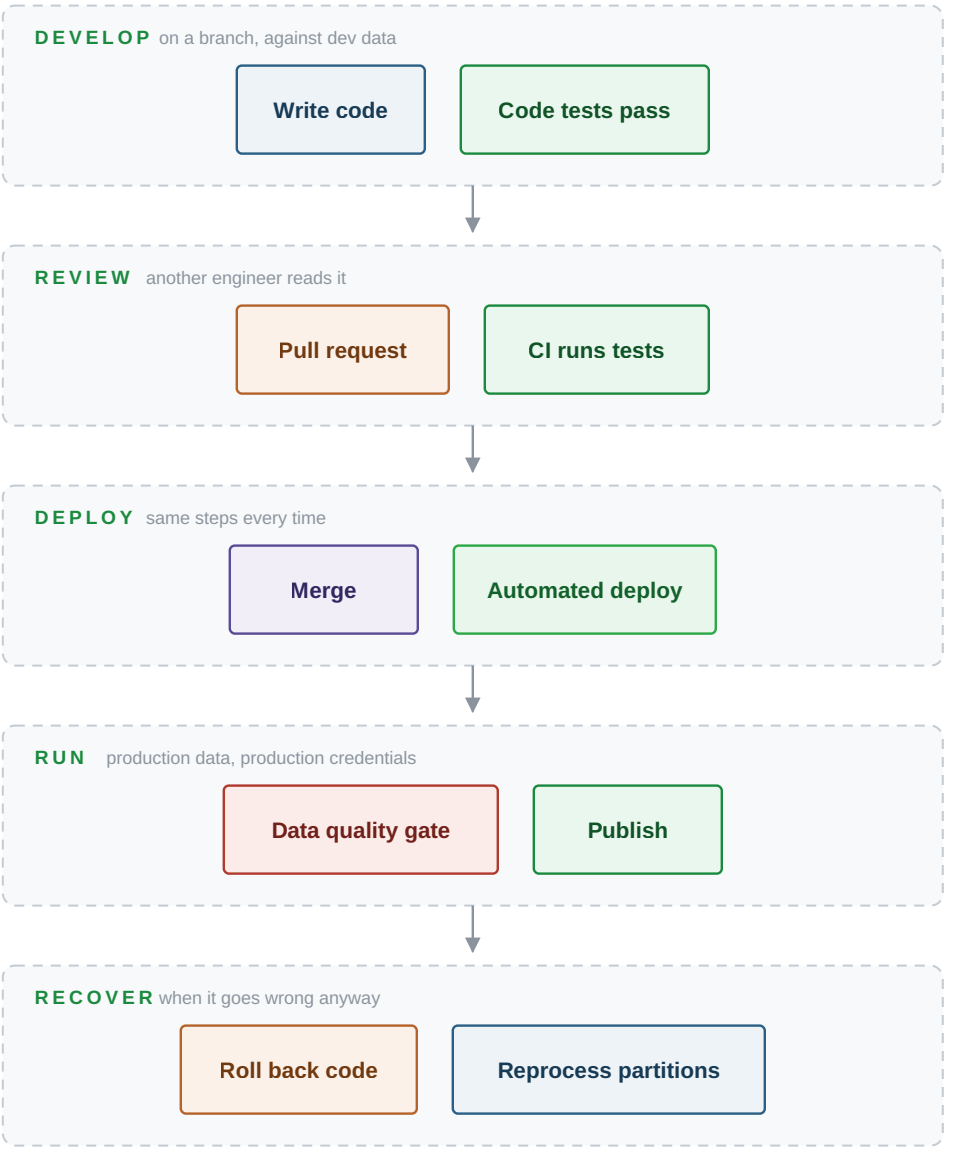


FIGURE 12.3 The path a change takes in a production team. Every gate exists because something got through once.

THE HONEST BAR FOR A BEGINNER

You are not expected to arrive knowing CI/CD. You are expected to know *why each of these exists* and to be able to say which ones your project has and which it does not. A candidate who says "my project is in Git with tests on the transformation functions, but I have no CI and I deploy by hand" is more credible than one who claims a production-grade platform.

12.8 When the consumer is a model

Increasingly the thing reading your gold tables is not a dashboard but a machine learning model, and that changes what "correct" means in three specific ways. This is not a chapter on machine learning — it is the part of ML that lands on a data engineer's desk.

A feature pipeline is a data pipeline with a stricter contract. A *feature* is an input to a model — a customer's order count in the last thirty days, their average basket value. It is computed by exactly the code you have been writing. What differs is that a silent change in a feature does not produce a visibly wrong dashboard; it produces a model that quietly gets worse.

Training/serving skew is the failure that defines this work. The model is trained on features computed one way, in batch, over history. At serving time the same feature is computed a second way, often in different code, sometimes in a different language. When the two definitions drift apart, the model receives inputs unlike anything it was trained on and its accuracy degrades with no error anywhere.

That is Chapter 12's whole theme wearing different clothes: the pipeline succeeded, and the numbers were wrong. The standard defences are the ones you already know — compute the feature once and serve both paths from the same definition, and reconcile the training and serving values as a data quality check.

Point-in-time correctness. When building a training set you must use the feature value *as it was* at the moment of the event, not as it is now. Joining today's customer tier onto a purchase from March leaks information the model would not have had, the model scores brilliantly in testing and fails in production. If that sounds familiar, it is Chapter 5's SCD Type 2 problem exactly — and it is why the ₹4,749 example matters beyond reporting.

Vector and embedding data, used in retrieval and RAG systems, is stored in a vector database or an index rather than a table, but everything upstream is unchanged: the same ingestion, the same chunking and cleaning, the same freshness and completeness checks, the same question of what happens when the source changes and the index does not get rebuilt.

WHAT THIS MEANS FOR YOU

You do not need to train models. You need to recognise that when a model is the consumer, the checks in 12.1 still apply and two more are added: *does the training definition match the serving definition*, and *was this feature value knowable at that point in time*. Both are data engineering problems, and both are asked about in interviews at companies doing ML.

COMMON BEGINNER MISTAKES WITH RELIABILITY

- Monitoring only whether the job ran.
- Alerting on everything, so nobody reads the alerts.
- Checks that run after publishing rather than before.
- Fixing the code and leaving the wrong history in place.
- Correcting numbers silently.
- No owner on a dataset, so an incident has nobody to route to.
- Writing an incident note with no prevention line.

Summary

- A crashing pipeline is easy. A succeeding pipeline with wrong data is the real failure mode.
- Monitor freshness, volume, nulls, uniqueness, referential integrity and reconciliation — before publishing.
- Choose fail, warn or quarantine per check, deliberately, and write the choice down.
- Data contracts make the producer's promise explicit and give the consumer something to point at.
- Debug by bisecting layers, and check the definition mismatch early because it is usually the answer.
- Fix the code, fix the history, and tell people. All three.

- Alert on the SLA being at risk rather than on failure that has already happened.

In the interview

Yesterday's dashboard shows revenue 40% below normal. You are the data engineer. What do you investigate?

SHORT ANSWER

I would establish whether the business is actually down before assuming a data problem, using an independent signal. Then I would check whether the analyst's definition matches the pipeline's — cancelled orders, timezone, which date column — because that is the most common cause. Then row counts by day at each layer to find where the data stops matching, and finally what changed: deployments, upstream schema, config.

DEEPER ANSWER

The distinction I would want early is whether row counts dropped or values dropped. The same number of rows with lower totals is a value problem — a units or currency change, or a transformation bug. Fewer rows is a load or filter problem. That halves the search immediately.

Then I bisect: gold against silver, silver against bronze, bronze against source. Whichever comparison breaks is the layer to look at, and that is a large part of why the medallion structure exists.

FOLLOW-UP: IT TURNS OUT A PARTNER SENT THEIR FILE TWICE. WHAT NOW?

Three things, in order. Stop the bleeding by reprocessing the affected partition with an idempotent load so the duplicates are replaced rather than added to. Then backfill any downstream tables built from it. Then tell everyone who saw the wrong number, because a silent correction costs more trust than the original error.

And then the prevention, which is the part that matters: a uniqueness check on the file's batch identifier, and a volume check that would have failed the run when row counts doubled.

SENIOR-LEVEL ANSWER

I would push back gently on the framing. The real question is not how I debug it but why it reached a dashboard at all. Doubling row counts should have failed a volume check before publication, and the incident is evidence that the table had no contract and no pre-publication gate. The output of this incident should be a check, an owner and an SLA — not just a corrected number.

PRACTICE

1. Take any table you have built and write its six checks as SQL. Decide fail, warn or quarantine for each.
2. Write the data contract for it, including grain, owner and freshness.
3. Deliberately double-load a day of data. Confirm your checks catch it. If they do not, you have found a gap.
4. Simulate a source dropping 40% of its volume and confirm the volume check fires.
5. Write an incident note for a failure you have actually seen, including the prevention line.

SOURCES AND VERIFICATION

The four failures in the opening table are the measured results from Chapters 3, 4, 5 and 7 of this book, each reproduced from a real execution rather than described. The check queries and the contract format are worked patterns.

Reference material: the data quality dimensions used here — freshness, volume, schema, nulls, uniqueness, referential integrity, reconciliation — follow the categories common to open-source data quality tooling such as Great Expectations, dbt tests and Soda.

PART SIX

Getting Hired

One project you built and broke on purpose, and the ability to reason out loud under mild pressure. Everything in the previous twelve chapters converges here.

13 · The Portfolio Project

14 · The Interview Loop

13

CHAPTER THIRTEEN

The Portfolio Project

THIS CHAPTER BUILDS ONE PROJECT WITH YOU, from an empty folder to a working pipeline you can defend in an interview. Nothing is assumed. Every command is shown, every output printed here is what actually appeared, and each stage is drawn before it is coded.

> VERSIONS AND DIALECTS
>
> The project runs on **Python 3** with `requests` and `duckdb`. The DAG targets **Airflow 2.4 and later**.

Almost every beginner portfolio has the same three faults: it reads a clean file that never changes, it runs once, and it has no failure handling. Interviewers have seen a thousand of them and they establish nothing, because none of the difficulty of this profession is anywhere in them.

The project below is deliberately awkward. That is the point, and it is what makes it worth talking about.

13.1 The whole system, before any code

Read this first. Everything after it is one of these five boxes.

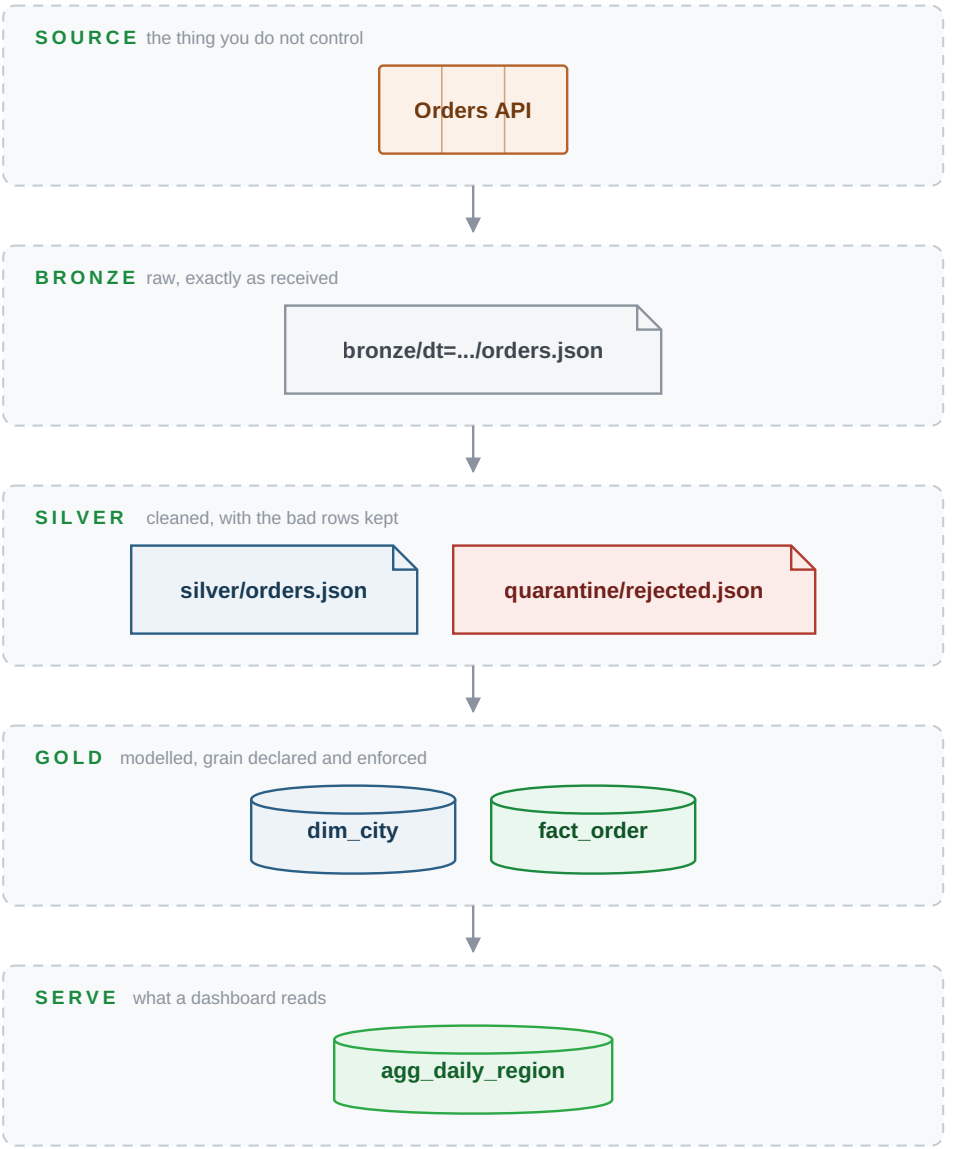


FIGURE 13.1 The complete system. Five zones, each a separate script, each runnable on its own. Nothing here needs a cloud account.

What you will have built, and what you will be asked about it

Before the code, the two things that actually matter. This table is the contract this chapter makes with you.

Stage	What you will be able to claim	What an interviewer will ask
1 · Ingest	"It survives a source that rate-limits and fails intermittently"	Which status codes do you retry, and which do you never retry?
2 · Clean	"Bad rows are quarantined with a reason, not dropped"	What do you do when 50% of rows are bad instead of 2%?
3 · Model	"One row means one order, enforced by a primary key"	What is the grain, and what happens if it is violated?
4 · Validate	"It refuses to publish when the data looks wrong"	Give me a case where the job succeeds and the data is still wrong
5 · Orchestrate	"Any day can be re-run safely and produces the same result"	What happens if this runs twice? What if it fails halfway?

Read the right-hand column again. Not one of those questions is about a tool. They are all about **behaviour under failure**, which is what this project is designed to demonstrate and what a tutorial project cannot.

Five stages, five scripts, run in order:

Stage	Script	What it does	Chapters used
1	`ingest.py`	API → bronze JSON	2, 4
2	`transform.py`	bronze → silver → gold, then validate	3, 4, 5, 8, 12
3	`publish.py`	gold → aggregate table	3, 5
4	`dag.py`	schedules the three above	10
5	break tests	proves what it survives	8, 12

WHY THIS SHAPE AND NOT SOMETHING BIGGER

You could build this on cloud infrastructure with Spark and Kafka. Do not, at least not first. A reviewer cannot run it, you will stop paying for it, and none of that scale demonstrates anything a recruiter can check in five minutes. *438 rows that exercise every failure mode beat forty million clean ones.* Chapter 14 explains why interviewers react to the second version and ignore the first.

13.2 Setting up, from an empty folder

Three things to install and one folder to make. Nothing else.

```
mkdir orders-pipeline && cd orders-pipeline
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install requests duckdb
```

You will end up with this. It is worth looking at now, because the shape of the folders *is* the architecture:

```
orders-pipeline/
├── mock_api.py                  the awkward source (Field Kit)
├── ingest.py                    stage 1
├── transform.py                 stage 2
├── publish.py                   stage 3
├── bronze/
│   └── dt=2026-03-17/
│       └── orders.json          raw, never edited
├── silver/
│   └── dt=2026-03-17/
│       └── orders.json          cleaned
├── quarantine/
│   └── dt=2026-03-17/
│       └── rejected.json        bad rows, with reasons
└── warehouse.db                 gold tables
```

Notice `dt=2026-03-17` appearing three times. That is Chapter 6's partitioning and Chapter 8's idempotency in a folder name: every stage writes into a directory named for the day it is processing, so re-running a day replaces that day and touches nothing else.

The source

`mock_api.py` is a small local API from the Field Kit that behaves like a real one, on purpose:

- nine pages of fifty records, so you must paginate
- HTTP 429 and 503 returned at intervals, so you must retry
- some rows have a text value where a number belongs
- some rows are missing a field entirely
- one order id appears twice

Start it in one terminal and leave it running:

```
python3 mock_api.py &
```

You can point the project at a real public API later. Start here, because a source that misbehaves *on demand* teaches far more than one that misbehaves at random.

13.3 Stage one — ingest

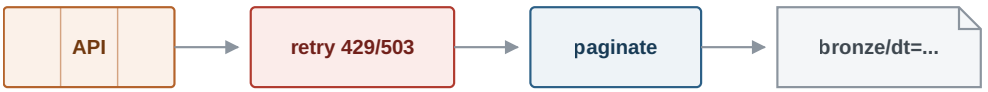


FIGURE 13.2 Stage one. Retry what might succeed, stop on what will not, and write the result untouched.

What this stage must survive: the source failing intermittently, and the source having more data than one response can hold.

```
"""Stage 1 — pull a paginated, rate-limited API into the bronze zone."""
import json, logging, sys, time
from pathlib import Path
import requests

logging.basicConfig(level=logging.INFO,
    format="%(asctime)s %(levelname)-7s %(message)s", datefmt="%H:%M:%S")
log = logging.getLogger("ingest")

BASE = "http://127.0.0.1:8777/orders"
RETRYABLE = {429, 500, 502, 503, 504}

def fetch_page(session, page, attempt=1):
    r = session.get(BASE, params={"page": page}, timeout=(5, 30))
    if r.status_code in RETRYABLE:
        if attempt > 5:
            raise RuntimeError(f"page {page} failed after {attempt}")
        wait = min(2 ** attempt, 30) * 0.05
        log.warning("page %d got %d, retry %d in %.2fs",
                    page, r.status_code, attempt, wait)
        time.sleep(wait)
        return fetch_page(session, page, attempt + 1)
    r.raise_for_status()          # 4xx stops here, deliberately
    return r.json()

def main(run_date):
    out = Path(f"bronze/dt={run_date}")
    out.mkdir(parents=True, exist_ok=True)
    session, page, rows = requests.Session(), 1, []
    while True:
        payload = fetch_page(session, page)
        rows.extend(payload["data"])
        # stop on the API's own count, not on an empty page
        if page >= payload["total_pages"]:
```

```
            break
        page += 1
    (out / "orders.json").write_text(json.dumps(rows))
    log.info("fetched %d pages, wrote %d rows",
             payload["total_pages"], len(rows))
    return 0

if __name__ == "__main__":
    sys.exit(main(sys.argv[1]))
```

Run it:

```
python3 ingest.py 2026-03-17
```

```
23:18:51 WARNING page 6 got 429, retry 1 in 0.10s
23:18:51 INFO    fetched 9 pages, wrote 438 rows
```

What you should see. A warning line for each failure the source threw at you, then 438 rows. Run it again and the failures will land on different pages, because the source misbehaves at intervals — that is realistic and it is why the retry loop exists rather than a fixed sleep.

Four things in that code are there for a reason, and all four are missing from most tutorials.

The timeout is a **pair**: five seconds to open the connection, thirty to receive the body. A single number leaves one of them unbounded, and an unbounded request is how a nightly job is still running at nine in the morning.

Only **retryable codes are retried**. A 400 means your request is wrong and will be wrong the next five times too.

The wait **doubles** each attempt. Hammering a server that is already overloaded is how a small problem becomes an outage.

The loop stops on the API's **own page count**, not on an empty page. Pipelines that stop on an empty page silently truncate the day an API returns a blank page in the middle.

WHY BRONZE IS NEVER CLEANED

Nothing is fixed at this stage. The malformed rows are written exactly as they arrived. When you find a bug in your cleaning logic next month, this file is what lets you fix it — the API will not still have that day's data. Chapter 6, section 6.10.

13.4 Stage two, part one — clean and quarantine

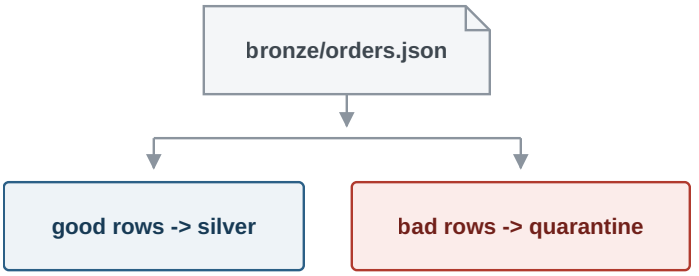


FIGURE 13.3 Stage two splits the input rather than dropping anything. Every rejected row keeps its reason.

What this stage must survive: rows that are wrong, without losing them and without failing the whole run.

```
good, quarantine, seen = [], [], set()

for r in raw:
    problem = None
    if r["order_id"] in seen:
        problem = "duplicate order_id"
    elif "city" not in r:
        problem = "missing city"
    else:
        try:
            r = dict(r, amount=float(r["amount"]))
        except (TypeError, ValueError):
            problem = f"bad amount {r['amount']!r}"

    if problem:
        quarantine.append(dict(r, _reason=problem))
    else:
        seen.add(r["order_id"])
        good.append(r)
```

```
23:18:53 INFO    bronze rows: 438
23:18:53 INFO    silver: 428 good, 10 quarantined
```

Ten rows rejected out of 438, and the quarantine file records why:

```
10 rows quarantined
   6  missing city
   4  bad amount
```

```
sample: {'order_id': 90000000, 'customer_id': 100000,
         'status': 'DELIVERED', 'order_ts': '2026-03-17T00:00:00'}
        -> missing city
```

Why quarantine rather than drop. Both approaches publish the same 428 rows. Only one can answer "what happened to those ten orders?" A pipeline that drops silently loses information nobody can ever recover; a pipeline that quarantines turns a mystery into a ten-second lookup. Chapter 12, section 12.2.

13.5 Stage two, part two — the model

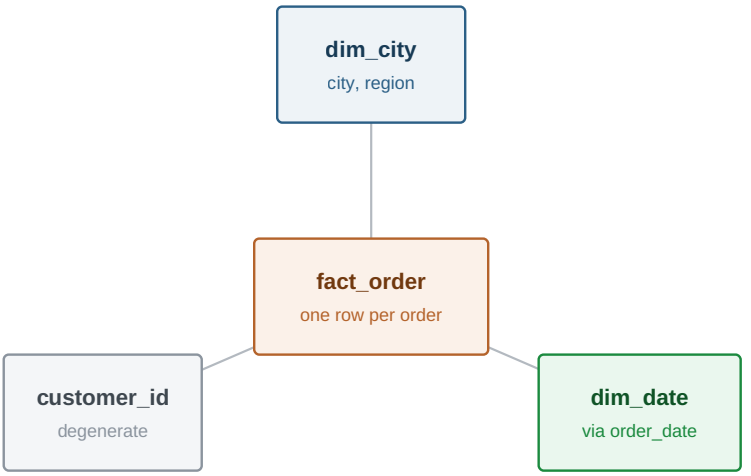


FIGURE 13.4 The gold model. A small star schema, but a real one: a declared grain, a surrogate key, and a dimension the fact joins to.

The grain is written down before anything else, and then enforced by the database rather than trusted to the code:

```
-- grain: one row per order
CREATE TABLE IF NOT EXISTS dim_city (
    city_key   INTEGER,          -- surrogate key (Chapter 5)
    city       VARCHAR,
    region     VARCHAR);

CREATE TABLE IF NOT EXISTS fact_order (
    order_id    INTEGER PRIMARY KEY,   -- the grain, enforced
    customer_id INTEGER,               -- degenerate dimension
```

```
    city_key    INTEGER,              -- joins to dim_city
    order_date  DATE,
    status      VARCHAR,
    amount      DECIMAL(10,2));
```

The dimension is small and readable, which is exactly what a dimension should be:

```
 city_key |   city     | region
----------+------------+--------
        1 | Bengaluru  | South
        2 | Delhi      | North
        3 | Hyderabad  | South
        4 | Kochi      | South
        5 | Mumbai     | West
        6 | Pune       | West
```

And the fact carries keys and numbers, nothing else:

```
 order_id | customer_id | city_key | order_date | status    | amount
----------+-------------+----------+------------+-----------+--------
 90000001 |      100001 |        1 | 2026-03-17 | DELIVERED | 236.00
 90000002 |      100002 |        3 | 2026-03-17 | DELIVERED | 273.00
 90000003 |      100003 |        4 | 2026-03-17 | CANCELLED | 310.00
 90000004 |      100004 |        2 | 2026-03-17 | RETURNED  | 347.00
```

The load itself is two statements, and the first one is the important one:

```
# idempotent: replace this date's partition rather than adding to it
con.execute("DELETE FROM fact_order WHERE order_date = ?", [run_date])
con.execute("""
    INSERT INTO fact_order
    SELECT s.order_id, s.customer_id, d.city_key,
           CAST(s.order_ts AS DATE), s.status, s.amount
    FROM stg s JOIN dim_city d ON d.city = s.city
""")
```

```
23:18:53 INFO    gold: fact_order now 428 rows
```

Delete-then-insert on the day being processed is Chapter 8's idempotency. It is one line, and it is the reason the first break test in section 13.7 passes.

13.6 Stage two, part three — the gate

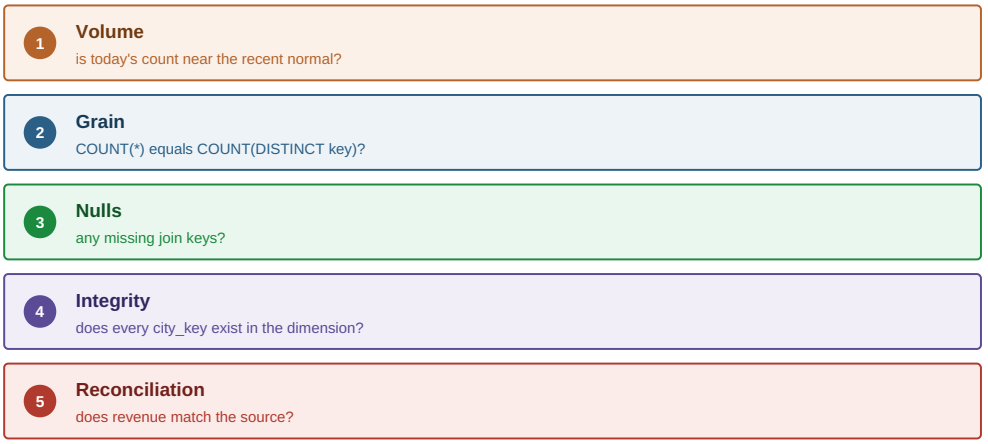


FIGURE 13.5 Five checks, run before anything downstream can see the table. A failure returns exit code 1 and the pipeline stops.

```
SELECT
  (SELECT COUNT(*) FROM fact_order
    WHERE order_date = :d)                              AS rows_today,
  (SELECT COUNT(*) - COUNT(DISTINCT order_id)
     FROM fact_order)                                   AS dupes,
  (SELECT COUNT(*) FROM fact_order
    WHERE customer_id IS NULL)                          AS null_keys,
  (SELECT COUNT(*) FROM fact_order f
     LEFT JOIN dim_city d ON d.city_key = f.city_key
    WHERE d.city_key IS NULL)                           AS orphans,
  (SELECT ROUND(SUM(amount),2) FROM fact_order
    WHERE order_date = :d)                              AS revenue;
```

```
  rows loaded           428
  duplicate keys        0
  null join keys        0
  orphan city keys      0
  revenue               980,694.00
  verdict               PASS

exit=0
```

The exit code is the whole point. Lecture Zero, section 0.7: the scheduler reads it. A failing verdict returns 1, the orchestrator marks the task failed, and nothing downstream runs on a bad table. A script that prints “WARNING” and exits zero has told the scheduler everything is fine.

13.7 Stage three — publish

Only reached if the gate passed. This is the table a dashboard would actually query.

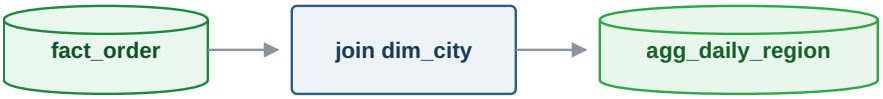


FIGURE 13.6 The serving layer. One small aggregate at a declared grain, replaced daily.

```
-- grain: one row per region per day
INSERT INTO agg_daily_region
SELECT f.order_date, d.region,
       COUNT(*)                       AS orders,
       COUNT(DISTINCT f.customer_id) AS customers,
       ROUND(SUM(f.amount), 2)        AS revenue
FROM fact_order f
JOIN dim_city d ON d.city_key = f.city_key
WHERE f.order_date = :d AND f.status = 'DELIVERED'
GROUP BY f.order_date, d.region;
```

```
23:18:55 INFO    published 3 region rows for 2026-03-17

  region      orders  customers       revenue
  South          126         45    300,934.00
  West            88         30    188,948.00
  North           44         15     98,692.00
```

Two decisions here are worth defending out loud. `COUNT(DISTINCT customer_id)` rather than `COUNT(*)`, because orders and customers are different questions and mixing them is how “we have 41,000 customers” becomes a number nobody can reproduce. And the `status` filter lives in the aggregate rather than in the fact, so the fact keeps cancelled and returned orders and a future question about them is still answerable.

13.8 Stage four — break it on purpose

Almost nobody writes this section, and it is the one that gets you hired. Four deliberate failures, and what the pipeline actually did.

Break 1 — run the whole thing twice

```
23:18:55 INFO    gold: fact_order now 428 rows
  rows loaded           428
  revenue               980,694.00
  verdict               PASS
```

Identical. 428 rows and ₹980,694 on both runs.

Compare Chapter 8, where an appending load turned ₹13,698 into ₹41,094 over three runs. The difference is one `DELETE` before the `INSERT`. **This is the single most valuable thing in the project**, because "safe to re-run" is what makes a pipeline operable, and most beginner projects cannot claim it.

Break 2 — feed it malformed rows

Already handled in stage two: six rows missing a field, four with `"N/A"` where a number belongs. Ten quarantined with reasons, 428 published, exit code 0. The pipeline **degraded rather than failed**, which is correct at this rejection rate.

The follow-up an interviewer will ask: *what if half the rows were bad?* Then continuing is wrong, and the rejection rate should itself be a check. That threshold belongs in configuration, and section 13.10 puts it in the README as a known gap.

Break 3 — remove the deduplication guard

```
ConstraintException: Constraint Error: Duplicate key "order_id: 90000001"
                     violates primary key constraint
table unchanged: True
```

The primary key caught what the code would have let through, and the transaction rolled back leaving the table untouched. **Two independent defences against the same failure** — the code checks, and the database enforces. That is a good answer to "how do you prevent duplicates?", because it does not depend on your code being perfect.

Break 4 — the source sends 40% of its volume

```
rows loaded 170 against expected ~428
volume check: FAIL_VOLUME  <- caught
```

This is the one that matters. The job did not crash. Every row loaded successfully. The table is valid, queryable and **wrong** — revenue understated by sixty per cent on a dashboard nobody would have questioned. Only the volume check saw it.

That paragraph, said out loud, is worth more than any tool on your résumé, because it demonstrates that you understand the failure mode this entire profession is organised around.

THE REFRAME

Most candidates describe what their project *does*. Describe what it *survives*. "It handles rate limits, quarantines bad rows with reasons, is safe to re-run, and refuses to publish when volume drops below seventy per cent of normal" is a sentence that separates you from everyone else in the queue.

13.9 Stage five — orchestrate it



FIGURE 13.7 The DAG. Three tasks, in order, each parameterised by the logical date so a re-run of any day is correct.

```
with DAG(
    dag_id="orders_daily",
    default_args={"owner": "you", "retries": 3,
                  "retry_delay": timedelta(minutes=5),
                  "retry_exponential_backoff": True},
    schedule="0 2 * * *",
    start_date=datetime(2026, 3, 1),
    catchup=False,                    # Chapter 10, section 10.5
    max_active_runs=1,
) as dag:

    ingest    = BashOperator(task_id="ingest",
                    bash_command="python3 ingest.py {{ ds }}")
    transform = BashOperator(task_id="transform",
                    bash_command="python3 transform.py {{ ds }}")
    publish   = BashOperator(task_id="publish",
                    bash_command="python3 publish.py {{ ds }}")

    ingest >> transform >> publish
```

`transform.py` exits non-zero on a failed check, so `publish` never runs on a bad table. That single behaviour is the whole reason the exit-code discipline exists, and it connects Lecture Zero to Chapter 12 in one line of a DAG file.

13.10 The README that does the work

Recruiters and interviewers read the README, not the code. Write it for them.

```
# Daily orders pipeline

## What it is
Paginated API -> bronze JSON -> cleaned silver -> star schema in gold,
orchestrated by Airflow, gated by data quality checks.

## Grain
fact_order        one row per order, enforced by a primary key
agg_daily_region  one row per region per day

## What it survives
- HTTP 429 and 503 from the source (exponential backoff, 5 attempts)
- Malformed rows (quarantined with a reason, not dropped)
- Duplicate order ids (checked in code, enforced by the primary key)
- Being re-run (partition delete-then-insert, verified identical)
- A source sending 40% volume (volume check blocks publishing)

## Measured
438 rows over 9 pages, retries absorbed on the flaky pages
428 published, 10 quarantined (6 missing city, 4 bad amount)
Re-run identical: 428 rows, revenue 980,694.00
40% volume simulation: FAIL_VOLUME, publish blocked

## Run it
python3 mock_api.py &
python3 ingest.py    2026-03-17
python3 transform.py 2026-03-17
python3 publish.py   2026-03-17

## Known gaps
The 70% volume threshold is hard-coded; it belongs in config.
No look-back window yet for late-arriving orders (Chapter 8, section 8.7).
Quarantine rate is logged but not itself checked.
```

The **what it survives** and **known gaps** sections are what an interviewer will ask about. Everything else is scaffolding. A candidate who lists their own project's weaknesses is demonstrating judgement, not admitting failure.

13.11 When it goes wrong

You will hit these. Each one is a real error with a real cause.

What you see	What it means	Fix
`ConnectionRefusedError`	The mock API is not running	Start it in another terminal
`KeyError: 'city'`	You read a field that some rows lack	Use `.get()`, or quarantine the row
`ValueError: could not convert string to float`	A text value in a number column	That is the point — quarantine it
`IntegrityError: duplicate key`	Your dedup guard is missing or wrong	Section 13.4
Row count grows on every run	Appending instead of replacing	Section 13.5
`FileNotFoundError: bronze/dt=...`	Stage two ran before stage one	Run in order; the DAG enforces this
Everything passes but revenue looks low	Check the `status` filter and the date	Section 13.7

13.12 Levelling it up

Once the five stages work, each of these is a weekend, and each maps onto a chapter and onto a line in a job advert.

Add	Replaces	Chapter	What it demonstrates
Write Parquet instead of JSON	The bronze and silver writes	6	Format and compression judgement
Move storage to S3 or ADLS	Local folders	7	Cloud object storage and IAM
Replace the transform with Spark	The Python cleaning loop	9	Distributed processing, plans
Rebuild gold with dbt	The gold SQL	8.9	Tests, lineage, materializations
Read from Kafka instead of the API	Stage one	11	Streaming, offsets, consumer groups
Add SCD Type 2 to `dim_city`	The dimension load	5	History and surrogate keys
Add a look-back window	The date filter	8.7	Late-arriving data

Do them **one at a time**, and keep the previous version working. A project with a git history showing “added Parquet, measured 6× smaller” and “added volume check after loading 40% of a day” is a project with a story. Chapter 14 is about telling it.

13.13 Two more projects, and what each one proves

One project done properly beats four done shallowly. But once the first is finished and defensible, a second gives you range, and a third gives you a story about growth. Here are the two worth building, in order.

Project 2 — the streaming path

What it proves: that you understand why streaming exists, and when it does not.

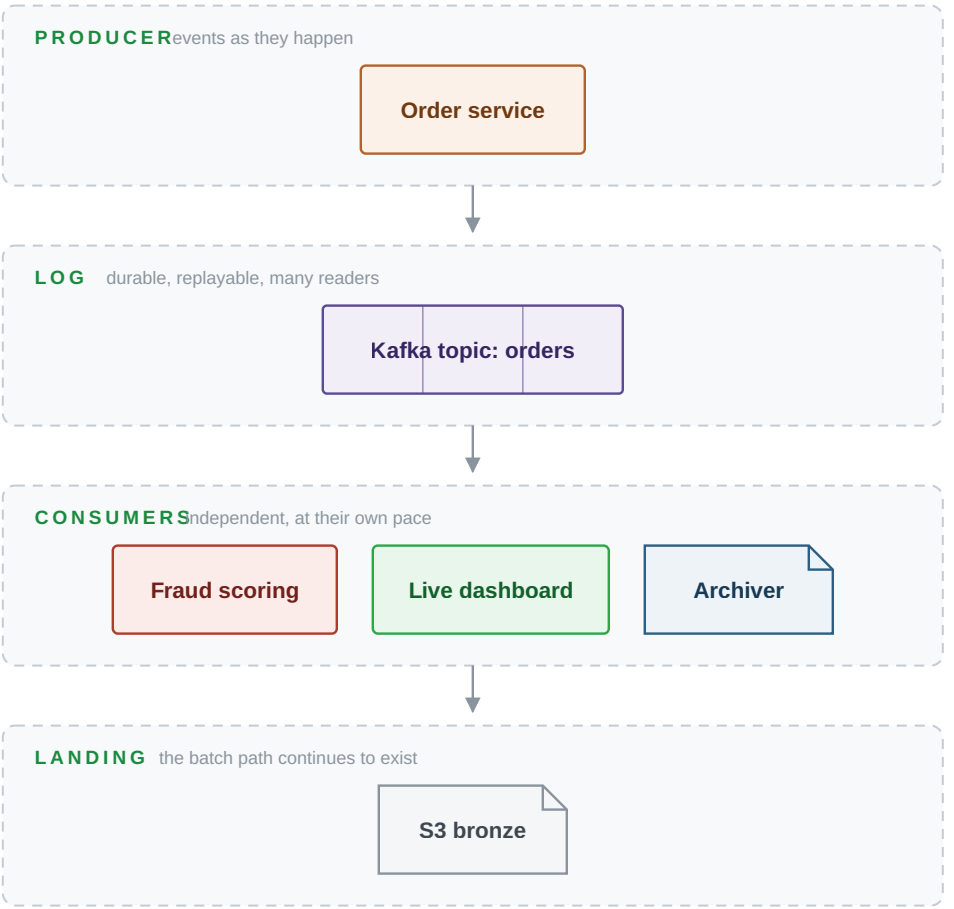


FIGURE 13.8 Project 2. The same orders, but the decision has moved from tomorrow morning to two seconds from now.

Scope, honestly: one topic with three partitions, a producer, two consumer groups, and a dead-letter queue. Docker Compose, one afternoon of setup, a weekend of building.

What you must be able to demonstrate, because these are the questions:

- Produce with and without a key, and show where ordering holds and where it does not.
- Start a third consumer in a group of two and watch the rebalance.
- Feed it a poison message, watch lag stop moving, then add the dead-letter queue and watch it recover.
- Set retention to five minutes, stop a consumer for ten, and confirm the data is gone.

That last one is the exercise nobody does, and it converts “retention is a setting” into “retention is a deadline” permanently. Chapter 11.

The interviewer’s question: *why is this streaming and not batch?* The honest answer for a portfolio project is usually “the fraud path needs seconds, the reporting path does not, so both exist and share one topic.” Saying that shows judgement. Saying “streaming is more real-time” does not.

Project 3 — the warehouse path

What it proves: that you can model, test and document data the way an analytics team actually works.

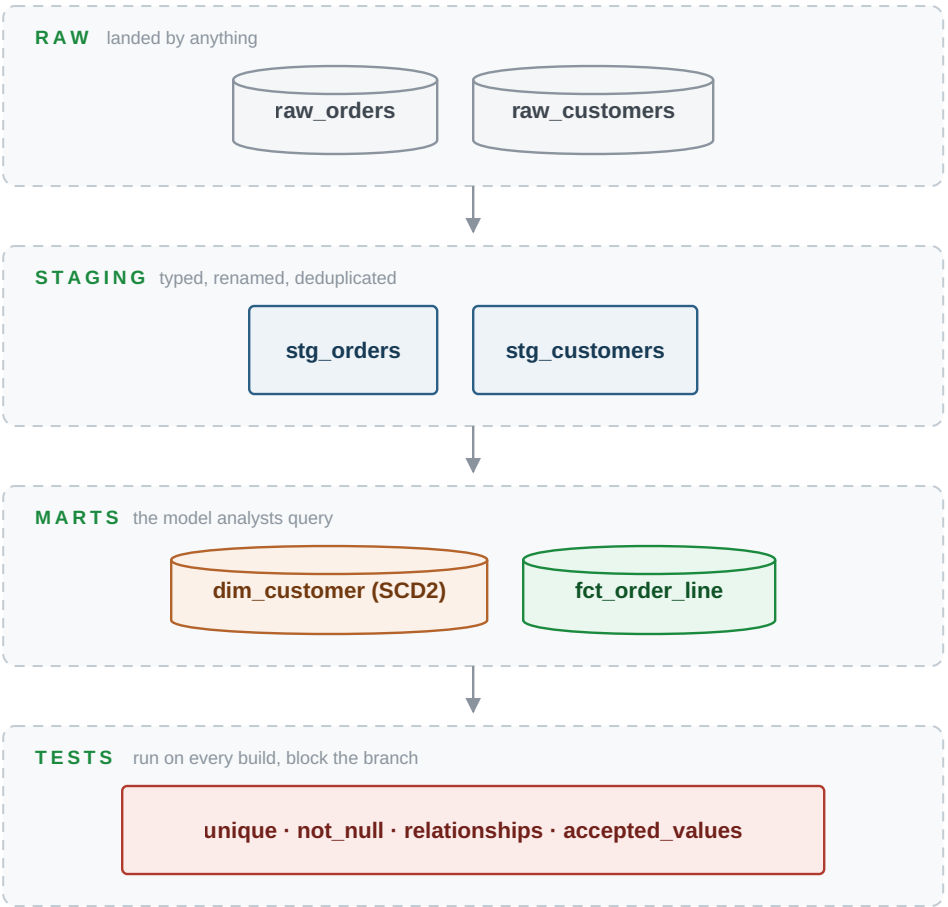


FIGURE 13.9 Project 3. Loading is trivial; the value is in the modelling layer and its tests.

Scope: DuckDB or a free warehouse tier, dbt Core, six to ten models, and tests on every one. Chapter 8, section 8.9.

What you must be able to demonstrate:

- The DAG dbt derives from your `ref()` calls, as a picture.
- A test failing and blocking everything downstream of it.
- An incremental model with a `unique_key`, run twice, producing identical rows.
- SCD Type 2 on the customer dimension, with the two revenue numbers from Chapter 5 reproduced on your own data.

The interviewer's question: *what is the grain of your fact table, and how do you know it holds?* You point at the `unique` test on the key. That is a better answer than most working engineers give.

Which to build, and when

	Project 1 · Batch pipeline	Project 2 · Streaming	Project 3 · Warehouse
Build it	First, always	After 1	After 1
Time	One week of evenings	One weekend	One weekend
Proves	Reliability and failure handling	Event thinking, ordering, lag	Modelling, testing, lineage
Best for	Every DE role	Roles naming Kafka or Flink	Roles naming dbt or analytics engineering
Skip it if	Never	The adverts you target never mention streaming	You already show SCD2 in project 1

HOW TO TALK ABOUT THREE PROJECTS WITHOUT RAMBLING

Lead with one. "I built a daily pipeline that survives a flaky source, quarantines bad rows and refuses to publish when volume drops — happy to walk through it." Mention the others only if asked for range, in one sentence each. A candidate who describes three projects unprompted sounds like they have finished none of them properly.

COMMON MISTAKES WITH PORTFOLIO PROJECTS

- A clean file that never changes, so nothing in the pipeline is ever tested.
- A notebook rather than runnable scripts with exit codes.
- A README that lists technologies instead of describing behaviour.
- Never running it twice, so idempotency is untested and unclaimable.
- Cloud infrastructure nobody can reproduce, on an account you will stop paying for.
- Big data for its own sake.
- Four shallow projects instead of one you can defend for twenty minutes.
- No known-gaps section, which reads as either perfect or unexamined. Interviewers assume the second.

Summary

- Draw the system before writing code. Five stages, five scripts, each runnable alone.
- Make the source awkward. Pagination, rate limits, malformed rows and duplicates are the point.
- Keep bronze unchanged so you can reprocess when you find a bug.
- Quarantine bad rows with a reason rather than dropping them silently.
- Declare the grain and enforce it with a primary key, so code and database defend it independently.
- Replace the partition rather than appending, then prove it by running twice.
- Validate before publishing and exit non-zero on failure, so the scheduler can act.
- Break it four ways and record what happened. That section is the product.
- Write the known gaps down. It reads as judgement, not weakness.

PRACTICE

1. Build all five stages from an empty folder. One week of evenings, and do not extend the deadline.
2. Reproduce all four break tests and record the actual output, as this chapter does.

3. Add a fifth break: kill the process halfway through the load and confirm the table is not left half-written.
4. Make the volume threshold configurable, then add a check on the quarantine rate itself.
5. Add a look-back window for late orders and prove it recovers a row the watermark would have missed.
6. Pick one row from the levelling-up table and do it. Commit before and after, so the change is visible in your history.
7. Write the README, give it to someone non-technical, and ask them what the project does. Rewrite until they can tell you.

SOURCES AND VERIFICATION

Every stage of this project was executed while the chapter was written, against a local mock API built to return HTTP 429 and 503 at intervals and to serve malformed and duplicated records. All output blocks are actual results: the retry log, 438 rows fetched over nine pages, 428 published against 10 quarantined, the dimension and fact table contents, the published regional aggregate, the identical re-run, the primary key constraint error, and the volume check failing at 170 rows.

The API server, all five stages and the break-test scripts are in the companion Field Kit, so the entire project can be reproduced offline with only `requests` and `duckdb` installed.

14

CHAPTER FOURTEEN

The Interview Loop

You have the project. What remains is the part that is a speaking exam rather than an engineering one: reasoning out loud, under mild pressure, in front of someone deciding whether to hire you.

14.1 Designing a system in an interview

System design rounds are not about the right answer. They are about whether you ask before you build.



FIGURE 14.1 The order to work through. Candidates who start at step five and name tools do badly; candidates who spend four minutes on step one do well.

Clarify first. Who consumes this? How fresh must it be? What is the cost of being wrong? Is history required? These four questions determine the entire architecture, and asking them is most of the signal an interviewer is looking for.

Estimate out loud. Three million orders a day at roughly 2 KB each is about 6 GB. Clickstream at four hundred events per order is a different order of magnitude entirely. You do not need precision; you need to show that you size things before choosing tools.

Choose the shape from the freshness requirement, using Chapter 1's rule: what decision changes if this is five hours old instead of five minutes?

Say the trade-offs. "I would use micro-batch rather than streaming here, because the SLA is hourly and streaming would add permanent on-call cost for freshness nobody uses." That sentence is worth more than any diagram.

A worked example, briefly

Design analytics for a food delivery company.

Clarify: two consumers — operations needs live order status within seconds, finance needs daily reconciled revenue. That is two different systems, and saying so early is the main insight.

Estimate: three million orders a day, twenty events each, so sixty million events. Small for streaming, far too large for a single machine.

Shape: streaming path for operations, batch path for finance. Same Kafka topic feeds both.

Layers: app events to Kafka; Flink or Spark Structured Streaming to a serving store for live status; the same topic archived to Parquet in object storage for the batch path; Spark builds bronze, silver and gold; a star schema at order-line grain in the warehouse.

Failure: at-least-once with idempotent consumers, a dead-letter queue, a three-day look-back for late events, reconciliation of daily revenue against the payment provider before publishing.

Trade-offs: the live path is eventually consistent and finance's numbers are the authority; the two will disagree intraday, and that must be documented rather than discovered.

14.2 Three designs, worked

The framework is only useful once you have watched it run. Here are three problems that cover most of what gets asked, worked at the level an interviewer expects — not exhaustively, but enough that you can see the shape of a good answer.

One note before we start. These are interview prompts about unnamed companies, deliberately set at a larger scale than ShopKart's 1.2 million orders a day from Chapter 1. That is on purpose: interviewers pick round numbers, and part of what is being tested is whether you re-derive the sizing from the number you were given rather than reaching for one you already know.

Design 1 — Analytics for an e-commerce company

"Design the data platform for a company doing three million orders a day."

Clarify first. Who consumes it, and how fresh must it be? Assume two answers: finance needs daily revenue reconciled and correct; the merchandising team wants yesterday's category performance by 9 a.m. Neither needs seconds. **That single answer removes streaming from the design**, and saying so early is most of the signal.

Estimate out loud. Three million orders at roughly 2 KB is about 6 GB a day. Clickstream at a few hundred events per order is two orders of magnitude larger — call it low terabytes. So the orders pipeline is small and the behavioural pipeline is the one that drives infrastructure choices. Sizing the two separately is the point.

Shape. Batch, nightly, with the orders path and the clickstream path sized differently.

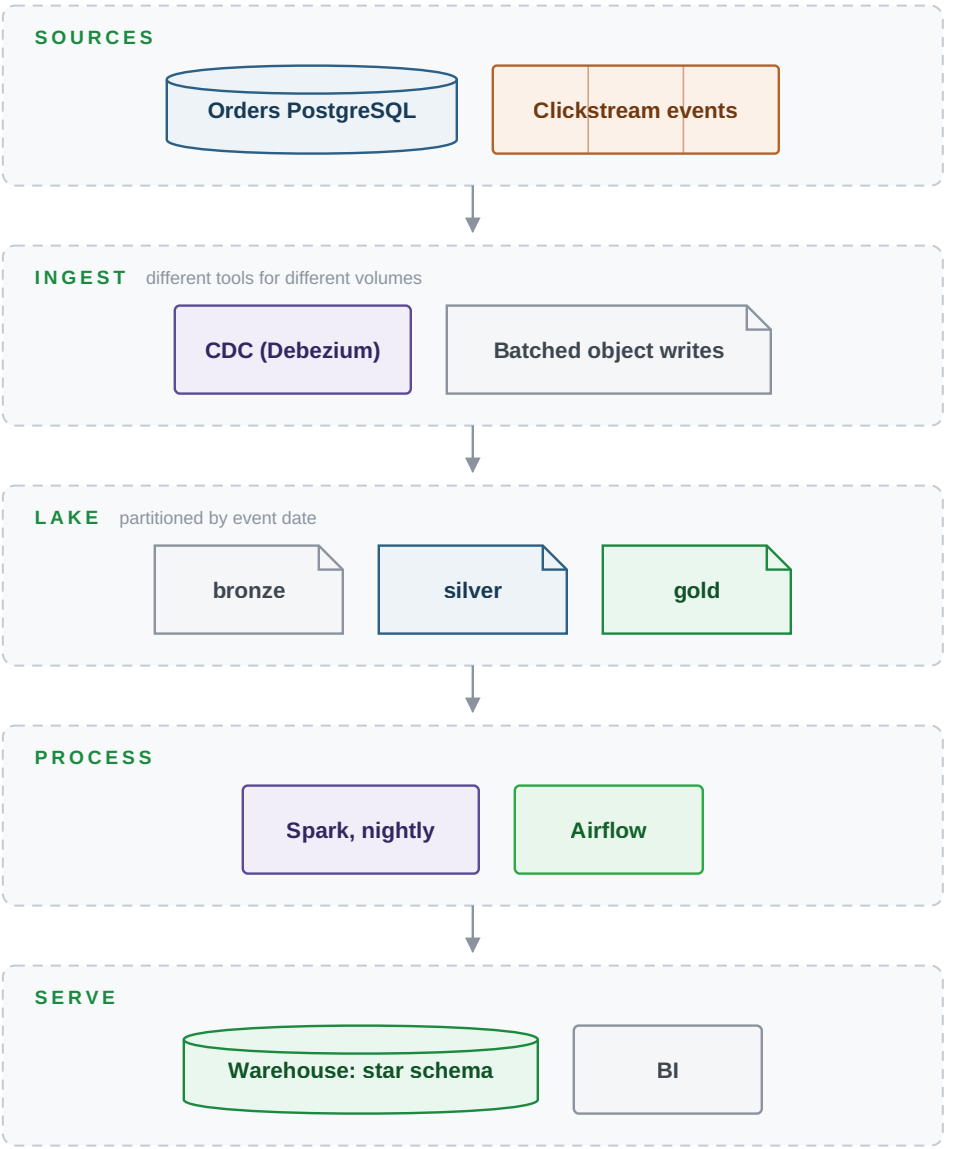


FIGURE 14.2 Design 1. The orders path is small and must be exactly right; the clickstream path is large and can tolerate approximation. Treating them identically is the common mistake.

Model. Fact at order-line grain, conformed dimensions for customer, product, date and seller, SCD Type 2 on customer attributes that affect reporting. Chapter 5.

Failure handling. Idempotent partition replacement, a three-day look-back for late orders, reconciliation of daily revenue against the source before publish.

Trade-offs, said out loud. “I would not stream this. The SLA is hours, and streaming would add permanent on-call cost for freshness nobody uses. If fraud detection is added later, that is a second consumer of the same event stream, not a redesign of this one.”

Design 2 — Real-time fraud detection

“Score every order for fraud within one second.”

The interesting part is that this is **two systems**, and candidates who build one fail the round.

Clarify. What is the decision, and what is the cost of being wrong? Blocking a legitimate order costs a customer; allowing a fraudulent one costs money. That asymmetry decides the threshold, and asking about it is a strong move.

Estimate. Three million orders a day is about 35 a second average, with festival peaks perhaps ten times that. **That is a small streaming problem** — say so, because candidates routinely over-engineer here.

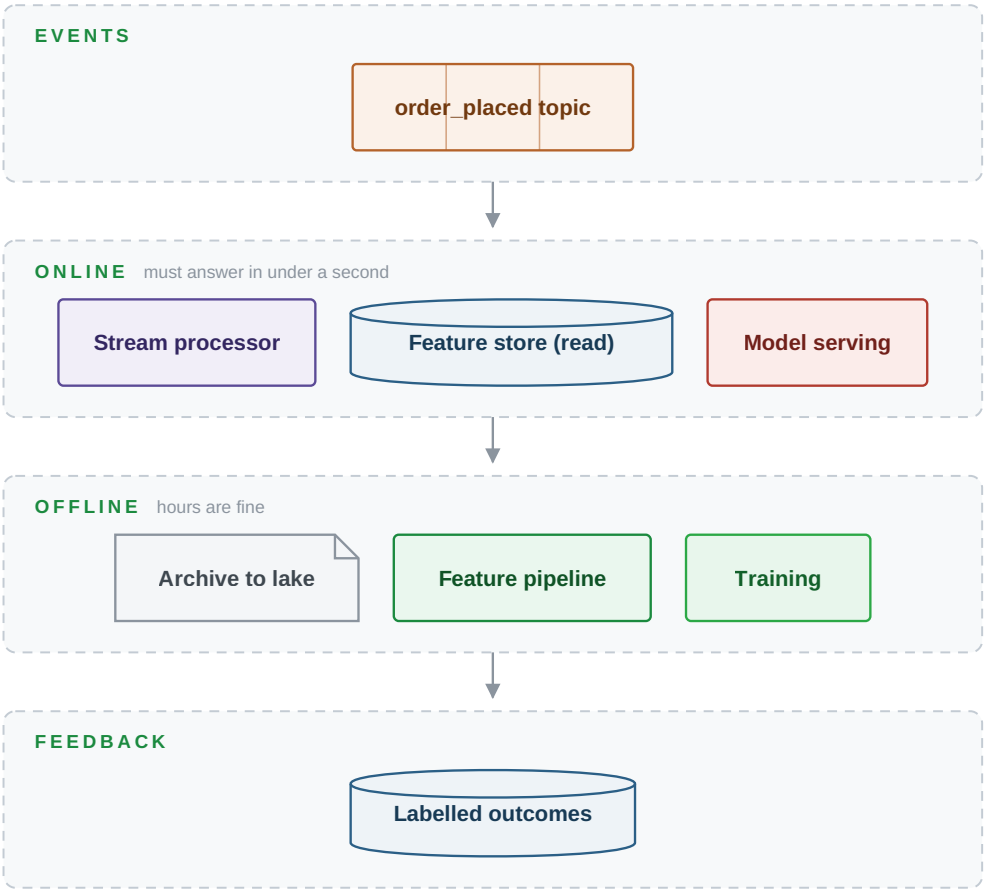


FIGURE 14.3 Design 2. The online path answers in milliseconds from pre-computed features; the offline path computes those features and trains the model. Both read the same event log.

The hard part is not latency, it is features. A useful fraud feature is "orders by this customer in the last ten minutes", which the online path cannot compute by scanning history in 200 ms. So features are precomputed and kept in a low-latency store, and the offline pipeline maintains them.

Which is exactly Chapter 12's training/serving skew problem. If the offline feature and the online feature are computed by different code, the model degrades silently. Naming that unprompted marks you out.

Data quality on a streaming path is a question interviewers ask and candidates skip. You cannot run last night's reconciliation query against a stream, so the checks change shape: alert on consumer lag rather than on freshness, monitor the rate of events failing schema

validation, watch the dead-letter queue depth, and — the one that matters most here — reconcile the feature values used online against the same features recomputed in batch. That last check is what catches training/serving skew before the model degrades.

Trade-offs. At-least-once delivery with an idempotent scorer, rather than exactly-once. Ordering per customer matters, so key by customer id, and accept the hot-key risk from Chapter 11.

Design 3 — A platform serving both

"One company, both of the above, one team."

The insight being tested is whether you build two disconnected stacks or one log with two consumers.

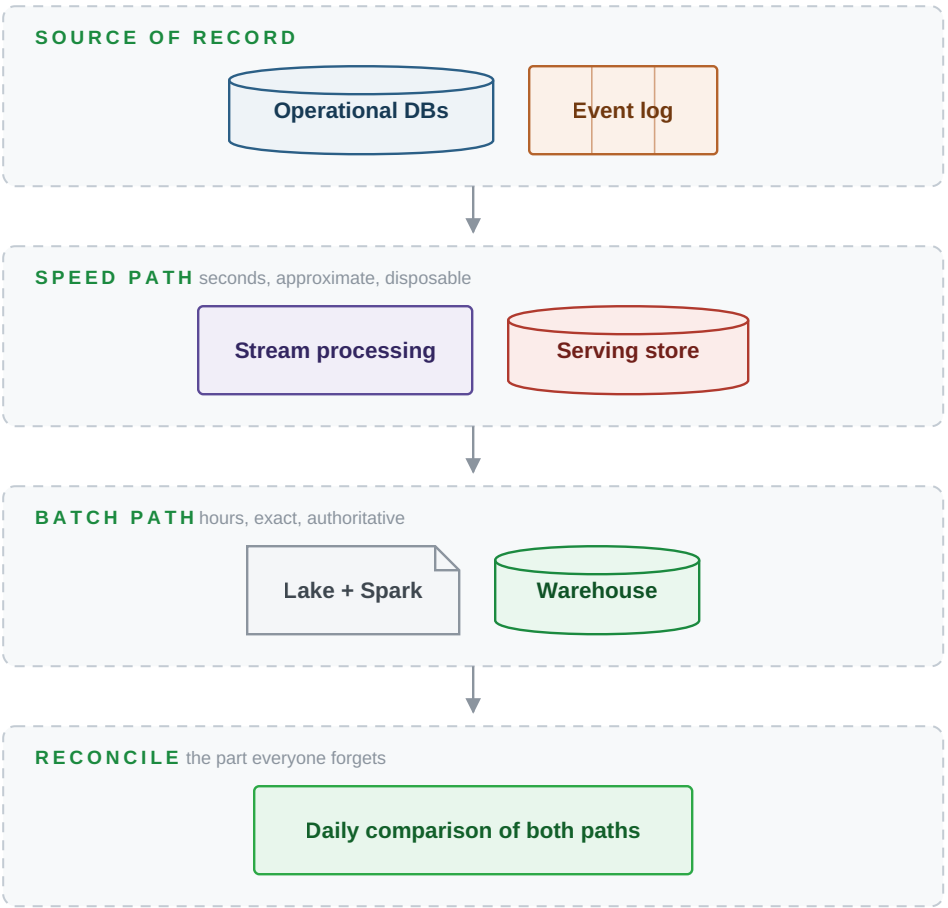


FIGURE 14.4 Design 3. One event log, two paths. The batch path is the source of truth for reported numbers; the streaming path is for decisions that cannot wait.

The answer that separates candidates: the two paths *will* disagree intraday, and that must be a documented property rather than a discovered bug. Name which one is authoritative — the batch path, for anything reported or audited — and reconcile them daily as a quality check. A candidate who says "and we compare them every morning, because if they drift, one of them is wrong and I want to know which" is describing a system they have thought about operating.

WHAT IS ACTUALLY BEING SCORED

Not the diagram. Interviewers are checking whether you asked about consumers and freshness before choosing anything, whether you sized the problem, whether you named a trade-off you rejected, and whether you know what breaks. A modest design with those four things beats an elaborate one without them, every time.

14.3 The loop

A data engineering loop at an Indian product company is typically four to six rounds.

Round	What is tested	Where it is in this book
Screening	Can you describe what you have built	Chapter 1
SQL	Joins, windows, grain, the interview patterns	Chapter 3
Python or coding	File handling, APIs, error handling, occasionally DSA	Chapter 4
Data modeling	Grain, star schema, SCD	Chapter 5
Big data / Spark	Partitions, shuffle, skew, plans	Chapters 6 and 9
System design	The framework in 14.1 and the worked designs in 14.2	This chapter
Behavioural	Incidents, trade-offs, working with others	Chapter 12

Two observations about weighting. SQL appears in almost every loop and is the most common reason for rejection, so it repays preparation more than anything else. And candidates consistently over-prepare on algorithm puzzles and under-prepare on grain, idempotency and failure, which is where the real differentiation happens.

14.4 Talking about your work

Four answers you should be able to give fluently. Practise them out loud and timed; an answer that reads well in your head collapses under a stranger's gaze.

"Tell me about your pipeline." Ninety seconds. Source, ingestion method, storage, transformation, model, orchestration, checks, and who consumes it. Chapter 1 is the template . End with the guarantee you provide, not the tools you used.

"Tell me about something that went wrong." Situation, what you saw first, what you checked, what it turned out to be, what you fixed, and what exists now that did not before. The last part is the whole answer; without it the story is just a story.

"Why did you build it that way?" Name the alternative and why you rejected it. "I used micro-batch rather than streaming because the freshness requirement was hourly." Engineers who cannot name a rejected alternative have not made a decision, they have followed a tutorial.

"What would you do differently?" Answer honestly and specifically. A candidate with no critique of their own work has either not used it in anger or is not being straight with you.

> **THE REFRAME THAT CHANGES INTERVIEWS**
>
> Stop describing tools and start describing guarantees. "I built a Spark pipeline" says nothing. "I own a table that is complete for day D by 06:00, at order-line grain, reconciled to source within 0.1%, with late data corrected inside 24 hours" tells an interviewer you have done the job. Every technical choice then follows from that sentence, and you can defend all of them.

14.5 Thirty days of revision

This assumes you have already read the book. It is a revision plan, not a first pass.

Days	Focus
1–2	Lecture 0 and Chapters 1–2. Terminal fluency, the four resources
3–8	Chapter 3. All eight SQL patterns, written from memory, twice
9–11	Chapter 4. Rebuild the ingestion script from scratch

Days	Focus
12–15	Chapter 5. Design three star schemas; implement SCD2 yourself
16–18	Chapters 6–7. Reproduce the storage measurements and the idempotency failure
19–23	Chapter 9. Read ten execution plans; build and fix a skewed job
24–25	Chapters 10–11. Build the DAG; run Kafka in Docker
26–27	Chapter 12. Write checks and a contract for your own project
28	Chapter 14. Three system designs out loud, timed
29	Polish the project README and the four answers in 14.4
30	Mock interview with someone who will be harsh

Summary

- One project you built and broke on purpose beats five tutorial projects.
- Make the source awkward, run it on a schedule, model the output, validate before publishing, and document what breaks.
- In system design, clarify and estimate before choosing anything, then state your trade-offs out loud.
- SQL is the highest-frequency round and the most common rejection.
- Describe guarantees, not tools.
- Practise the four answers out loud and timed.

> **PRACTICE**
>
> 1. Do three system designs out loud, timed at twenty-five minutes: e-commerce analytics, real-time fraud, ride-hailing.
> 2. Record yourself answering “tell me about your pipeline” in ninety seconds. Watch it back. It will be worse than you think, and better the third time.
> 3. Find someone who will tell you your answer was weak, and let them.

SOURCES AND VERIFICATION

The measurements referenced in the sample README — the CSV to Parquet reduction and the two-million-row scale — are the figures produced in Chapter 6 of this book. The project structure, system design framework and round table are the author’s recommendations rather than measured results, and the round structure in 14.3 is presented as a typical pattern rather than a survey.

Glossary

Every term this book relies on, in the plainest words that are still accurate. Where a term has a chapter that explains it properly, the chapter is named.

ACID — Four promises a database makes about a transaction: it happens completely or not at all, it never leaves broken data behind, concurrent work does not interfere, and once confirmed it survives a crash. *Ch 4.*

Adaptive Query Execution (AQE) — Spark watching its own job while it runs and changing the plan when reality differs from the estimate. Mainly used to split oversized partitions caused by skew. *Ch 9.*

Additive measure — A number you can safely add up across anything. Revenue is additive; a percentage is not. *Ch 5.*

Anti-join — Finding rows in one table that have no match in another. “Which orders have no payment?” *Ch 3.*

API — A defined way for one program to ask another for something, so neither has to know how the other works inside. *Ch 2.*

Avro — A row-based file format that stores its own schema. Good for writing and streaming; Parquet is better for reading. *Ch 6.*

Backfill — Re-processing history, usually because you fixed a bug or added a column. Done one partition at a time, through the same code as the daily run. *Ch 8.*

Batch processing — Collecting data over a period, then processing it in one go. Cheaper and simpler than streaming, and the right default. *Ch 1.*

Bronze / silver / gold — Three layers of a data lake. Bronze is raw and untouched, silver is cleaned and typed, gold is modelled and ready to query. Having three lets you find where a wrong number went wrong. *Ch 6.*

Broadcast join — Sending a small table to every machine so the large table never has to move. The single most valuable Spark optimisation. *Ch 9.*

Cache — Keeping something you will need again close by instead of recomputing it. In Spark, keeping a DataFrame in memory between steps. *Ch 9.*

CDC (change data capture) — Reading a database's own internal change log to see every insert, update and delete, instead of repeatedly querying its tables. Barely touches the source, and it catches deletes. *Ch 8.*

Cloud — A data centre you rent by the hour, with an API in front of it. You pay for time, storage and compute are separate, and nothing is permitted until you say so. *Ch 7.*

Coalesce — Reducing the number of Spark partitions without moving data across the network. Used to avoid writing thousands of small files. *Ch 9.*

Columnar storage — Keeping all the values of one column together on disk, so a query that needs two of forty columns reads only two. The reason analytical systems are fast. *Ch 1, 6.*

Conformed dimension — A dimension table shared identically by several fact tables, so "customer tier" means the same thing everywhere. *Ch 5.*

Consumer group — A set of Kafka consumers sharing the work of a topic. Each partition goes to exactly one of them, so partition count caps your parallelism. *Ch 11.*

Consumer lag — How far behind a Kafka consumer is. The single most important number in a streaming system, because if lag exceeds retention, data is deleted before it is read. *Ch 11.*

CTE (common table expression) — A named intermediate result inside a query, written with `WITH`. Makes long SQL readable, which is how you find bugs in it. *Ch 3.*

DAG (directed acyclic graph) — Work with arrows showing what depends on what, and no loops. Both Airflow pipelines and Spark jobs are DAGs. *Ch 9, 10.*

Data contract — A written promise from whoever produces data: this is the schema, this is how fresh it will be, this is who to contact, and this is how much notice you get before it changes. *Ch 12.*

Data lake — Files, usually Parquet, on cheap object storage, queried by whatever engine you point at them. Cheap and open, with none of a database's guarantees until you add a table format. *Ch 6.*

Data warehouse — A system that loads data into its own managed storage and gives you strong guarantees, fast queries and governance, at a higher price per terabyte. *Ch 6.*

dbt — A tool that manages the SQL transformations inside your warehouse: dependencies, tests, documentation and version control. It does not move or process data. *Ch 8.*

Dead-letter queue — A side topic where messages that cannot be processed are parked with their error, so one bad message does not block a partition forever. *Ch 11.*

Degenerate dimension — A dimension attribute with nowhere else to live, kept as a plain column on the fact table. An order id is the usual example. *Ch 5.*

Delta Lake / Apache Iceberg — Table formats: a transaction log kept alongside Parquet files that records which files make up the table right now. Everything else — ACID, time travel, `MERGE`, compaction — follows from that. *Ch 6.*

Denormalisation — Deliberately repeating data so queries need fewer joins. Right for analytics, wrong for applications. *Ch 5.*

Dimension table — The context you filter and group by: customers, products, dates. Short, wide, changes slowly. *Ch 5.*

Driver — In Spark, the process that plans the job and schedules the work. It does not process distributed data, but it does run every line of your program outside a transformation and receives results from actions like `collect()`, `take()` and `toPandas()`. *Ch 9.*

Egress — The charge for reading data *out of* a cloud provider's network. Storage is cheap; getting data out is not. *Ch 7.*

ELT — Extract, load, then transform inside the destination. Won over ETL because storage became cheap and reprocessing matters. *Ch 8.*

Event time vs processing time — When something happened, versus when your system saw it. They differ, sometimes by hours, and aggregating by the wrong one is why hourly totals never reconcile. *Ch 11.*

Exchange — The word in a Spark execution plan that means a shuffle. What you look for in every plan you read. *Ch 9.*

Executor — In Spark, a worker process that actually runs tasks and holds cached data. *Ch 9.*

Exit code — The number a program returns when it finishes. Zero means success. Every scheduler on earth decides whether your job worked by reading it. *Lecture 0.*

Fact table — Measurements of events: orders, payments, clicks. Long, narrow, grows constantly, mostly numbers and keys. *Ch 5.*

Fan-out — When a join matches one row against several and silently multiplies your numbers. The most common serious defect in analytical SQL. *Ch 3.*

Freshness check — Asking whether a table actually contains data for the period it should. The most valuable single data quality check. *Ch 12.*

Gaps and islands — A SQL pattern for finding runs of consecutive values, such as days a customer ordered without a break. *Ch 3.*

Grain — What one row of a table means. The most important idea in data modeling, and the one most often left unwritten. *Ch 5.*

IAM (identity and access management) — Who is allowed to do what to which resource. Deny by default, everywhere. *Ch 7.*

Idempotent — Running it twice leaves the same result as running it once. The property that makes retries, backfills and late data safe rather than dangerous. *Ch 8.*

Incremental load — Reading only what changed since last time, rather than everything. Necessary at scale, and the source of every hard problem in Chapter 8.

Index — A structure that lets a database find a small number of rows quickly. Stops helping when a query deliberately reads most of the table. *Ch 1, 4.*

Late-arriving data — Records that reach you after your pipeline has already moved past their timestamp. Invisible to a naive watermark, and recovered with a look-back window. *Ch 8.*

Lakehouse — A data lake with a table format on top, so you keep the lake's cost and openness and get back the warehouse's guarantees. *Ch 6.*

Lazy evaluation — Spark building up a plan and doing nothing until you ask for a result. What lets it optimise across the whole chain. *Ch 9.*

Logical date — In Airflow, the date a run *represents*, not the date it executes. A run for the 17th retried on the 19th still processes the 17th. *Ch 10.*

MERGE / upsert — One statement that updates a row if the key exists and inserts it if it does not. How you make a load idempotent. *Ch 8.*

Medallion architecture — The bronze, silver and gold layering, under its marketing name. *Ch 6.*

Narrow transformation — Spark work where each output partition comes from exactly one input partition, so nothing moves across the network. *Ch 9.*

Normalisation — Storing every fact exactly once, so an update touches one row. Right for applications, awkward for analytics. *Ch 5.*

NULL — Unknown. Not zero, not empty. Any comparison with it is neither true nor false, which is why `NOT IN` against a nullable list silently returns nothing. *Ch 3.*

Object storage — S3, ADLS, GCS. Looks like a filesystem and is not: no real directories, objects cannot be edited in place, and listing is a query. *Ch 6, 7.*

OLTP vs OLAP — Systems that *run* the business versus systems that *analyse* it. They differ physically, in how bytes are laid out on disk, not just by convention. *Ch 1.*

Orchestration — Running jobs when their inputs are ready, rather than at a fixed time. The thing cron cannot do. *Ch 10.*

Parquet — A columnar file format that stores its own schema and per-column statistics. The default for a data lake. *Ch 6.*

Partition (storage) — Splitting a dataset into directories by a column's value, so a filter can skip whole directories unopened. *Ch 6.*

Partition (Spark) — A slice of data that one task processes. Partition count is how much parallelism is available. *Ch 9.*

Predicate pushdown — Sending your filter down to the file reader so it skips data without decompressing it. *Ch 6.*

Quarantine — Setting bad rows aside with a recorded reason instead of dropping them silently or failing the whole job. *Ch 4, 12.*

Reconciliation — Checking that a total in your table matches the same total at the source, within a stated tolerance. The strongest quality check available and the least used. *Ch 12.*

Referential integrity — Every key in a fact table existing in its dimension. Orphans mean a broken or late load. *Ch 12.*

Repartition — Changing Spark's partition count with a full shuffle. Use it to fix skew; use `coalesce` to reduce file counts. *Ch 9.*

Retention — How long a streaming system keeps messages. It is a deadline for your consumers, not a storage setting. *Ch 11.*

SCD (slowly changing dimension) — What to do when a dimension attribute changes. Type 1 overwrites and loses history; Type 2 adds a new row and keeps it. The choice decides whether last month's report changes. *Ch 5.*

Salting — Adding a random suffix to a hot key so its rows spread across partitions, then aggregating twice. The manual fix for skew. *Ch 9.*

Serverless — You supply code, the provider supplies the machine for the length of one request. Excellent for short bursty work, poor for long or steady work. *Ch 7.*

Sessionization — Grouping a user's events into sessions by breaking on a gap of inactivity. *Ch 3.*

Shuffle — Moving rows across machines so that everything with the same key ends up together. Required by joins and grouping, and the most expensive thing Spark does. *Ch 9.*

Skew — One key holding most of the rows, so one task does most of the work while the rest finish early. *Ch 9.*

SLA / SLO — A promise to your consumers about freshness and correctness, and the stricter internal target you run against. *Ch 12.*

Small files problem — Thousands of tiny files being far slower to read than one large file of the same size, because seeks cost more than bytes. *Ch 2, 6.*

Star schema — One fact table with its dimensions arranged around it, each exactly one join away. The constraint is the value. *Ch 5.*

Streaming — Processing each event as it arrives. Justified when a decision genuinely changes with seconds of freshness, and expensive when it does not. *Ch 1, 11.*

Surrogate key — A meaningless integer the warehouse generates, so one customer can have several versions over time. *Ch 5.*

Time travel — Reading a table as it was at an earlier version, using a table format's transaction log. *Ch 6.*

Topic / partition / offset — A Kafka stream, the ordered slices it is split into, and a message's position within a slice. Ordering is guaranteed inside a partition and nowhere else. *Ch 11.*

Transaction log — The list of which files make up a table right now. The one idea that turns a folder of Parquet into a real table. *Ch 6.*

Watermark (batch) — A stored marker of how far the last run got, so the next run knows what is new. *Ch 8.*

Watermark (streaming) — A statement that the system no longer expects events older than a given point, so a window can be closed. *Ch 11.*

Wide transformation — Spark work that needs data from many input partitions, and therefore shuffles. *Ch 9.*

Window function — A calculation across a set of rows that returns a value for every row without collapsing them. Running totals, rankings, comparisons with the previous row. *Ch 3.*

XCom — Airflow's mechanism for passing a value between tasks. Carry pointers, never data. *Ch 10.*

END MATTER

The Field Kit

Every number in this book was produced by code that was actually run. That code is yours.

- **The verification scripts** — the ShopKart dataset and every script behind every measured result: the 44.9% join fan-out, the SCD Type 2 comparison that moved ₹4,749 between two cities, the storage benchmarks, the load that tripled revenue, and the Spark execution plans. Run them, change the numbers, and check the book.
- **The portfolio project** — the mock API and all five stages from Chapter 13, so you can build the project offline with nothing but `requests` and `duckdb`.
- **The interview question bank** — Data Engineering and Data Analytics questions, organised by topic and by level.

Free at **prepnplaced.com/data-engineering-book**.
The Field Kit is on GitHub:
`github.com/analyticsdurgesh/data-engineering-book-field-kit`

Further Reading

This book is a path into the field, not the last word on any part of it. Six places to go next, each better than this book on its own subject.

Martin Kleppmann, *Designing Data-Intensive Applications*. The best book written about distributed data systems. Read it after this one, when the vocabulary is familiar and you want the depth underneath it.

Joe Reis and Matt Housley, *Fundamentals of Data Engineering*. The lifecycle view, vendor-neutral and thorough. Strong on the parts of the discipline this book treats briefly, particularly governance and architecture at organisational scale.

Ralph Kimball and Margy Ross, *The Data Warehouse Toolkit*. The standard reference for dimensional modeling. Chapter 5 of this book is an introduction to territory this covers exhaustively.

The official documentation for PostgreSQL, Apache Spark, Apache Kafka, Apache Airflow, Delta Lake and Apache Iceberg. Unfashionable advice, and still the highest-quality free technical writing available. The PostgreSQL documentation on query planning in particular is better than most paid courses.

Engineering blogs from companies at scale. Flipkart, Swiggy, Zomato, Uber, Netflix and Airbnb publish detailed accounts of their data platforms. Chapter 1 cites two of them. Reading how a real platform handles a real spike teaches things no tutorial can.

Your own production incidents. The most valuable reading available to you, and the only source on this list nobody else has.

About the Author

Durgesh Yadav is a Senior Data Engineer, a data engineering educator and mentor, and the founder of PrepNPlaced.

He teaches data engineering and data analytics, and mentors engineers moving into the field — many of them from non-technical backgrounds, which is the reader this book was written for.

PrepNPlaced is at **prepnplaced.com**.

Index

A

Adaptive Query Execution 160, 164–165, 231

additive 97–98, 104–106, 145, 231

anti-join 72, 231

at-least-once 181, 184, 222, 227

Avro 110–111, 118, 120, 231

awk 18–19, 24, 76, 200, 203, 219, 230, 235

B

backfill 87, 140, 149–151, 168, 170, 174–175, 191, 197, 231, 234

backoff 48–49, 167–168, 172–174, 212–213

broadcast 51, 155, 158–160, 162–165, 231

C

cache 44, 155, 161–162, 164, 232, 234

catchup 167, 170, 174–175, 212

CDC 34, 143–144, 150, 152, 224, 232

change data capture 143, 232

chunksize 85–86

collect 32, 51, 77, 79, 155, 157, 162, 164–165, 231, 233

columnar 22, 73–74, 91, 122, 232, 236

compression 22, 91, 111, 119, 127, 132–133, 214

conformed dimension 101, 106, 224, 232

consumer group 178, 184, 186, 232

consumer lag 180, 184–185, 226, 232

context manager 79

credentials 20–21, 47–51, 126, 134, 171, 174, 193–194

D

DAG 147–148, 154–155, 166–167, 169–175, 200, 202, 212–214, 218, 230, 232

data contract 189, 196, 198, 232

data lake 45, 110, 114–117, 122, 141, 231, 233, 235–236

data skew 160, 164, 179

dead-letter 181–182, 184–185, 216, 222, 227, 233

deduplicat 66–68, 91, 118, 141, 143, 150, 169, 211, 217

degenerate dimension 101, 106, 108, 207, 233

Delta Lake 109, 116–117, 120, 137, 142, 152, 233, 240

denormalis 31, 233

DENSE_RANK 66, 68, 75

DictReader 79, 82, 88, 91

dimension table 97, 108, 232–233

driver 86, 155, 162–163, 165, 233

E

environment variable 20, 24–25, 50–51

event time 182, 184, 234

exactly-once 184, 227

Exchange 3, 100, 110, 158–159, 163, 165, 234

executor 42, 45, 74, 155–159, 161–164, 167, 234

exit code 19, 24, 45–46, 77, 81, 89, 91, 151, 172, 188, 209, 211, 219, 234

F

fact table 37, 94, 97–101, 103–108, 159, 218, 220, 233–234, 236–237

fan-out 3, 63, 65, 73, 75, 94, 178, 187–188, 234, 239

freshness 32–33, 37–39, 173, 184, 188–190, 196, 198, 222, 225–226, 228–229, 234, 237
full load 39, 139, 150

G

gaps and islands 70, 234
grep 17–19, 24

H

HTTP 47–48, 115, 121, 203–204, 213, 220

I

Iceberg 116–117, 119–120, 233, 240
incremental load 39, 139, 235

L

LAG 18, 70–72, 75, 115, 144, 161, 164, 180–181, 184–185, 216, 218, 226, 232
lakehouse 32, 40, 115–116, 122, 138, 151, 235
late-arriving 28, 39, 105, 144, 152, 188, 213–214, 235
lazy evaluation 156, 165, 235
logging 83, 87–88, 91, 123, 204
logical date 167–169, 172, 174–175, 212, 235

M

medallion 118, 138, 197, 235
MERGE 87, 93, 103, 116–117, 119, 137, 141–143, 148–152, 156, 159–160, 163, 165, 193–194, 233, 235

N

narrow transformation 157, 163, 235
natural key 100–101, 105, 107
non-additive 98, 105
normalization 96
NOT EXISTS 61, 73–74, 88, 207

O

object storage 35, 44–45, 113–116, 118, 120–122, 125, 128–129, 133–134, 138, 214, 222, 235
offset 178–182, 214, 237
OLAP 30–31, 38, 109, 236
OLTP 30–31, 38, 97, 109, 236
operator 167–168, 171, 175, 212
OutOfMemory 43

P

pagination 219
pandas 17, 76, 84–87, 90–91, 154–155, 162, 233
partition pruning 112, 127
partitioning 112–114, 117, 119, 125, 133–134, 141, 158, 178
permissions 15, 20, 46, 48, 125, 129, 144, 193
predicate pushdown 110, 236
processing time 182, 184, 234

Q

quarantine 38, 82, 89, 91, 118, 125, 189, 196, 198, 201–203, 206–207, 211–214, 218–220, 236

R

RANK 65–68, 75, 238
reconciliation 38, 73, 107, 173, 188–189, 196, 198, 209, 222, 224, 226, 236
referential integrity 38, 147, 188, 196, 198, 236
replication 115, 180, 186
retry 48–49, 51, 76, 166–169, 172–175, 182, 202–205, 212, 220
row-oriented 31, 109
ROW_NUMBER 66–67, 71, 74–75, 141, 143
running total 67–68, 85, 238

S

salting 5, 161, 164–165, 236

SCD Type 2 93, 103, 106, 108, 195, 214, 218, 224
scheduler 19, 23–24, 46, 81, 89, 167, 169, 171–172, 174–175, 209, 219, 234
schema evolution 145, 181
secret 21, 24, 50–51, 123, 125–126, 129, 134, 193
semi-additive 98, 104–105
sensor 167, 169, 174–175
sessionization 71, 75, 237
shuffle 4, 45, 50–51, 86, 91, 127, 154, 156–160, 162–165, 228, 234, 236–238
silver 34–35, 60–61, 118–119, 124–125, 138, 146, 190–191, 197, 201–203, 206, 213–214, 222, 224, 231, 235
SLA 47, 70–72, 75, 114, 117, 122, 133, 135, 171, 189, 192–193, 197–198, 222, 225, 234, 237
slowly changing dimension 28, 101, 108
small files 25, 43–44, 52, 113–114, 116–120, 127, 162, 232, 237
snowflake schema 100
sort-merge 159
star schema 98–99, 105, 107–108, 207, 213, 222, 224, 228, 230, 237
status code 48, 51, 202
surrogate key 100–101, 103, 106, 207, 214, 237

T

time travel 116–117, 119–120, 237
topic 30, 177–178, 181–182, 184–186, 216–217, 222, 226, 232–233, 237, 239
transaction log 116–117, 119, 151, 237

U

upsert 87, 116–117, 141–142, 150, 169, 235

V

virtual environment 87

W

watermark 28, 139–140, 143–145, 148, 150–152, 182–184, 186, 220, 235, 237
wide transformation 4, 157, 238
window function 65, 68–70, 75, 238

X

XCom 170, 174–175, 238